

R85

MBY

Özdevinirler (Otomatlar)

**Kuramı ve
Biçimsel Diller**

Prof.Dr. Ünal Yarımağan

İkinci Basım

Ankara, Şubat 2011

Kitabın sonunda **Seçilmiş Kaynaklar** ile **Terimler Sözlüğü**'ne yer verilmiştir. Terimler sözlüğünde, kitapta kullanılan başlıca terimlerin hem İngilizceden-Türkçeye, hem de Türkçeden-İngilizceye karşılıkları yer almaktadır. Terimlerin Türkçe karşılıkları için Bilişim Terimleri Sözlüğü'nden [TBD-1996] olabildiğince yararlanılmaya çalışılmıştır. Terimler Sözlüğü'nde, TBD Bilişim Terimleri Sözlüğünden alınan belirli sayıda terim yanında, bu sözlükte yer almayan ya da bu sözlükteki karşılığını benimsemediğimiz terimlerin kitaptaki karşılıklarına da yer verilmiştir.

Ünal Yarımağan

Eylül 2003

Kitabın ikinci basımında içerik olarak hiçbir değişiklik yapılmadı. Yalnız belirlenen yazım yanlışları düzeltildi ve bölüm sonlarındaki sorulara eklemeler yapıldı. Yeni eklemelerle birlikte bölüm sonu soruları ve bunların yanıtları, sayfa sayısı olarak kitabın yarısına ulaştı. Bu nedenle yeni eklenen soruların tümünün değil, yalnız bir kısmının yanıtına yer verilmekle yetinildi. "Kitabın en özgün yanının soru ve yanıtlar" olduğu yönündeki değerlendirmemin ise, yeni eklenen sorularla birlikte daha da geçerlik kazandığını düşünüyorum.

Ünal Yarımağan

Şubat 2011

İçindekiler

1. Sonlu Özdevinirler	1
1.1. Sonlu Özdevinir (FA) Modeli	1
1.1.1. Deterministik Sonlu Özdevinir (DFA) Modeli	1
1.1.2. Deterministik Olmayan Sonlu Özdevinir (NFA) Modeli	5
1.1.3. <i>Lambda</i> (λ) Geçiş	8
1.1.4. Deterministik ve Deterministik Olmayan Sonlu Özdevinir Modellerinin <u>Denkliği</u>	13
1.1.5. İki Yönlü Sonlu Özdevinir Modeli	15
1.2. Çıkış Üreten Özdevinirler	19
1.3. Sonlu Özdevinirlerin İndirgenmesi	28
1.3.1. Ardıl, Öncel, Denk ve Ayırdedilebilir Durum Tanımları	29
1.3.2. İndirgeme Yöntemi	32
1.3.2.1. Mealy Makinelerinin İndirgenmesi	33
1.3.2.2. Moore Makinelerinin İndirgenmesi	35
1.3.2.3. Deterministik Sonlu Özdevinirlerin (DFA) İndirgenmesi	37
Sorular	40

2. Düzgün Kümeler ve Düzgün Deyimler	49
2.1. Düzgün Kümeler	49
2.2. Düzgün Deyimler	50
2.2.1. Düzgün Deyimlere Karşı Gelen Sonlu Özdevinirlerin Bulunması	51
2.2.2. Sonlu Özdevinirlerin Tanıdığı Kümelerin Birer Düzgün Deyim Olarak Bulunması	57
Sorular	60

3. Dilbilgisi ve Diller	69
3.1. Dilbilgisi ve Dilin Biçimsel Tanımı	70
3.2. Dilbilgisi ve Dillerin Sınıflandırılması	71
3.2.1 Tür-0 Dilbilgisi ve Dil	71
3.2.2. Tür-1 Dilbilgisi ve Dil	72
3.2.3. Tür-2 Dilbilgisi ve Dil	74
3.2.4 Tür-3 Dilbilgisi ve Dil	75
3.2.5. Sağ-Doğrusal ve Sol-Doğrusal Dilbilgisi	75
3.3. Düzgün Dil – Sonlu Özdevinir İlişkisi	78
3.3.1. Düzgün Dilbilgisinin Türettiği Dili Tanıyan Sonlu Özdevinirin Bulunması	78
3.3.2. Sonlu Özdevinirin Tanıdığı Dili Türeten Düzgün Dilbilgisinin Bulunması	84
3.4. Düzgün Dillerin Kimi Özellikleri	85
Sorular	87
4. Bağlamdan-Bağımsız Dilbilgisi ve Diller	99
4.1. Bağlamdan-Bağımsız Dilbilgisi	99
4.2. Türetme Ağacı	102
4.3. Dilbilgisinin Yalınlaştırılması	106
4.3.1. Özyineli Kural, Özyineli Değişken	106
4.3.2. Yok Edilebilir Değişken	106
4.3.3. Birim Türetme Kuralları	108
4.3.4. Yararsız Değişken, Simge ve Kurallar	110
4.4. Normal Biçimler	114
4.4.1. Chomsky Normal Biçimi	114
4.4.2. Greibach Normal Biçimi	116
Sorular	123
5. Yığıtlı Özdevinirler (Pushdown Automata)	135
5.1. Yığıtlı Özdevinirin Temel Modeli	135
5.2. Bağlamdan-Bağımsız Dilbilgisi (CFG)- Yığıtlı Özdevinir (PDA) İlişkisi	147
5.2.1. Verilen bir CFG'nin Eşdeğeri PDA'nın Bulunması	147
5.2.2. Verilen bir PDA'nın Eşdeğeri CFG'nin Bulunması	153
Sorular	158

6. Turing Makineleri	167
6.1. Turing Makinelerinin Temel Modeli	167
6.1.1. Soyut Makine Görünümü	168
6.1.2. Anlık Tanımlar (<i>Instantaneous Descriptions</i>)	169
6.1.3. Turing Makinesinin Tanıdığı Dil	170
6.2. Turing Makinesi Örnekleri	171
6.3. Turing Makinesi Modelinde Değişiklikler	180
6.4. Chomsky Sıradüzeni	181
Sorular	183
7. Ayırıştırma İşlemleri (Parsing)	199
7.1. Ayırıştırma Türleri	200
7.1.1. Yukarıdan-Aşağıya Ayırıştırma (<i>Top-down Parsing</i>)	200
7.1.2. Aşağıdan-Yukarıya Ayırıştırma (<i>Bottom-up Parsing</i>)	202
7.2. Deterministik Ayırıştırma	203
7.2.1. LL(k) Dilbilgileri	203
7.2.2. LL(k) Ayırıştırma	205
7.2.2.1. LL(k) Ayırtırmada Kullanılacak PDA Modeli	205
7.2.2.2. LL(k) Ayırıştırma Çizelgesinin Oluşturulması	206
7.2.3. LR(k) Dilbilgileri	209
7.2.4. LR(k) Ayırıştırma	210
7.2.4.1. LR(k) Ayırtırmada Kullanılacak PDA Modeli	210
7.2.4.2. İşaretli Kurallar	213
7.2.4.3. LR(1) Ayırıştırıcı Geçiş Çizeneğinin Oluşturulması	214
7.2.4.4. LR(1) Ayırıştırma Çizelgesinin Oluşturulması	218
7.2.4.5. LR(1) Ayırıştırma Sürecinin Yürütülmesi	220
Sorular	222

Seçilmiş Soruların Yanıtları	229
Bölüm 1	229
Bölüm 2	239
Bölüm 3	257
Bölüm 4	278
Bölüm 5	289
Bölüm 6	308
Bölüm 7	324
Seçilmiş Kaynaklar	337
Terimler Sözlüğü	339

Bölüm 1

Sonlu Özdevinirler

1.1. Sonlu Özdevinir (FA) Modeli

Sonlu özdevinir (*finite automata* : FA) modeli, kesikli giriş ve çıkışları olan matematiksel bir modeldir. Bu model birçok donanım ve yazılım sisteminin modellenmesinde kullanılır. Metin düzenleyici (*text editor*) ve derleyicilerin belirli kesimleri ile zamanuyumlu dizsel devreler (*synchronous sequential circuits*) sonlu özdevinir modeline uygun yazılım ve donanım sistemlerine örnek olarak verilebilir. Hatta sayısal bilgisayarlar bile bu modele uygun sistemler olarak görülebilir. Dolayısıyla bilgisayar bilimleri ve mühendisliği için sonlu özdevinir modeli, çok kullanılan önemli bir modeldir.

Sonlu özdevinirler için bu kitapta, modelin ingilizce adının baş harflerinden oluşan ve yaygın biçimde kullanılan FA kısa adı kullanılacaktır.

Sonlu özdevinir modelinin birçok türü vardır. Sonlu özdevinirleri öncelikle:

- Sonlu durumlu tanıyıcı (*finite state recognizer*)
- Çıkış üreten özdevinir

modelleri olarak sınıflandırmak mümkündür. Bu sınıfların her birinin de kendi içinde alt sınıfları vardır. Ancak kısaca “sonlu özdevinir” ya da FA denilince, temel model olan “deterministik sonlu durumlu tanıyıcı” modeli anlaşılır.

1.1.1. Deterministik Sonlu Özdevinir (DFA) Modeli

Temel modelde, deterministik sonlu özdevinir bir beşli olarak tanımlanır.

$$DFA = \langle Q, \Sigma, \delta, q_0, F \rangle$$

Q : Sonlu sayıda durum içeren Durumlar Kümesi

- Σ : Sonlu sayıda giriş simgesinden oluşan Giriş Alfabeti
- q_0 : Başlangıç durumu ($q_0 \in Q$)
Başlangıç durumu durumlar kümesinin bir elemanı olduğuna göre Q boş olmayan (içinde en azından q_0 bulunan) bir kümedir.
- F : Uç durumlar kümesi
Durumlar kümesinin bir altkümesidir : $F \subseteq Q$
- δ : Durum geçiş işlevi (*state transition function*)
Durum geçiş işlevi ya da kısaca geçiş işlevi, $(Q \times \Sigma)$ 'dan Q 'ya bir eşleme oluşturur.

Temel modelde geçiş işlevi ile her (durum, giriş simgesi) çiftine bir (durum) eşlenmektedir. Bu durumlardan ilki "şimdiki durum", ikincisi ise "sonraki durum" olarak adlandırılırsa, durum geçiş işlevi ile her (şimdiki durum, giriş simgesi) çiftine bir (sonraki durum) eşlendiği söylenebilir. Matematiksel olarak geçiş işlevi:

$$\delta(q_i, a) = q_j \quad \forall q_i \in Q, a \in \Sigma \Rightarrow q_j \in Q$$

olarak tanımlanır.

Yukarıdaki tanımdan anlaşıldığı gibi, temel FA modeli deterministik bir modeldir. Bu model kısaca DFA (*deterministic finite automata*) olarak adlandırılır. Kısaca FA (*finite automata*) denildiğinde de DFA modeli anlaşılır. Bu modele göre, FA q_0 durumundan başlar ve uygulanan her giriş simgesi ile yeni bir duruma geçer. Her an FA'nın bulunduğu durum kesin olarak bellidir.

Örnek 1.1. $M_{1.1} = \langle Q, \Sigma, \delta, q_0, F \rangle$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_2\}$$

$$\delta : \delta(q_0, 0) = q_0$$

$$\delta(q_0, 1) = q_1$$

$$\delta(q_1, 0) = q_0$$

$$\delta(q_1, 1) = q_2$$

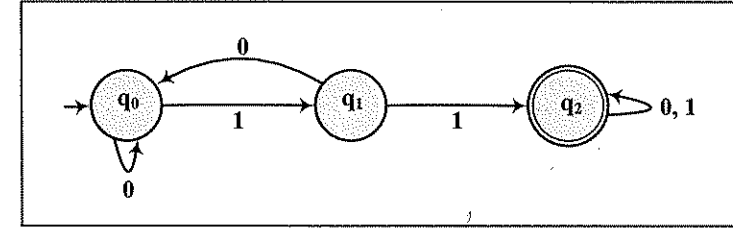
$$\delta(q_2, 0) = q_2$$

$$\delta(q_2, 1) = q_2$$

Geçiş Çizeneği

Örnek 1.1'de görüldüğü gibi, geçiş işlevinin matematiksel tanımı hem uzun, hem de anlaşılabilirliği zor olan bir tanımdır. Bu nedenle, geçiş işlevinin tanımı için genellikle "geçiş çizeneği (*transition diagram*)" olarak adlandırılan bir çizenecek kullanılır.

Yönlü bir çizge olan geçiş çizeneğinde her durum için bir düğüm bulunur. Durum geçişleri ise yönlü yaylar ile gösterilir ve yayların üzerine geçişe neden olan giriş simgesi yazılır. Geçiş çizeneğinde başlangıç durumu " $\rightarrow$ " ile, uç durumlar ise çift çember ile gösterilir. Örnek 1.1'deki FA için oluşturulan geçiş çizeneği Çizim 1.1'de görülmektedir.



Çizim 1.1. $M_{1.1}$ 'in Deterministik Geçiş Çizeneği

DFA'nın Tanıdığı Dizgiler Kümesi

Tanımlanan modele göre, DFA'nın girişine her seferinde bir giriş simgesi uygulanır. Her giriş simgesi DFA'nın bir sonraki duruma geçmesine neden olur. DFA, bulunduğu durumu yeni bir giriş simgesi uygulanana kadar korur. Uygulanan giriş simgelerinin sayısı ne kadar çok olursa olsun, deterministik olduğu için model çalışır. Başlangıçta q_0 durumunda bulunan DFA, belirli sayıda giriş simgesinden oluşan bir dizginin (*string*) uygulanmasından sonra belli bir duruma ulaşır. Uygulanan giriş dizgisini w , bu dizgi uygulandıktan sonra DFA'nın ulaştığı durumu da q_i ile gösterelim. q_0 , w ve q_i arasındaki ilişkiyi, durum geçiş işlevini kullanarak ve bu işlevin anlamını biraz genişleterek aşağıdaki gibi gösterebiliriz:

$$\delta(q_0, w) = q_i$$

q_0 durumundan başlayan ve w giriş dizgisinin uygulanmasından sonra q_i durumuna ulaşan DFA, eğer q_i bir uç durum ise w giriş dizgisini tanıır. Giriş alfabeti $\Sigma = \{I_1, I_2, \dots, I_k\}$ olan bir DFA'ya uygulanabilecek giriş dizgileri kümesi sonsuz bir kümedir. Giriş alfabetindeki simgelerden oluşan sonlu ya da sonsuz uzunluktaki her dizgi bu kümede yer alır. Giriş alfabetindeki simgelerden oluşan dizgilerin bir kısmı DFA tarafından tanınan, diğer bir kısmı ise tanınmayan dizgilerdir. Bu aşamada, bir sonlu özdevinir (M) tarafından tanınan dizgiler kümesini aşağıdaki gibi tanımlamak mümkündür:

$$T(M) = \{w \mid \delta(q_0, w) = q_i \in F\}$$

Buna göre DFA'ya uygulanabilecek dizgiler kümesi, DFA tarafından tanınan ve tanınmayan olmak üzere, iki altkümeye ayrılmaktadır. DFA'nın tanıdığı küme, DFA'yı başlangıç durumundan bir uç duruma götüren dizgilerin kümesidir. Örnek 1.1'de tanımlanan DFA'yı ($M_{1.1}$) incelersek, örneğin **0101** dizgisinin $M_{1.1}$ tarafından tanınmadığını; buna karşılık **0110** dizgisinin $M_{1.1}$ tarafından tanındığını görürüz. Çünkü q_0 durumundan başlayan $M_{1.1}$, **0101** uygulandıktan sonra q_1

durumuna ulaşır ve q_1 bir uç durum değildir. Buna karşılık **0110** dizgisi $M_{1,1}$ 'i başlangıç durumundan q_2 durumuna götürür ve q_2 bir uç durumdur. $M_{1,1}$ 'in tanıdığı dizgiler kümesi sonsuz bir kümedir. Bu kümedeki dizgilerden birkaç örnek aşağıda gösterilmiştir.

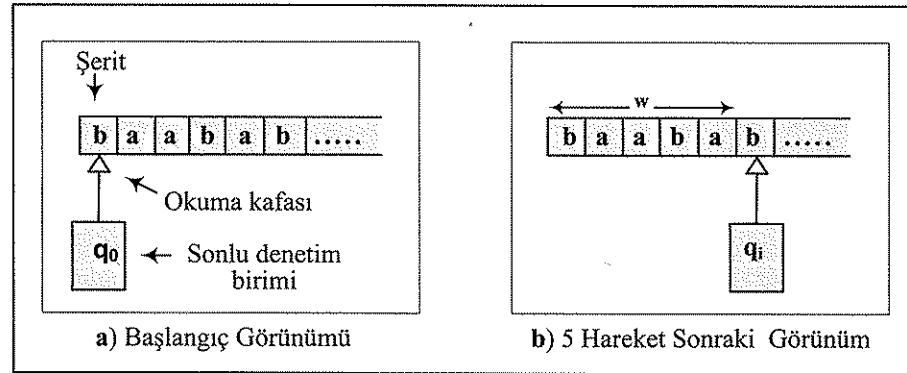
$$T(M_{1,1}) = \{11, 011, 110, 0110, 0110, 01011, \dots\}$$

Çizim 1.1'deki geçiş çizeneği incelendiğinde, bir dizginin $M_{1,1}$ tarafından tanınabilmesi için, dizgide ardarda iki tane **1** bulunmasının gerekli ve yeterli olduğu görülür. Buna göre, $T(M_{1,1})$ sözlü olarak, $\{0, 1\}$ alfabesinde, içinde **11** alt dizgisi bulunan dizgiler kümesi olarak tanımlanabilir.

DFA'nın Şeritli Makine Modeli

Tanımlandığı biçimiyle deterministik sonlu özdevinir modeli matematiksel bir modeldir. Deterministik bir sonlu özdevinir oluşturmak için, bir durumlar kümesi ile bir giriş alfabesi oluşturmak, durumlardan birini başlangıç durumu olarak seçmek, durumların bir kesimini uç (taniyan) durumlar olarak nitelemek ve bir geçiş işlevi tanımlamak yeterlidir. Oluşturulan DFA soyut bir makine olarak görülebilir ve kısaca "makine" olarak adlandırılır. Ancak "makine" adlandırmasının tamamen simgesel olduğu, oluşturulan sonlu özdevinirin makine ile hiç benzerliği olmayan bir şey de olabileceği unutulmamalıdır.

Suyut bir makine olan deterministik sonlu özdevinirin daha kolay anlaşılması için, somut bir şeritli makine modeli düşünülebilir. Bu amaçla, DFA için kullanılan somut makine modeli, bilinen bileşenlerden oluşan ve Çizim 1.2'de görülen bir modeldir.



Çizim 1.2. DFA'nın Şeritli Makine Modeli

Çizim 1.2.a'da görüldüğü gibi DFA'nın makine modeli 3 elemandan oluşmaktadır.

- Hücrelerden oluşan ve her hücresinde bir giriş simgesi bulunan bir miknatıslı şerit. Yalnız okunabilen şeridin sağ ucu sonsuzdur. Başlangıçta şerit üzerinde giriş simgelerinden oluşan bir dizgi kayıtlıdır.
- Bir sonlu denetim birim (SDB). SDB'nin sonlu sayıda durumu vardır. Bu durumlardan biri başlangıç durumudur ve SDB başlangıçta bu durumda bulunur.
- Bir okuma kafası. Şeridin okunması soldan sağa doğru, tek yönlü gerçekleşir. Belirli bir anda okuma kafası şeridin hücrelerinden biri üzerinde bulunur ve üzerinde bulunduğu hücrede kayıtlı simgeyi okuyabilir.

Çizim 1.2'deki modele göre, DFA (makine) aşağıda açıklanan biçimde çalışır:

- Başlangıçta şerit üzerinde bir giriş dizgisi kayıtlıdır ve okuma kafası şeridin ilk (en soldaki) hücresi üzerindedir.
- Makinenin her hareketinde aşağıdaki işlemler yapılır:
 1. Şeritten bir simge okunur.
 2. Okuma kafası bir sağdaki hücreye geçer.
 3. Sonlu denetim birimi bir sonraki duruma geçer.
- Belirli sayıda hareket sonunda okuma kafası belirli bir hücrenin üzerinde, sonlu denetim birimi ise belirli bir durumda (q_i) bulunur. Okuma kafasının üzerinde bulunduğu hücrenin solundaki hücrelerde yer alan dizgiyi w olarak adlandırılır. Eğer q_i bir uç durumsa, makine w 'yi tanır (Çizim 1.2.b).

1.1.2. Deterministik Olmayan Sonlu Özdevinir (NFA) Modeli

Sözlü olarak ya da bir düzgün deyimle (bkz 2.2.) tanımlanan belirli bir dizgiler kümesini taniyan DFA'nın deterministik geçiş çizeneğinin oluşturulması oldukça zor olabilir.

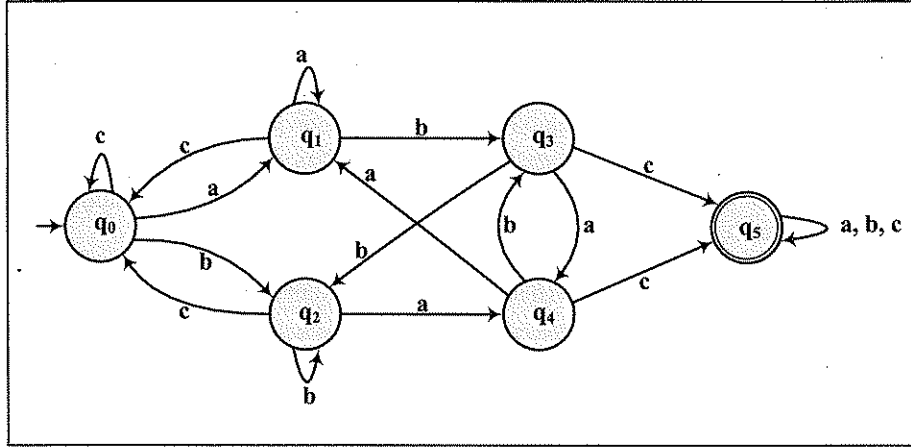
Örnek 1.2. " $\{a, b, c\}$ alfabesinde, içinde **abc** ve **bac** alt dizgilerinden en az biri, en az bir kez bulunan dizgiler kümesi"ni düşünelim ve bu kümeyi tanıyan DFA'yı $M_{1,2}$ olarak adlandırılır.

$M_{1,2}$ 'nin tanıdığı dizgilerden birkaç örnek aşağıda gösterilmiştir.

$$T(M_{1,2}) = \{\underline{abc}, \underline{bac}, c\underline{abcb}, bcca\underline{abcbac}, \dots\}$$

$M_{1,2}$ 'nin deterministik geçiş çizeneği ise Çizim 1.3'de görülmektedir. $M_{1,2}$ 'nin tanıdığı kümenin sözlü tanımı oldukça kolay olmasına karşın, geçiş çizeneğinin oldukça karmaşık olduğu görülmektedir. Üstelik, sözlü tanımdan

hareketle bu çizeneğin elde edilmesini sağlayan sistematik bir yöntem de yoktur.



Çizim 1.3. $M_{1.2}$ 'nin Deterministik Geçiş Çizeneği

Deterministik modelin kullanım gücü nedeniyle, kullanımı daha kolay ve daha esnek bir model olan deterministik olmayan (*non deterministic*) model geliştirilmiştir. Deterministik olmayan sonlu özdevinir (*non deterministic finite automata : NFA*) aşağıdaki gibi tanımlanır:

$$NFA = \langle Q, \Sigma, \delta, q_0, F \rangle$$

Bu tanımda Q , Σ , q_0 ve F 'nin anlamları deterministik model tanımındakilerle aynıdır. Deterministik modelde olduğu gibi deterministik olmayan modelde de Q durumlar kümesini, Σ giriş alfabesini, q_0 başlangıç durumunu, F ise uç durumlar kümesini göstermektedir. İki model arasındaki tek fark geçiş işlevinin tanımındadır. Deterministik modelde geçiş işlevi:

$$(Q \times \Sigma) \rightarrow Q$$

bir eşleme olarak tanımlanırken, deterministik olmayan modelde geçiş işlevi:

$$(Q \times \Sigma) \rightarrow \mathcal{P}(Q)$$

bir eşleme olarak tanımlanır. Buna göre deterministik modelde her durumdan her giriş simgesi ile bir ve yalnız bir duruma geçilirken, deterministik olmayan modelde bir durumdan bir giriş simgesi ile geçilebilecek durum sayısı sıfır, bir ya da birçok olabilmektedir.

Örnek 1.3. $M_{1.3} = \langle Q, \Sigma, \delta, q_0, F \rangle$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_3\}$$

$$\delta : \delta(q_0, 0) = \{q_0, q_1\}$$

$$\delta(q_0, 1) = \{q_0, q_2\}$$

$$\delta(q_1, 0) = \{q_3\}$$

$$\delta(q_1, 1) = \{\} = \Phi$$

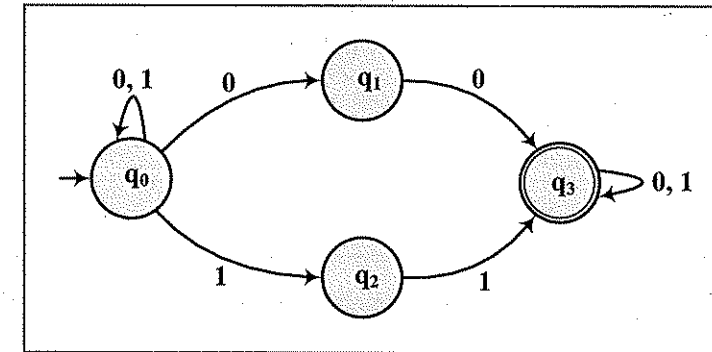
$$\delta(q_2, 0) = \Phi$$

$$\delta(q_2, 1) = \{q_3\}$$

$$\delta(q_3, 0) = \{q_3\}$$

$$\delta(q_3, 1) = \{q_3\}$$

$M_{1.3}$ 'ün tanımında görüldüğü gibi, deterministik olmayan modelde bazı durumlardan bazı giriş simgeleri ile birden çok duruma geçilebilmektedir. Bazı durumlardan bazı giriş simgeleri ile de hiçbir duruma geçilememektedir. Bu nedenle, deterministik modelde, başlangıç durumu ve uygulanan giriş dizgisi bilindiğinde makinenin hangi durumda bulunacağı kesin olarak bellidir. Buna karşılık, deterministik olmayan modelde başlangıç durumu ve uygulanan giriş dizgisi bilinen makinenin son durumu belirsiz olabilir. Örneğin başlangıç durumu q_0 olan $M_{1.3}$ 'e $w = 000$ giriş dizgisi uygulandığında, makinenin son durumu q_0 , q_1 ve q_3 durumlarından herhangi biri olabilir.

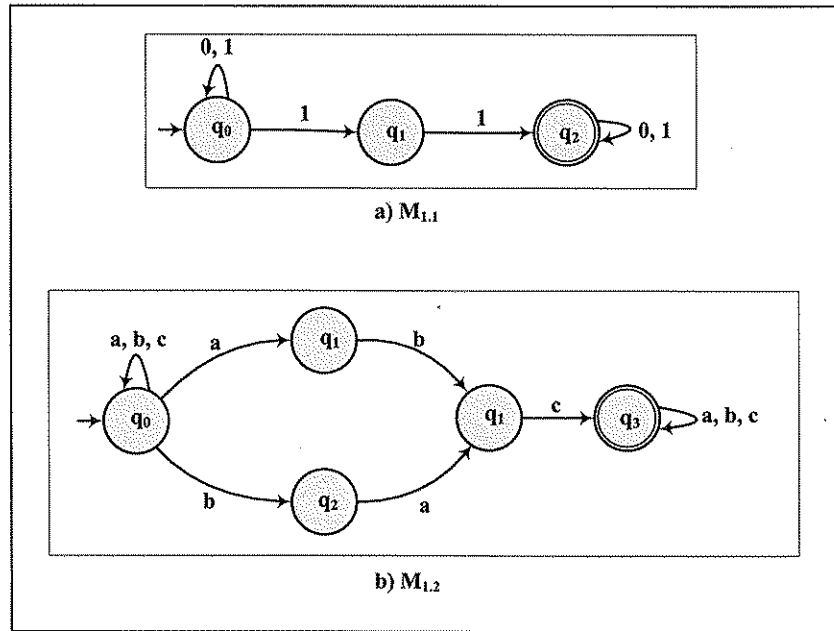


Çizim 1.4. $M_{1.3}$ 'ün Deterministik Olmayan Geçiş Çizeneği

Deterministik olmayan bir sonlu özdevinirin (NFA) bir giriş dizgisini tanıması için, geçiş çizeneğinde, başlangıç durumundan başlayarak bu giriş dizgisi ile bir uç duruma ulaşan bir yol bulunabilmesi gerekir. Örneğin $M_{1,3}$, $w = 10001$ giriş dizgisini tanıır. Çünkü bu giriş dizgisine karşı gelen ve q_0 'dan başlayarak tek uç durum olan q_3 'te son bulan bir yol

$$(q_0 \rightarrow q_0 \rightarrow q_1 \rightarrow q_3 \rightarrow q_3 \rightarrow q_3 \text{ ya da } q_0 \rightarrow q_0 \rightarrow q_0 \rightarrow q_1 \rightarrow q_3 \rightarrow q_3)$$

bulunabilmektedir. Buna karşılık $M_{1,3}$, $w = 010$ giriş dizgisini tanımaz. Çünkü q_0 ve q_3 durumları arasında bu giriş dizgisine karşı gelen bir yol bulmak mümkün değildir. Deterministik olmayan geçiş çizeneklerinin oluşturulması ve okunması, deterministik çizeneklere göre daha kolaydır. Örneğin Çizim 1.4 incelendiğinde, $M_{1,3}$ 'ün tanıdığı kümenin " $\{0, 1\}$ alfabesinde, içinde 00 ve 11 alt dizgilerinden en az biri, en az bir kez bulunan dizgiler kümesi" olduğu kolaylıkla bulunabilir. Oysa Çizim 1.3 incelenerek, $M_{1,2}$ 'nin tanıdığı kümeyi bulmak, başka bir deyişle Çizim 1.3'deki deterministik geçiş çizeneğini oluşturmak hiç de kolay değildir. Bir karşılaştırma olanağı sağlamak için Çizim 1.5'de $M_{1,1}$ ve $M_{1,2}$ 'nin deterministik olmayan geçiş çizenekleri verilmiştir.



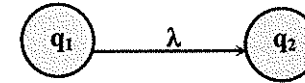
Çizim 1.5. $M_{1,1}$ ve $M_{1,2}$ 'nin Deterministik Olmayan Geçiş Çizenekleri

1.1.3. Lambda (λ) Geçişi

Sonlu özdevinirlerde durum geçişleri, giriş simgelerinin işlenmesi ile gerçekleşir. Makine belli bir durumdayken, bir giriş simgesinin uygulanması makinenin bir sonraki duruma geçmesine neden olur. Geçiş işlevi ile tanımlanan "sonraki durum" deterministik modelde tek bir durumdur. Deterministik olmayan modelde ise "sonraki durum" durumlar kümesinin bir altkütmesi olup bu altkümeye sıfır, bir ya da birçok durum bulunabilir. Şimdiye kadar yapılan tanımlara göre, DFA ya da NFA modeline uygun bir makine, giriş simgesi uygulanmadığı sürece bulunduğu durumu korur. *Lambda* (λ) geçişi ile, deterministik olmayan modelle genişletilen ve kullanımı kolaylaştırılan sonlu özdevinir tanımı daha da genişletilir. *Lambda* ve *lambda*-geçiş soyut kavramlardır. *Lambda* boş simge olarak düşünülebilir. *Lambda*-geçiş ise, hiçbir giriş simgesi uygulanmadan (ya da işlenmeden) gerçekleşen durum geçişine karşı gelir. Bir makinenin q_1 ve q_2 durumları arasında

$$\delta(q_1, \lambda) = q_2$$

biçiminde tanımlanan λ -geçiş varsa, q_1 durumunda bulunan makinenin, hiçbir giriş simgesi işlenmeden, kendinden q_2 durumuna geçebileceği anlaşılr.



Lambda geçişi, modelin esnekliğini artıran; geçiş çizeneklerinin daha kolay oluşturulmasını ve okunmasını sağlayan soyut bir kavramdır. Diğer geçişler gibi, *lambda* geçişi de tek yönlüdür. q_1 ve q_2 durumları arasındaki q_1 'den q_2 'ye λ -geçiş, hiçbir giriş simgesi uygulanmadan makinenin q_1 durumundan q_2 durumuna geçebileceği gösterir. Buna göre, q_1 durumunda bulunan makinenin aynı zamanda q_2 durumunda da bulunduğu anlamı çıkar. Ancak bunun tersi doğru değildir.

q_1 ve q_2 durumları arasında, q_1 'den q_2 'ye λ -geçiş varsa, işlevsel olarak:

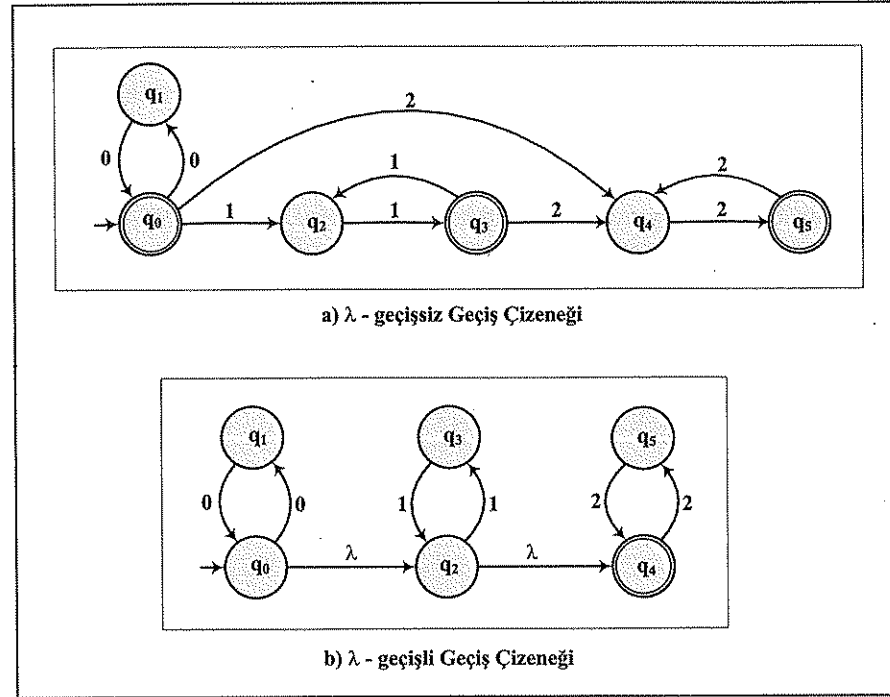
- eğer q_1 başlangıç durumu ise, q_2 de başlangıç durumu,
- eğer q_2 bir uç durum ise, q_1 de bir uç durum

niteliği kazanır.

Örnek 1.4. *Lambda* geçişlerinin sonlu özdevinir modeline sağladığı esnekliği göstermek için, tanıdığı küme $\{0, 1, 2\}$ alfabesinde:

$$T(M_{1,4}) = \{0^{2n} 1^{2m} 2^{2k} \mid n \geq 0, m \geq 0, k \geq 0\}$$

biçiminde tanımlanan makineyi düşünelim. $M_{1,4}$ 'ün, λ -geçiş kullanmadan oluşturulan geçiş çizeneği Çizim 1.6.a'da, λ -geçiş kullanılarak oluşturulan geçiş çizeneği ise Çizim 1.6.b'de görülmektedir. λ -geçişsiz ve λ -geçişli geçiş çizenekleri incelendiğinde, λ -geçişlerinin sonlu özdevinir modeline kazandırdığı esneklik ve kullanım kolaylığı açık biçimde görülmektedir.



Çizim 1.6. $M_{1.4}$ için Geçiş Çizenekleri : a) λ -Geçişsiz b) λ -Geçişli

λ -Geçişsiz Eşdeğer Geçiş Çizeneğinin Bulunması

λ -geçişlerinin kullanımı, geçiş çizeneklerinin kullanımını kolaylaştırır ancak gücünü arttırmaz. Başka bir deyişle, λ -geçişleri kullanılarak oluşturulan her geçiş çizeneğine denk, λ -geçişsiz bir geçiş çizeneği bulunabilir. İki geçiş çizeneğinin denkliği, tanıdıkları kümenin eşitliği anlamını taşımaktadır. Buna göre, λ -geçişli bir geçiş çizeneği tarafından tanınan, ancak λ -geçişsiz hiçbir geçiş çizeneği tarafından tanınmayan hiçbir dizgiler kümesi yoktur. Bu da, geçiş çizeneklerinde λ -geçişi kullanmanın, NFA modelinin ifade gücünde hiçbir değişiklik yapmadığını gösterir.

λ -geçişli bir geçiş çizeneği verildiğinde, λ -geçişlerini tek tek yok ederek eşdeğer bir geçiş çizeneği elde etmek mümkündür. Bu kapsamda, q_1 ve q_2 durumları arasında q_1 'den q_2 'ye λ -geçişi varsa:

- q_2 durumundan başlayan her durum geçişine ($\forall \delta(q_2, a) = q_k$) karşılık q_1 durumundan başlayan ve aynı giriş simgesi ile aynı duruma ulaşan bir durum geçişi ($\delta(q_1, a) = q_k$) eklendikten,
- eğer q_1 durumu başlangıç durumu ise, q_2 durumu da başlangıç durumu yapıldıktan, ve

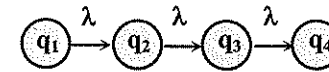
- eğer q_2 durumu bir uç durum ise, q_1 durumu da uç durum yapıldıktan sonra, λ -geçişi silinebilir.

Örnek 1.5. $M_{1.5}$ makinesi, $\{a, b, c\}$ alfabesinde, sıfır ya da iki tane a ile, ya da çift sayıda b (0, 2, 4, ... tane b) ile başlayıp cc ile biten dizgiler kümesini tanıyan makine olsun. $M_{1.5}$ 'in tanıdığı kümedeki dizgilerden birkaç örnek aşağıda görülmektedir:

$$T(M_{1.5}) = \{cc, aacc, bbcc, bbbcc, bbbbbcc, \dots\}$$

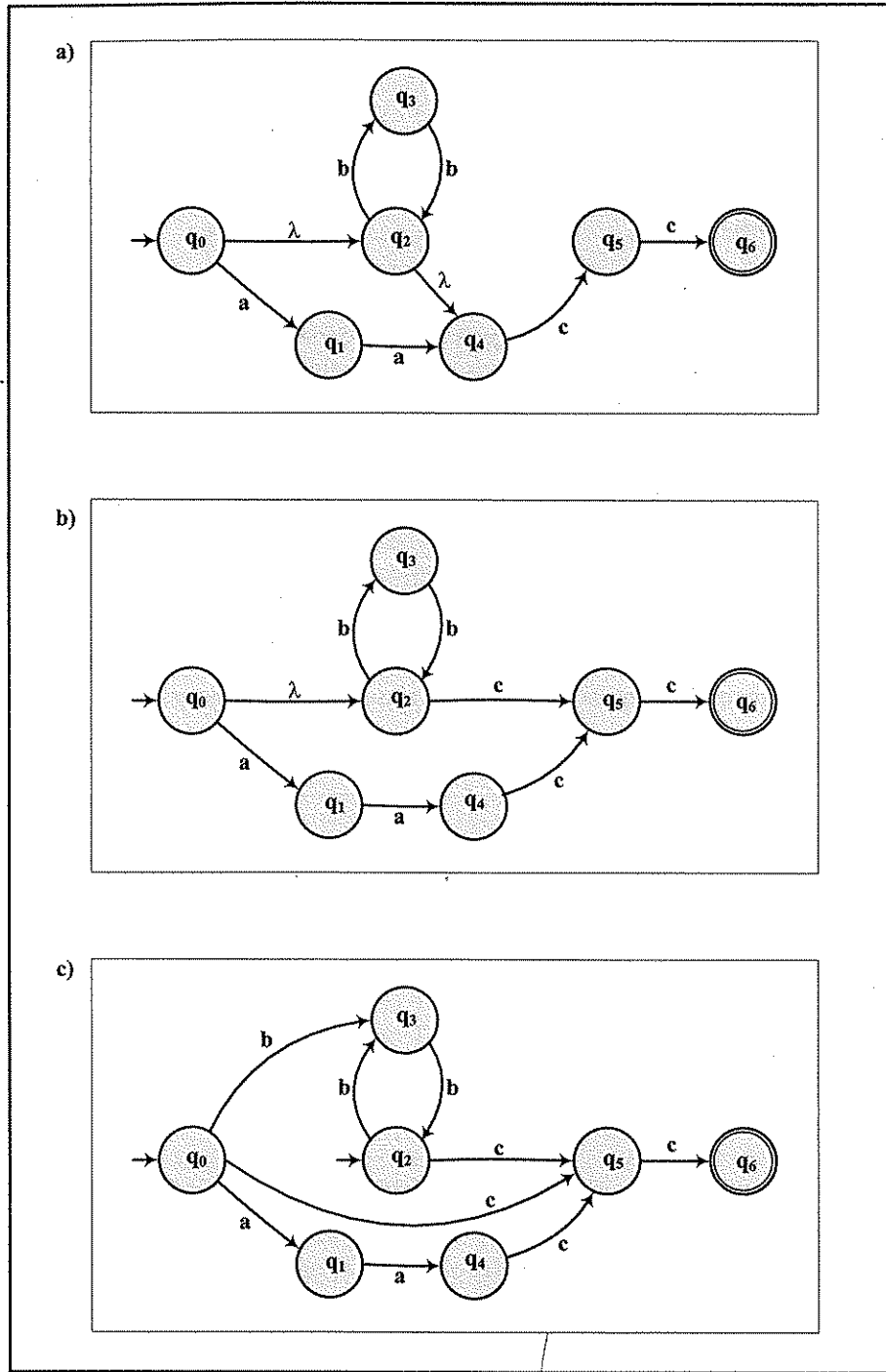
$M_{1.5}$ için oluşturulan λ -geçişli geçiş çizeneği Çizim 1.7.a'da görülmektedir. Bu çizeneğe eşdeğer, λ -geçişsiz geçiş çizeneğini elde edebilmek için, önce q_2 ile q_4 arasındaki, sonra da q_0 ile q_2 arasındaki λ -geçişleri yok edilir. q_2 ile q_4 arasındaki λ -geçişini yok edebilmek için q_2 ile q_5 arasına c -geçişi eklenir ve Çizim 1.7.b'deki geçiş çizeneği elde edilir. q_0 ile q_2 arasındaki λ -geçişini yok edebilmek için ise q_0 ile q_3 arasına b -geçişi, q_0 ile q_5 arasına c -geçişi eklenip q_2 durumu başlangıç durumu yapılır ve Çizim 1.7.c'deki λ -geçişsiz geçiş çizeneği elde edilir.

Eğer λ -geçişleri yok edilecek geçiş çizeneğinde, zincir yapısında birden çok λ -geçişi varsa, yok etme işlemini zincirin ucundan başlatarak geriye doğru sürdürmekte yarar vardır. Örneğin q_1 ile q_2 ; q_2 ile q_3 ; q_3 ile de q_4 arasında üç λ -geçişi varsa:



- önce q_3 ile q_4 arasındaki,
- sonra q_2 ile q_3 arasındaki,
- son olarak da q_1 ile q_2 arasındaki

λ -geçişi yok edilir.

Çizim 1.7. Geçiş Çizeneğinde λ -Geçişlerinin Yok Edilmesi

1.1.4. Deterministik ve Deterministik Olmayan Sonlu Özdevinir Modellerinin Denkliği

Acaba deterministik ve deterministik olmayan özdevinir modelleri denk midir? Eğer her iki model ile tanımlanabilen kümeler sınıfı aynı ise bu iki model denktir. Yok eğer modellerden biri ile tanımlanabilen ancak diğeri ile tanımlanamayan kümeler varsa, bu iki model denk değildir. Tanımlarına göre, deterministik olmayan model deterministik modeli de kapsadığına, başka bir deyişle her DFA aynı zamanda bir NFA olduğuna göre, deterministik bir FA tarafından tanınan, ancak hiçbir NFA tarafından tanınmayan bir küme düşünülemez. Ancak NFA tarafından tanınan her küme için, bu kümeyi tanıyan bir DFA bulmak mümkün müdür? Eğer bu sorunun yanıtı evet ise, deterministik ve deterministik olmayan modellerin denk olduğu söylenebilir.

Yukarıdaki sorunun yanıtını bir başka soruya yanıt arayarak bulmaya çalışalım. Bir dizgi ve bir DFA verildiğinde, geçiş çizeneğini kullanarak bu dizginin bu DFA tarafından tanınıp tanınmadığını bulmak kolaydır. Ancak bir dizgi ve bir NFA verildiğinde, geçiş çizeneğini kullanarak bu dizginin bu NFA tarafından tanınıp tanınmadığını bulmak kolay değildir.

Örnek 1.6. $M_{1.6} = \langle Q, \Sigma, \delta, q_0, F \rangle$

$$Q = \{A, B, C\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = A$$

$$F = \{C\}$$

$$\delta : \delta(A, 0) = \{A\}$$

$$\delta(A, 1) = \{B, C\}$$

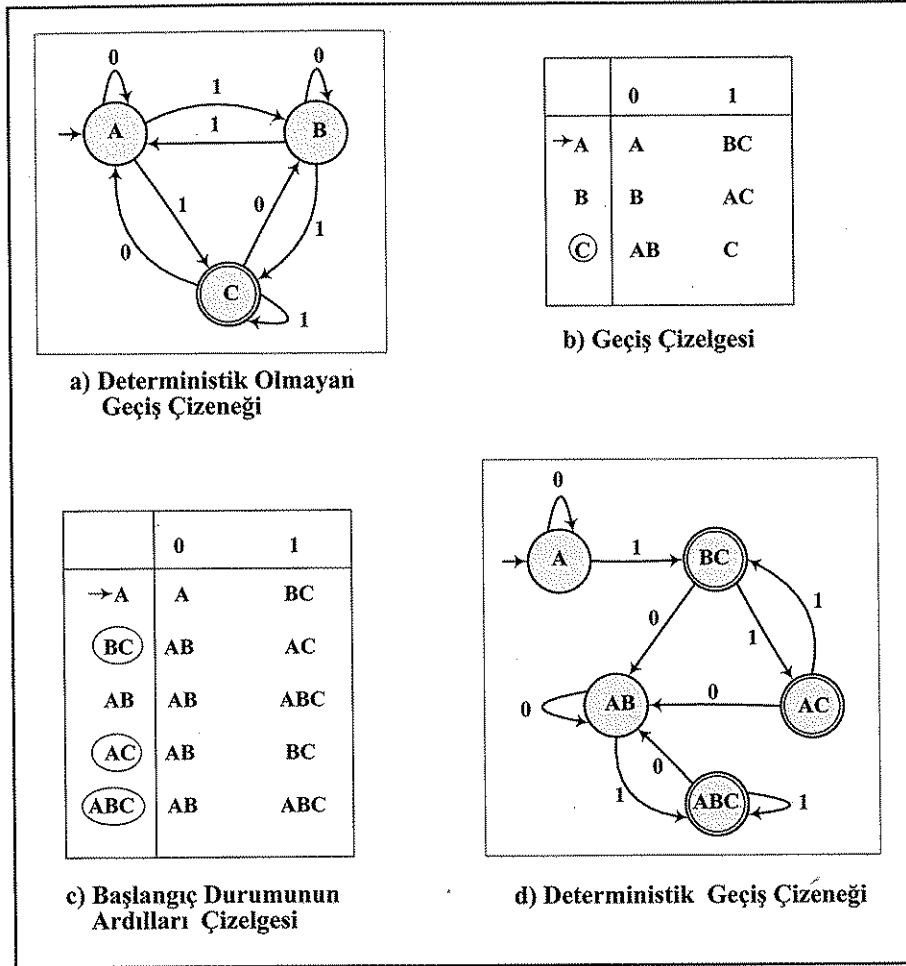
$$\delta(B, 0) = \{B\}$$

$$\delta(B, 1) = \{A, C\}$$

$$\delta(C, 0) = \{A, B\}$$

$$\delta(C, 1) = \{C\}$$

Örnek olarak $M_{1.6}$ 'yı düşünelim. Deterministik olmayan modele göre tanımlanan bu makinenin geçiş çizeneği Çizim 1.8.a'da görülmektedir. Başlangıç durumu A, tek uç durum da C olduğuna göre, verilen bir dizginin $M_{1.6}$ tarafından tanınabilmesi için, A ve C durumları arasında, bu dizgiye karşı gelen bir yolun bulunabilmesi gerekir. Bu işlemin sistemli bir şekilde yapılmasını sağlayacak bir yöntem bulmaya çalışalım. Bu amaçla Çizim 1.8.b'de görülen geçiş çizeneğinden yararlanabiliriz. Geçiş çizelgesi, geçiş çizeneği ile aynı bilgiyi içeren ve makinenin her durumundan, her giriş simgesi ile hangi durumlara gidilebileceğini gösteren bir çizelgedir. Başka bir deyişle geçiş çizelgesi, her durumun her giriş simgesine karşı gelen ardıllarını kolaylıkla bulmayı



Çizim 1.8. NFA'ya Denk DFA'nın Bulunması

sağlar. Ancak verilen bir w giriş dizgisinin $M_{1,6}$ tarafından tanınabilmesi için, başlangıç durumu olan A 'nın w -ardılları arasında tek uç durum olan C 'nin yer alması gerekir. Bu amaçla, Çizim 1.8.c'de görülen ve başlangıç durumunun tüm ardıllarını içeren çizelgeyi hazırlayabiliriz. Bir w giriş dizgisi verildiğinde, bu ardıl çizelgesi yardımıyla, w 'nin $M_{1,6}$ tarafından tanınıp tanınmadığı kolaylıkla bulunabilir. Örneğin $w = 011$ ise, A 'nın 011 -ardılı AC olduğu ve AC bir uç durum içerdiği için de 011 dizgisinin $M_{1,6}$ tarafından tanındığı; buna karşılık $w = 110$ ise, A 'nın 110 -ardılının AB olduğu ve AB içinde bir uç durum bulunmadığı için de 110 dizgisinin $M_{1,6}$ tarafından tanınmadığı kolaylıkla bulunabilir.

$M_{1,6}$ 'nın başlangıç durumunun ardıllarını gösteren Çizim 1.8.c'deki çizelge ile deterministik bir özdevinir tanımlanmaktadır. Bu özdeviniri $M_{D1,6}$ olarak adlandıralım. $M_{D1,6}$ 'nın biçimsel tanımı aşağıdaki gibidir.

$$M_{D1,6} = \langle Q, \Sigma, \delta, q_0, F \rangle$$

$$Q = \{A, BC, AB, AC, ABC\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = A$$

$$F = \{BC, AC, ABC\}$$

$$\delta : \delta(A, 0) = A$$

$$\delta(A, 1) = BC$$

$$\delta(BC, 0) = AB$$

$$\delta(BC, 1) = AC$$

$$\delta(AB, 0) = AB$$

$$\delta(AB, 1) = ABC$$

$$\delta(AC, 0) = AB$$

$$\delta(AC, 1) = BC$$

$$\delta(ABC, 0) = AB$$

$$\delta(ABC, 1) = ABC$$

$M_{D1,6}$ 'nin geçiş çizeneği Çizim 1.8.d 'de görülmektedir. $M_{D1,6}$, verilen bir w giriş dizgisinin $M_{1,6}$ tarafından tanınıp tanınmadığını bulmak üzere geliştirildi. Dolayısıyla $M_{1,6}$ ve $M_{D1,6}$ 'nin tanıdığı dizgiler kümesinin aynı olduğu ve bu iki makinenin denk olduğu açıktır. Başka bir deyişle $M_{D1,6}$, deterministik olmayan $M_{1,6}$ makinesiyle aynı kümeyi tanıyan, dolayısıyla $M_{1,6}$ 'ya denk, deterministik bir makinedir. Bir kez bulduktan sonra, deterministik makinenin durumlarına $A, B, C, \dots$ ya q_0, q_1, q_2 gibi adlar vererek durumları yeniden adlandırmak mümkündür.

1.1.5. İki Yönlü Sonlu Özdevinir Modeli

Sonlu özdevinirlerin temel modeli olan DFA modeli tek yönlü bir modeldir. Çizim 1.2'de soyut bir makine olarak sunulan bu modelde, okuma şeridi soldan sağa doğru tek yönlü işlenir. DFA'nın her hareketinde, şeritten bir simge okunur ve okuma kafası bir sağdaki hücreye geçer. Tek yönlü modelde okuma kafası şerit üzerinde sola doğru hareket edemez.

İki yönlü sonlu özdevinir modelinde ise okuma kafasının hem sağa, hem de sola doğru hareket etmesi mümkündür. Makinenin her hareketinde okuma kafası, üzerinde

bulunduğu hücredeki simgeyi okuduktan sonra ya bir sağdaki, ya da bir soldaki hücreye geçer.

İki yönlü sonlu özdevinirlerin yalnız deterministik modeli incelenecek ve bu model 2DFA olarak adlandırılacaktır. 2DFA bir beşli olarak aşağıdaki gibi tanımlanır.

$$2DFA = \langle Q, \Sigma, \delta, q_0, F \rangle$$

2DFA'nın tanımında yer alan durumlar kümesi (Q), giriş alfabesi (Σ), başlangıç durumu (q_0) ve uç durumların (F) anlamları, DFA tanımındakilerle aynıdır (bkz 1.1.1). DFA ve 2DFA arasındaki tek fark geçiş işlevininin tanımındadır. 2DFA modelinde geçiş işlevi:

$(Q \times \Sigma)$ 'dan $Q \times \{R, L\}$ 'ye bir eşleme

olarak tanımlanır. Bu tanımda R (right) ve L (left), okuma kafasının bir sağa mı yoksa bir sola mı geçeceğini gösterir.

Örnek 1.7. $M_{1,7} = \langle Q, \Sigma, \delta, q_0, F \rangle$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_0, q_1, q_2\}$$

$$\delta : \delta(q_0, 0) = (q_0, R)$$

$$\delta(q_0, 1) = (q_1, R)$$

$$\delta(q_1, 0) = (q_1, R)$$

$$\delta(q_1, 1) = (q_2, L)$$

$$\delta(q_2, 0) = (q_0, R)$$

$$\delta(q_2, 1) = (q_2, L)$$

Geçiş işlevinin çizelge gösterimi (geçiş çizelgesi):

	0	1
$\rightarrow q_0$	(q_0, R)	(q_1, R)
q_1	(q_1, R)	(q_2, L)
q_2	(q_0, R)	(q_2, L)

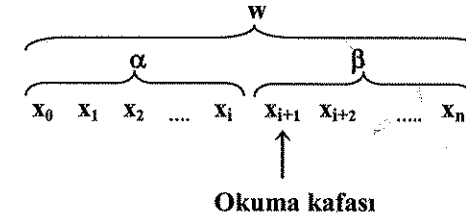
Örnek 1.7'deki $M_{1,7}$ makinesi, $\{0, 1\}$ alfabesinde içinde 11 alt dizgisi bulunmayan dizgileri tanıyan bir 2DFA'dır. Tüm durumlar uç durum olduğuna göre, bir dizginin $M_{1,7}$ tarafından tanınabilmesi için, sonlu sayıda hareket sonunda, tüm dizginin işlenmesi ve okuma kafasının dizginin sağına geçmesi yeterlidir. Tek yönlü modelde her zaman sağlanan bu koşul, iki yönlü özdevinirlerde her zaman sağlanamaz. İki

nedenle, okuma kafası sonlu hareket sonunda dizginin sağına ulaşamaz. Bu nedenlerden birincisi okuma kafasının hareketlerinin bir döngü oluşturması, ikincisi ise şeridin ilk hücresi üzerinde bulunan okuma kafasının sola ilerlemesini gerektiren bir hareketle karşılaşılmasıdır.

Bir dizginin 2DFA tarafından tanınıp tanınmadığını bulmak pek kolay değildir. Bunun için şeritli makine modeli üzerinde okuma kafasının hareketlerinin izlenmesi gerekir. Ancak hareketler iki yönlü olduğu ve aynı hücre üzerinden okuma kafası defalarca geçebildiği için bu işlem oldukça uzun ve güç olabilir. Diğer bir yöntem ise, şeritli makine modelini kullanmadan, hareketlerin anlık tanımlarla izlenmesidir.

Anlık Tanım (ID : Instantaneous Description)

Makinenin giriş dizgisi w olsun. Başlangıçta okuma kafası w 'nin ilk (en soldaki) simgesi üzerindedir. Belirli sayıda hareket sonunda, okuma kafası w 'nin belirli bir simgesinin üzerinde bulunur. Bu durumda, w 'nin bir kesimi okuma kafasının solunda yer alır. w 'nin okuma kafasının solunda kalan kesimini α ile, geri kalan kesimini ise β ile gösterelim. Buna göre okuma kafası β 'nin ilk (en soldaki) simgesi üzerinde bulunmaktadır.



Anlık tanım makinenin bulunduğu durum ile giriş dizgisinin okuma kafasının solunda ve sağında kalan kesimlerini gösteren bir üçlüdür.

$$ID = (\alpha, q_i, \beta)$$

Eğer ID_1 'den tek bir hareketle ID_2 'ye geçiliyorsa, bu iki anlık tanım arasındaki ilişki:

$$ID_1 \vdash ID_2$$

ile gösterilir.

Eğer ID_1 'den bir dizi hareket sonunda ID_2 'ye geçilebiliyorsa, bu ilişki de:

$$ID_1 \vdash^* ID_2$$

ile gösterilir.

Anlık tanım ve anlık tanımlar arasındaki geçiş ilişkisine dayalı olarak, iki yönlü sonlu bir özdevinirin (2DFA) tanıdığı dizgiler kümesi aşağıdaki gibi tanımlanır:

$$T(M) = \{w \mid (q_0, w) \vdash^* (w, q_i), q_i \in F\}$$

Yukarıdaki tanımda, (q_0, w) başlangıç anlık tanımıdır (ID_0). Başlangıçta okuma kafası w 'nin ilk simgesi üzerinde bulunduğu ve okuma kafasının solunda dizginin hiçbir simgesi bulunmadığı için ID_0 'da α yoktur. (w, q_i) ise, makinenin bir uç durumunda ($q_i \in F$), okuma kafasının ise w 'nin sağındaki boş hücre üzerinde bulunduğu son ID 'dir. Son ID 'de, w 'nin tümü okuma kafasının solunda bulunduğu için bu kez β yoktur.

Anlık tanımları kullanarak, $w_1 = 1000101$ ve $w_2 = 100110$ giriş dizgilerinin iki yönlü sonlu bir özdevinir olan $M_{1,7}$ makinesi tarafından tanınıp tanınmadığını inceleyelim:

a) $w_1 = 1000101$

$(q_0, 1000101) \vdash (1, q_1, 000101) \vdash (10, q_1, 00101) \vdash (100, q_1, 0101) \vdash$
 $(1000, q_1, 101) \vdash (100, q_2, 0101) \vdash (1000, q_0, 101) \vdash (10001, q_1, 01) \vdash$
 $(100010, q_1, 1) \vdash (10001, q_2, 01) \vdash (100010, q_0, 1) \vdash (1000101, q_1)$

Sonuç : $w_1 = 1000101$ dizgisi $M_{1,7}$ tarafından tanınır.

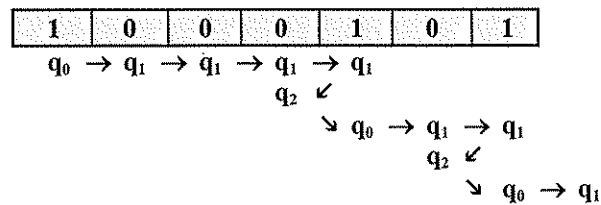
b) $w_2 = 100110$

$(q_0, 100110) \vdash (1, q_1, 00110) \vdash (10, q_1, 0110) \vdash (100, q_1, 110) \vdash$
 $(10, q_2, 0110) \vdash (100, q_0, 110) \vdash (1001, q_1, 10) \vdash (100, q_2, 110) \vdash$
 $(10, q_2, 0110) \vdash (100, q_0, 110) \dots\dots\dots$

Sonuç : ID geçişlerinde döngü olduğu için $w_2 = 100110$ dizgisi $M_{1,7}$ tarafından tanınmaz.

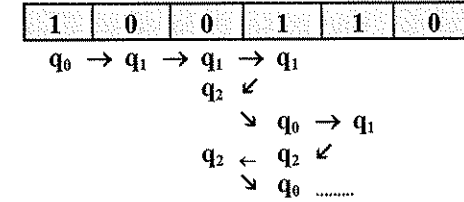
Giriş dizgilerinin 2DFA tarafından tanınıp tanınmadığını, ID 'ler arası geçiş ilişkileri yerine, şerit üzerindeki hareketleri ve durum değişimlerini inceleyerek de bulabiliriz. Bu yöntemle $w_1 = 1000101$ ve $w_2 = 100110$ giriş dizgilerinin $M_{1,7}$ makinesi tarafından tanınıp tanınmadığı aşağıdaki gibi bulunabilir.

a) $w_1 = 1000101$



Sonuç : $w_1 = 1000101$ dizgisi $M_{1,7}$ tarafından tanınır.

b) $w_2 = 100110$



Sonuç : Makinenin hareketlerinde döngü olduğu ve $w_2 = 100110$ dizgisinin $M_{1,7}$ tarafından tanınmadığı yukarıda açık biçimde görülmektedir. Şeridin bir hücresi altındaki durumlar dizisinde aynı durumun birden çok kez yer alması döngünün göstergesidir.

Tek ve İki Yönlü Sonlu Özdevinirlerin Denklığı

Daha önce, deterministik olmayan her sonlu özdevinire denk (aynı kümeyi tanıyan) deterministik bir sonlu özdevinirin bulunabileceği, dolayısıyla NFA ve DFA modellerin denk olduğunu gördük. Benzer biçimde, iki yönlü her sonlu özdevinire denk, tek yönlü bir sonlu özdevinir bulunabilir. Dolayısıyla 2DFA ve DFA modellerinin ifade güçleri denktir. İki modelin denklığı, verilen bir 2DFA'ya denk tek yönlü bir DFA'nın bulunmasını sağlayan yöntem ile ispat edilebilir. Böyle bir yöntem vardır. Ancak bu yönteme bu kitapta yer verilmeyecektir.

1.2. Çıkış Üreten Özdevinirler

Bölümün başında sonlu özdevinirlerin öncelikle:

- Sonlu durumlu tanıyıcı (*finite state recognizer*)
- Çıkış üreten özdevinir

modelleri olarak sınıflandırıldığı belirtilmişti. Tanıyıcı model, bir giriş afabesindeki simgelerden üretilebilecek dizgilerin bir kesimini tanıyan bir modeldir. Bu model biçimsel dillerle ilgili sözdizim çözümleme (*syntax analysis*), ayrıştırma (*parsing*) ve benzeri uygulamalar başta olmak üzere birçok yazılım uygulamasında kullanılır. Sonlu durumlu tanıyıcı modelin deterministik ve deterministik olmayan türlerinin bulunduğu, ve bu iki türün modelleme gücü açısından denk olduğu da yukarıdaki kesimlerde görüldü.

Çıkış üreten özdevinir modeli, adından da anlaşıldığı gibi, girişine uygulanan bir giriş dizgisine yanıt olarak, bir çıkış dizgisi üreten bir modeldir. Bu açıdan çıkış üreten özdevinir modelini, giriş dizgilerini çıkış dizgilerine dönüştüren bir model olarak görmek de mümkündür. Böylece sonlu özdevinirlerin iki ana türü, "tanıyıcılar" ve "dönüştürücüler" olarak nitelenebilir. Aslında tanıyıcıların da çıkış ürettikleri düşünülebilir. Madem ki bir tanıyıcı girişine uygulanan giriş dizgilerinin bir kısmını tanımakta, diğer bir kısmını ise tanımamaktadır, FA modeli ikili çıkış üreten bir

model olarak görülebilir. Modelin ürettiği çıkışlar, örneğin uç durumlar için "1 : tanıdı", diğer durumlar için "0 : tanımadı" olabilir. Bu açıdan düşünüldüğünde ise, çıkış üreten sonlu özdevinir modeli, tanıyıcı modeli de kapsayan, daha geniş bir model olarak görülebilir.

Çıkış üreten özdevinirlerin Moore ve Mealy makinesi olarak adlandırılan iki türü vardır. Moore makinesi durum düzeyinde çıkış üreten bir model iken, Mealy makinesi durum geçişi düzeyinde çıkış üreten bir modeldir. Aşağıda Moore ve Mealy makineleri ayrı ayrı incelenmektedir.

Moore Makinesi

Moore Makinesi, bir altılı olarak tanımlanır.

$$M = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$$

Q : Sonlu durumlar kümesi

Σ : Giriş alfabesi (sonlu bir küme)

Δ : Çıkış alfabesi (sonlu bir küme)

δ : Durum geçiş işlevi : $(Q \times \Sigma)$ 'dan Q 'ya bir eşleme

λ : Çıkış işlevi : Q 'dan Δ 'ya bir eşleme

q_0 : Başlangıç durumu : $q_0 \in Q$

Moore makinesi, DFA modelinin genellemesi olarak görülebilir. Başka bir deyişle DFA modeli özel bir Moore makinesi olarak düşünülebilir. Gerçekten de eğer Moore modelinde çıkış alfabesi olarak $\{0, 1\}$ alfabesi alınır ve çıkış işlevi ile uç durumlara 1, uç olmayan durumlara da 0 değeri eşlenirse:

$$\Delta = \{0, 1\}, \quad \forall q_i \in F \Rightarrow \lambda(q_i) = 1, \quad \forall q_j \notin F \Rightarrow \lambda(q_j) = 0$$

DFA modeli elde edilir.

Moore makinesinin durum geçiş işlevi (δ) ile çıkış işlevi (λ) bir çizelge ya da çizenek ile gösterilebilir. Moore makinesini tanımlamak için oluşturulan çizelge çoğunlukla "Durum Çizelgesi (State Table)", çizenek ise çoğunlukla "Durum Çizeneği (State Diagram)" olarak adlandırılır. Örnek 1.8'de tanımlanan $M_{1.8}$ Moore makinesinin durum çizenek ve çizelgesi Çizim 1.9'da görülmektedir.

Örnek 1.8.

$M_{1.8}$ makinesi, girişine uygulanan ikili sayı X ise, çıkışında $z = \text{Mod}(X, 5)$ değerini üreten Moore makinesi olarak tanımlanıyor.

$$M_{1.8} = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$$

$$Q = \{A, B, C, D, E\}$$

$$\Sigma = \{0, 1\}$$

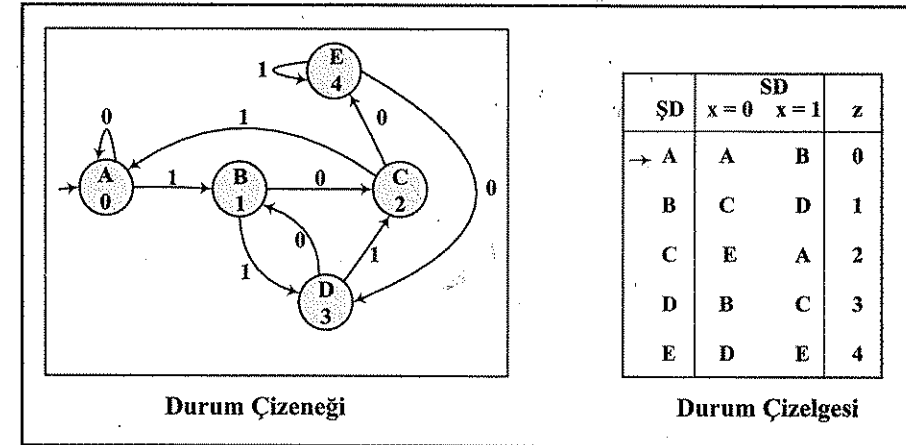
$$\Delta = \{0, 1, 2, 3, 4\}$$

$$q_0 = A$$

δ ve λ 'nın tanımı, Çizim 1.9.a'daki Durum Çizeneği ile, Çizim 1.9.b'deki Durum Çizelgesinde yer almaktadır.

$M_{1.8}$ makinesine örneğin $w = 1011110$ giriş dizgisi uygulandığında, makinenin durumları ve üreteceği çıkışlar aşağıdaki gibi değişecektir.

Giriş : 1 0 1 1 1 1 0
Durum: A → B → C → A → B → D → C → E
Çıkış : 0 1 2 0 1 3 2 4



Çizim 1.9. Örnek Moore Makinesi ($M_{1.8}$)

Mealy Makinesi

Moore makinesi gibi, Mealy makinesi de bir altılı olarak tanımlanır.

$$M = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$$

Altılının çıkış işlevi (λ) dışındaki 5 elemanın tanımı Moore makinesindekilerle aynıdır. Başka bir deyişle Moore ve Mealy makineleri arasındaki tek fark çıkış işlevindedir. Moore makineleri için Q 'dan Δ 'ya bir eşleme olarak tanımlanan çıkış işlevi Mealy makineleri için $(Q \times \Sigma)$ 'dan Δ 'ya bir eşleme olarak tanımlanır.

Örnek 1.9.

$M_{1,9}$ makinesi giriş alfabesi $\{0, 1\}$, çıkış alfabesi ise $\{0, 1, 2\}$ olan ve ürettiği çıkış ile, son iki giriş simgesinden kaç tanesinin değerinin bir öncekinden farklı olduğunu gösteren Mealy makinesi olarak tanımlanıyor. Makinenin üreteceği ilk 2 çıkış değeri belirlenirken, başlangıç durumundan önceki iki giriş değerinin **00** olduğu varsayılacaktır.

$$M_{1,9} = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$$

$$Q = \{A, B, C, D\}$$

$$\Sigma = \{0, 1\}$$

$$\Delta = \{0, 1, 2\}$$

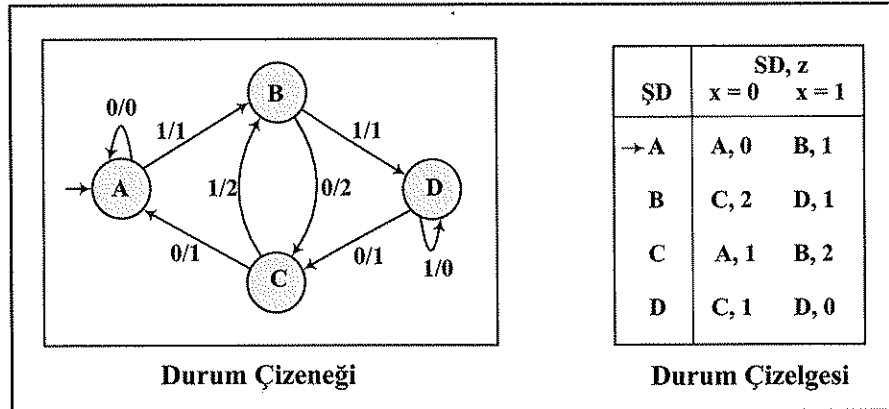
$$q_0 = A$$

δ ve λ , Çizim 1.10.a'daki Durum Çizeneği ve Çizim 1.10.b'deki Durum Çizelgesi ile tanımlanmaktadır. Yapılan tasarıma göre, makinenin durumları, önceki iki giriş simgesi değerinin **00**, **01**, **10** ya da **11** olmasına göre sırasıyla **A**, **B**, **C** ya da **D** olmaktadır.

Örnek bir giriş dizgisi ile bu giriş dizgisine karşılık makinenin üretmesi gereken çıkış dizgisi aşağıdaki gibidir:

X: 0 1 1 0 1 0 1 0 1 1 1 0 1 0 0 1 1 1 0 0 0 0 1

Z: 0 1 1 1 2 2 2 2 2 1 0 0 1 2 2 1 1 1 0 1 1 0 0 1



Çizim 1.10. Örnek Mealy Makinesi ($M_{1,9}$)

Moore ve Mealy Makinelerinin Eşdeğerliği

Moore ve Mealy makinelerinin her ikisi de, çıkış üreten sonlu özdevinir modelleridir. Birer altılı olarak tanımlanan Moore ve Mealy modellerinde altılının 5 elemanı ortaktır. Moore ve Mealy modellerini ayıran çıkış işlevidir. Moore modelinde, makine her durum için bir çıkış simgesi üretmektedir. Belirli bir durumda bulunan bir Moore makinesine, n giriş simgesinden oluşan bir giriş dizgisi uygulandığında, makine $(n+1)$ uzunluğunda bir çıkış dizgisi üretecektir. Başlangıçtaki durum q_0 , uygulanan n giriş simgesi sonrasındaki durumlar da $q_1, q_2, \dots, q_n$ ile gösterilerek, Moore makinesinin giriş dizgisi, durum geçişleri ve çıkış dizgisi ilişkisi aşağıdaki gibi özetlenebilir. Hiçbir giriş simgesi uygulamadan, bulunduğu başlangıç durumuna göre Moore makinesinin bir çıkış simgesi (z_0) ürettiği dikkati çekmektedir.

$$\begin{array}{ccccccccccc} x_1 & & x_2 & & x_3 & & \dots & & x_{n-1} & & x_n \\ q_0 & \rightarrow & q_1 & \rightarrow & q_2 & \rightarrow & q_3 & \rightarrow & \dots & \rightarrow & q_{n-2} & \rightarrow & q_{n-1} & \rightarrow & q_n \\ z_0 & & z_1 & & z_2 & & z_3 & & \dots & & z_{n-2} & & z_{n-1} & & z_n \end{array}$$

Mealy modelinde ise, makine her durum geçişi için bir çıkış simgesi üretmektedir. Belirli bir durumda bulunan bir Mealy makinesine, n giriş simgesinden oluşan bir giriş dizgisi uygulandığında, makine n uzunluğunda bir çıkış dizgisi üretecektir. Çünkü Mealy modelinde, giriş simgesi uygulanmadığı sürece makine çıkış üretmemektedir. Başlangıçtaki durum q_0 , uygulanan n giriş simgesi sonrasındaki durumlar da $q_1, q_2, \dots, q_n$ ile gösterilerek, Mealy makinesinin giriş dizgisi, durum geçişleri ve çıkış dizgisi ilişkisi aşağıdaki gibi özetlenebilir.

$$\begin{array}{ccccccccccc} x_1 & & x_2 & & x_3 & & \dots & & x_{n-1} & & x_n \\ q_0 & \rightarrow & q_1 & \rightarrow & q_2 & \rightarrow & q_3 & \rightarrow & \dots & \rightarrow & q_{n-2} & \rightarrow & q_{n-1} & \rightarrow & q_n \\ z_1 & & z_2 & & z_3 & & \dots & & z_{n-1} & & z_n \end{array}$$

Buna göre n uzunluğundaki X giriş dizgisine karşılık Mealy makinesi Z_{Mealy} çıkış dizgisini üretirken, Moore makinesi $z_0 Z_{Moore}$ çıkış dizgisini üretmektedir. Z_{Mealy} ve Z_{Moore} çıkış dizgileri, n uzunluğunda dizgilerdir.

Eğer aynı giriş ve aynı çıkış alfabesine sahip bir Mealy ve bir Moore makinesinin ürettiği Z_{Mealy} ve Z_{Moore} çıkış dizgileri, tüm X giriş dizgileri için aynı ise, bu iki makine birbirine denktir. Başka bir deyişle, Moore makinesinin başlangıçta ürettiği z_0 çıkış simgesi dışarda tutulursa, uygulanan giriş dizgisi ne olursa olsun, birbirine denk Mealy ve Moore makineleri aynı çıkış dizgisini üretecektir.

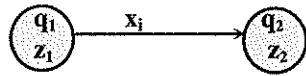
Moore Makinesine Eşdeğer Mealy Makinesinin Bulunması

Önerme 1.1. $M_1 = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$ bir Moore makinesi olsun. M_1 'e eşdeğer Mealy makinesi aşağıdaki gibidir:

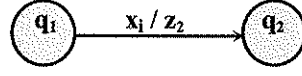
$$M_2 = \langle Q, \Sigma, \Delta, \delta, \lambda', q_0 \rangle, \quad \lambda'(q, a) = \lambda(\delta(q, a))$$

Yukarıdaki tanıma göre, bir Moore makinesi ile bu makineye eşdeğer Mealy makinesi arasındaki tek fark çıkış işlevindedir. Bilindiği gibi Moore makinesinde her duruma bir çıkış simgesi eşlenirken, Mealy makinesinde her durum geçişine (durum, giriş simgesi çiftine) bir çıkış simgesi eşlenir. Bu kitapta önermenin ispatına yer verilmeyecektir. Bir Moore makinesine eşdeğer Mealy makinesi bulunurken, her durum geçişine eşlenen çıkış değeri, durum geçişinin ucundaki sonraki duruma Moore makinesinde eşlenen çıkış değerine eşittir. Bu kuralı aşağıdaki çizimle özetlemek mümkündür.

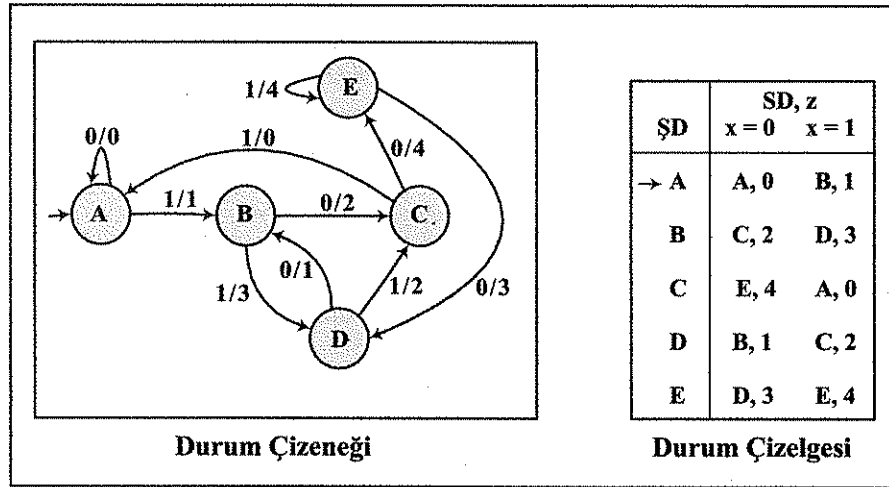
Moore makinesi



Mealy makinesi



Örnek 1.8'deki Moore makinesine ($M_{1.8}$) eşdeğer Mealy makinesinin, Önerme 1.1'e göre bulunan durum çizeneği ve çizelgesi Çizim 1.11'de görülmektedir. Bir Moore makinesi ile bu makineye eşdeğer Mealy makinesinin hem durum sayılarının hem de durumlar arası geçiş sayılarının eşit olduğu görülmektedir.

Çizim 1.11. $M_{1.8}$ Moore Makinesine Eşdeğer Mealy Makinesi

Mealy Makinesine Eşdeğer Moore Makinesinin Bulunması

Önerme 1.2. $M_2 = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$ bir Mealy makinesi olsun. M_2 'ye eşdeğer M_1 Moore makinesi aşağıdaki gibi bulunur:

$$M_1 = \langle Q', \Sigma, \Delta, \delta', \lambda', q'_0 \rangle$$

$$Q' = Q \times \Delta$$

$$q'_0 = [q_0, z_1] \quad (z_1 = \text{çıkış simgelerinden rasgele seçilmiş biri})$$

$$\delta'([q_i, z_k], x_j) = [\delta(q_i, x_j), \lambda(q_i, x_j)]$$

$$\lambda'([q_i, z_k]) = z_k$$

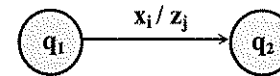
Önerme 1.2'ye göre, bir Mealy makinesi (M_2) ile bu makineye eşdeğer Moore makinesinin (M_1) tanımını oluşturan altılıların dördü birbirinden farklıdır.

- M_2 'nin her [durum, çıkış simgesi] çiftine karşılık, M_1 makinesinin bir durumu vardır. Buna göre M_2 makinesin durum sayısı n , çıkış simgesi sayısı ise m ise, M_1 makinesinin $(n \times m)$ durumu bulunacaktır. Örneğin M_9 Mealy makinesinin 4 durumu, 4 de çıkış simgesi vardır. M_9 Mealy makinesinin, Önerme 1.2'ye göre bulunacak eşdeğeri Moore makinesinin $4 \times 4 = 16$ durumu olacak ve bu durumlar sembolik olarak $[A, 0], [A, 1], \dots, [D, 4]$ diye gösterilecektir.
- M_1 'in durumlarından, sembolik gösterimde birinci elemanı q_0 olanlar arasından rasgele seçilen biri başlangıç durumu yapılır. Örneğin çıkış alfabetinde 4 simge bulunan M_9 Mealy makinesinin başlangıç durumu A olduğuna göre, bu makineye eşdeğer Moore makinesinin başlangıç durumu $[A, 0], [A, 1], [A, 2]$ ve $[A, 3]$ arasından rasgele seçilen biri olacaktır.
- M_1 'in geçiş işlevi

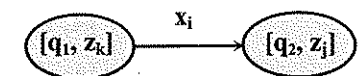
$$\delta'([q_i, z_k], x_i) = [\delta(q_i, x_i), \lambda(q_i, x_i)]$$

olarak tanımlanmaktadır. Bu tanımın daha kolay anlaşılmasını sağlamak için aşağıdaki çizim oluşturulabilir.

Mealy makinesi



Moore makinesi



Bir Mealy makinesine eşdeğer Moore makinesinin, yukarıdaki çizimle açıklanmaya çalışılan durum geçiş işlevi, sözlü olarak da şöyle anlatılabilir. Mealy makinesinde, q_1 durumundan q_2 durumuna x_1 geçişi sırasında z_1 çıkış simgesi üretiliyorsa, eşdeğer Moore makinesinde, birinci elemanı q_1 olan her durumdan, x_1 giriş simgesi ile, birinci elemanı q_2 , ikinci elemanı ise z_1 olan duruma bir geçiş oluşturulur. Buna göre eğer çıkış alfabetinde m çıkış simgesi varsa, Mealy makinesindeki her durum geçişine karşılık, eşdeğer Moore

makinesinde m geçiş bulunacaktır. Böylece bir Mealy makinesinin her durumuna karşılık eşdeğer Moore makinesinde m durum bulunduğu gibi, Mealy makinesindeki her durum geçişine karşılık eşdeğer Moore makinesinde m durum geçişi bulunacaktır.

Örnek 1.10.

$M_{1.10}$ Mealy makinesi aşağıdaki gibi tanımlanıyor :

$$M_{1.10} = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$$

$$Q = \{A, B, C\}$$

$$\Sigma = \{0, 1\}$$

$$\Delta = \{0, 1\}$$

$$q_0 = A$$

$$\delta(A, 0) = B \quad \lambda(A, 0) = 0$$

$$\delta(A, 1) = A \quad \lambda(A, 1) = 1$$

$$\delta(B, 0) = B \quad \lambda(B, 0) = 1$$

$$\delta(B, 1) = C \quad \lambda(B, 1) = 1$$

$$\delta(C, 0) = A \quad \lambda(C, 0) = 0$$

$$\delta(C, 1) = C \quad \lambda(C, 1) = 0$$

$M_{1.10}$ Mealy makinesine eşdeğer Moore makinesini ($M'_{1.10}$) bulalım.

$$M'_{1.10} = \langle Q', \Sigma, \Delta, \delta', \lambda', q'_0 \rangle$$

$$Q' = \{[A, 0], [A, 1], [B, 0], [B, 1], [C, 0], [C, 1]\}$$

$$q'_0 = [A, 0]$$

$$\delta'([A, 0], 0) = [B, 0] \quad \lambda([A, 0]) = 0$$

$$\delta'([A, 1], 0) = [B, 0] \quad \lambda([A, 1]) = 1$$

$$\delta'([A, 0], 1) = [A, 1] \quad \lambda([B, 0]) = 0$$

$$\delta'([A, 1], 1) = [A, 1] \quad \lambda([B, 1]) = 1$$

$$\delta'([B, 0], 0) = [B, 1] \quad \lambda([C, 0]) = 0$$

$$\delta'([B, 1], 0) = [B, 1] \quad \lambda([C, 1]) = 1$$

$$\delta'([B, 0], 1) = [C, 1]$$

$$\delta'([B, 1], 1) = [C, 1]$$

$$\delta'([C, 0], 0) = [A, 0]$$

$$\delta'([C, 1], 0) = [A, 0]$$

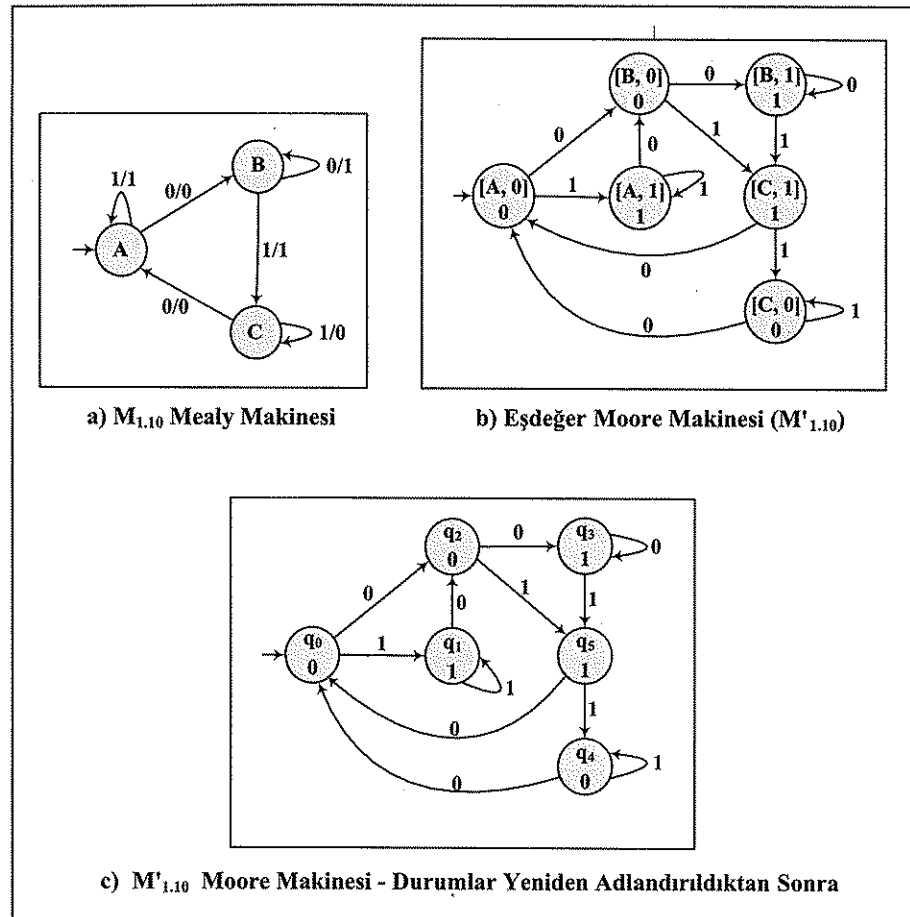
$$\delta'([C, 0], 1) = [C, 0]$$

$$\delta'([C, 1], 1) = [C, 0]$$

Çizim 1.12'de $M_{1.10}$ Mealy makinesi ile bu makineye eşdeğer Moore makinesinin durum çizeneği görülmektedir. Önerme 1.2'ye göre bir Mealy makinesine eşdeğer Moore makinesini bulurken, Moore makinesinin durumlarının [durum adı, çıkış simgesi] ikilileri ile gösterilmesi gerekir. Ancak eşdeğer Moore makinesi bulunduğundan sonra, bu makinenin durumları yeniden adlandırılabilir. Nitekim, $M'_{1.10}$ Moore makinesinin Çizim 1.12.c'de görülen durum çizeneği, makinenin durumları:

[A, 0]	için	q_0
[A, 1]	için	q_1
[B, 0]	için	q_2
[B, 1]	için	q_3
[C, 0]	için	q_4
[C, 1]	için	q_5

adları kullanılarak oluşturulmuştur.



Çizim 1.12. Mealy Makinesine Eşdeğer Moore Makinesinin Bulunması

1.3. Sonlu Özdevinirlerin İndirgenmesi

Sonlu özdevinirlerle ilgili olarak şimdiye kadar birden çok model tanımlandı. Tanımlanan modelleri aşağıdaki gibi özetlemek mümkündür:

1. Tanıyıcı model. Sonlu özdevinirlerin temel modeli olan ve kısaca FA olarak adlandırılan bu modelin deterministik (DFA) ve deterministik olmayan (NFA) türleri vardır.
2. Çıkış üreten özdevinir modeli. Bu modelin de Mealy ve Moore modeli (ya da makinesi) olarak adlandırılan iki türü bulunmaktadır.

Gerek FA modelinde gerekse çıkış üreten sonlu özdevinir modelinde, tanımlanan sonlu özdevinirin (makinenin) karmaşıklığı durum sayısı ile doğru orantılıdır. Bu nedenle, bir sonlu özdevinir verildiğinde, bu sonlu özdevinirin indirgenmesi önem taşıyabilir. Bir sonlu özdevinirin indirgenmesi ya da yalınlaştırılması, bu sonlu özdevinire denk, durum sayısı en küçük sonlu özdevinirin bulunmasıdır. Sonlu özdevinirlerin indirgenmesi DFA, Moore ve Mealy modelleri için görülecektir. Bunun için önce belirli tanımların yapılması gerekir.

1.3.1. Ardıl, Öncel, Denk ve Ayırdedilebilir Durum Tanımları

DFA, Moore ve Mealy modellerinin en genişi Mealy modelidir. Çünkü tanıyıcı (FA) model, Moore modelinin çıkış alfabesi {kabul, red} olan bir alt türü olarak görülebilir. Moore modeli ise, Mealy modelinin, çıkış işlevi giriş alfabesinden bağımsız olan bir alt türü olarak görülebilir. Bu nedenle tanımlar verilirken Mealy modeli esas alınacaktır ve örnek olarak aşağıda tanımlı verilen $M_{1,11}$ makinesi kullanılacaktır.

Örnek 1.11.

Mealy türü $M_{1,11}$ makinesinin giriş ve çıkış alfabeleri {0, 1} alfabetesidir. Başlangıç durumu A olan makinenin geçiş ve çıkış işlevleri aşağıdaki durum çizelgesi ile tanımlanmaktadır.

ŞD	SD, z	
	x = 0	x = 1
→ A	A, 0	D, 1
B	C, 0	E, 1
C	G, 0	E, 1
D	G, 0	F, 1
E	E, 1	C, 0
F	B, 0	D, 1
G	B, 0	E, 1

Ardıl

M makinesinin S_1 durumundan x giriş simgesi ile S_2 durumuna geçiliyorsa:

$$S_1 \xrightarrow{x} S_2$$

S_1 'in x -ardılı S_2 'dir. w bir giriş simgeleri dizgisi olmak üzere, eğer S_1 durumundan w giriş dizgisi ile S_2 durumuna geçiliyorsa, S_1 'in w -ardılı S_2 'dir. Örneğin $M_{1,11}$ makinesinde A durumunun 1-ardılı D 'dir. Aynı makinenin B durumunun 011-ardılı ise C 'dir.

x bir giriş simgesi, w ise giriş simgelerinin bir dizgisi olmak üzere, deterministik modellerde, bir durumun x ve w ardılı her zaman tek bir durumdur. Deterministik olmayan modellerde ise x ve w ardıları, durumların birer altkümesidir. Geçiş çizeneğinde yer aldığı için bir durumun x -ardılı kolaylıkla bulunabilir. Bir durumun $w = x_1x_2 \dots x_k$ ardılını bulmak için ise, geçiş çizeneği kullanılarak önce x_1 -ardılı bulunur; daha sonra x_1 -ardılın x_2 -ardılı, .. vb bulunarak w -ardılı elde edilir. Örneğin $M_{1,3}$ makinesinde q_0 durumunun 0-ardılı $\{q_0, q_1\}$ 'dir. $\{q_0, q_1\}$ yerine q_0q_1 yazılabilir. Buna göre q_0 durumunun 0-ardılı q_0q_1 , 1-ardılı q_0q_2 , 100-ardılı ise $q_0q_1q_3$ 'dür. Aynı makinenin q_1 durumunun 1-ardılı ile q_2 durumunun 01-ardılı ise yoktur (boş kümedir).

Öncel

M makinesinin $S_1, S_2, \dots$ ve S_i durumlarından, x giriş simgesi ile S_j durumuna geçiliyorsa, S_j durumunun x -önceli, $\{S_1, S_2, \dots, S_i\}$ 'dir. Kullanılan model deterministik olsa bile, bir durumun x ve w öncelleri, durumların birer altkümesidir. Örneğin $M_{1,11}$ makinesi için B durumunun 0-önceli FG , 001-önceli ise AEF 'dir. Aynı makine için D durumunun 0 ve 110 öncelleri ise yoktur. Bir makinenin x -öncellerini bulmak için öncel çizelgesi adı verilen bir çizelge oluşturulabilir. $M_{1,11}$ makinesi için oluşturulan öncel çizelgesi aşağıda görülmektedir.

ŞD	Önceki Durum	
	$x = 0$	$x = 1$
A	A	-
B	FG	-
C	B	E
D	-	AF
E	E	BCG
F	-	D
G	CD	-

Denk Durumlar

M makinesi S_1 ve S_2 durumlarından herhangi birinde iken, hangi giriş simgesi uygulanırsa uygulansın makine hep aynı çıkış simgesini üretiyorsa, bu durumlara

1-denk durumlar denir. Örneğin $M_{1,11}$ makinesinin A ve B durumları 1-denktir. C ve F durumları da 1-denktir. Buna karşılık E ve G durumları 1-denk değildir.

M makinesi S_1 ve S_2 durumlarından herhangi birinde iken, uzunluğu n ya da daha kısa olan hangi giriş dizgisi uygulanırsa uygulansın, makine hep aynı çıkış dizgisini üretiyorsa, bu durumlara **n-denk** durumlar denir. Örneğin $M_{1,11}$ makinesinin A ve D durumları 2-denktir. B ve C durumları 3-denktir. B ve C durumları aynı zamanda 4-denktir, 5-denktir, ...vb. Buna karşılık A ve D durumları 3-denk değildir.

M makinesi S_1 ve S_2 durumlarından herhangi birinde iken, uzunluğuna bakılmaksızın, hangi giriş dizgisi uygulanırsa uygulansın, makine hep aynı çıkış dizgisini üretiyorsa, bu durumlara **denk** durumlar denir. Örneğin $M_{1,11}$ makinesinin B ve C durumları denk durumlardır. D ve F durumları da denktir. Ancak 2-denk olan A ve D durumları denk değildir.

Uygulamada, tüm n 'ler için n -denk olan durumlara denk durumlar denir. n ve n 'den küçük tüm k değerleri ($\forall k \leq n$) için k -denk olan iki duruma ise n -denk denir. Örneğin $M_{1,11}$ makinesinin A ve F durumları 2-denk durumlardır. Bu durumların 2-denk olarak nitelenmesi, 3-denk olmadıkları anlamına gelir. Ancak 2-denk olan durumlar doğal olarak 1-denktir. Ancak durum denkliği her zaman en büyük n değeri üzerinden belirtilir.

Ayırdedilebilir Durumlar

M makinesin S_1 ve S_2 durumlarını ayırdetmek için eğer en az n uzunluğunda bir giriş dizgisi gerekli ise bu durumlara **n-ayırdedilebilir** durumlar denir. Eğer S_1 ve S_2 durumları n -ayırdedilebilir ise, bu iki durum $(n-1)$ -denk'tir. Örneğin $M_{1,11}$ makinesinin A ve E durumları 1-ayırdedilebilirdir. B ve D durumları ise 2-ayırdedilebilirdir. Buna karşılık A ve F durumları 3-ayırdedilebilirdir. D ve F durumlarını ise hiçbir giriş dizgisi ile ayırdetmek mümkün değildir. Denk olan D ve F durumları ayırdedilemez durumlardır.

$$A \xrightarrow[Z=0]{X=0} A \quad E \xrightarrow[Z=1]{X=0} E \quad A \text{ ve } E \text{ 1-ayırdedilebilir}$$

$$B \xrightarrow[Z=11]{X=10} E \quad D \xrightarrow[Z=10]{X=10} B \quad B \text{ ve } D \text{ 2-ayırdedilebilir}$$

$$A \xrightarrow[Z=010]{X=010} G \quad F \xrightarrow[Z=011]{X=010} E \quad A \text{ ve } F \text{ 3-ayırdedilebilir}$$

Makine Denkliği

M_1 ve M_2 , giriş ve çıkış alfabeleri aynı olan iki makine olsun. M_1 makinesinin durumlarını $S_{11}, S_{12}, S_{13}, \dots$ olarak; M_2 makinesinin durumlarını ise $S_{21}, S_{22}, S_{23}, \dots$ olarak gösterelim. Eğer M_1 makinesi S_{11} , M_2 makinesi de S_{21} durumundayken, her iki makineye aynı giriş dizgisi uygulandığında, giriş dizgisi ne olursa olsun her iki makine de aynı çıkış dizgisini üretiyorsa, bu iki durum (M_1 makinesinin S_{11} durumu ile M_2 makinesi de S_{21} durumu) denktir.

Eğer M_1 makinesinin her durumu için M_2 makinesinde bu duruma denk bir durum varsa; M_2 makinesinin her durumu için de M_1 makinesinde bu duruma denk bir durum varsa, M_1 ve M_2 makineleri denktir.

$$\left. \begin{array}{l} \forall S_{1i} \in M_1 \rightarrow \exists S_{2j} \in M_2 : S_{1i} \equiv S_{2j} \\ \forall S_{2i} \in M_2 \rightarrow \exists S_{1j} \in M_1 : S_{2i} \equiv S_{1j} \end{array} \right\} M_1 \equiv M_2$$

Makine İndirgeme

Bir M makinesi verildiğinde, bu makinenin indirgenmesi ya da yalınlaştırılması, bu makineye denk makinelerden durum sayısı en küçük olanın bulunması anlamını taşır. Bu makineyi $M_{\min}$ ya da M^* ile gösterebiliriz. Bir makineye denk bir ya da birden çok en küçük makine olabilir.

1.3.2. İndirgeme Yöntemi

Sonlu özdevinirlerin indirgenmesi için denklik bölümleri (*equivalence partitions*) kullanılır. Bir M makinesi için, P_k ile gösterilen k -denklik bölümlemesi, k -denk durumların aynı bölümde yer aldığı bir bölümlenmedir. Örneğin bir Mealy makinesi olan $M_{1,11}$ için 0-denklik bölümlemesi

$$P_0 = (ABCDEFGG)$$

tek bölüm içerir. Çünkü, hiçbir giriş simgesi uygulanmadan, bir Mealy makinesinin durumlarını ayırtmak mümkün değildir. $M_{1,11}$ için 1-denklik bölümlemesi ise

$$P_1 = (ACGGBB)(DEEFDE)$$

iki bölüm içerir. Çünkü makinenin E dışındaki tüm durumları 1-denktir. Makinenin denklik bölümlemesini bulmak için sırasıyla $P_0, P_1, P_2, \dots$ bulunur. Denklik bölümlerinin türetilmesi

$$P_{k+1} = P_k$$

elde edilinceye kadar sürdürülür. $P_{k+1} = P_k$ elde edildiğinde, denklik bölümlemesinin

$$P = P_k$$

olduğu anlaşılır ve türetme kesilir.

P_m denklik bölümlemesinden, P_{m+1} denklik bölümlemesinin türetilmesi için Önerme 1.3'den yararlanılır.

Önerme 1.3.

M makinesinin S_1 ve S_2 durumunun $(m+1)$ -denk olması için aşağıdaki iki koşulun sağlanması gerekli ve yeterlidir.

- S_1 ve S_2 m -denk olmalı (P_m denklik bölümlemesinde aynı bölümde bulunmalı).
- Tüm x giriş simgeleri için S_1 ve S_2 durumlarının x -ardılları da m -denk olmalı (P_m denklik bölümlemesinde aynı bölümde bulunmalı).

1.3.2.1. Mealy Makinelerinin İndirgenmesi

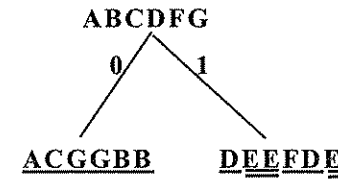
Mealy makinelerinin indirgenmesi $M_{1,11}$ makinesi örnek alınarak incelenecektir. Daha önce belirtildiği gibi, hiçbir giriş uygulanmadan Mealy makinesinin durumları ayırtedilemez. Başka bir deyişle Mealy makinesinin tüm durumları 0-denktir.

$$P_0 = (ABCDEFGG)$$

P_1 denklik bölümlemesini bulmak için durum çizelgesinin incelenmesi yeterlidir. $M_{1,11}$ 'in durum çizelgesinde, E dışındaki tüm durumlar için, x geçişi sırasında üretilen çıkış değerinin birinci kolonda 0, ikinci kolonda ise 1 olduğu görülür. Buna göre E dışındaki durumlar 1-denktir.

$$P_1 = (ACGGBB)(DEEFDE)$$

P_2 denklik bölümlemesini bulmak için Önerme 1.3'den yararlanılır. $M_{1,11}$ makinesinin iki durumunun 2-denk olabilmesi için, hem bu iki durumun, hem de bu iki durumun 0 ve 1-ardıllarının 1-denk olması; bunun için de P_1 denklik bölümlemesinde aynı bölümde bulunması gerekir.



P_1 bölümlemesinde aynı bölümde bulunan $ACGGBB$ durumlarının 0-ardılları, P_1 bölümlemesinde aynı bölümde yer aldığı için, 1-denktir. Ancak bu durumlarının 1-ardıllarının tümü 1-denk değildir. Burdan, ADF durumlarının kendi aralarında, BCG durumlarının da kendi aralarında 2-denk olduğu anlaşılır ve aşağıdaki P_2 denklik bölümlemesi elde edilir:

$$P_2 = (ADF)(BCG)(E)$$

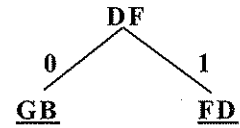
P_3 denklik bölümlemesini bulmak için **ADF** ve **BCG** durumlarının 0 ve 1-ardılları incelenir:



Yapılan incelemede **ADF** durumlarının 1-ardıllarının P_2 'de aynı bölümde yer aldığı, ancak 0-ardıllarının P_2 'de aynı bölümde yer almadığı görülür. Bu nedenle **DF** durumlarının 3-denk olduğu, ancak **A** durumunun **D** ve **F** durumlarına 3-denk olmadığı anlaşılır ve P_3 'de **A** durumu **DF** durumlarından ayrılır. **BCG** durumlarına gelince, bu durumların hem 0 hem de 1-ardılları P_2 'de aynı bölümde yer almaktadır. Buna göre **BCG** durumları 3-denk durumlardır ve P_3 denklik bölümlemesinde aynı bölümde yer alacaklardır. Sonuç olarak P_3 denklik bölümlemesi aşağıdaki gibi oluşur:

$$P_3 = (A)(DF)(BCG)(E)$$

P_4 denklik bölümlemesini bulmak için **DF** ve **BCG** durumlarının 0 ve 1-ardıllarını incelemek gerekir. **BCG** durumlarının, daha önce incelenen ve yukarıda yer alan 0 ve 1-ardılları P_3 'de de aynı bölümde yer almaktadır. Bu nedenle **BCG** durumları P_4 'de de aynı bölümde yer alacaktır.



DF durumlarının ardılları incelendiğinde ise, bu iki durumun 0 ve 1-ardıllarının P_3 'de aynı bölümde yer aldığı görülür. Bu nedenle **DF** durumları P_4 'de aynı bölümde yer alacaktır. Sonuç olarak, P_4 denklik bölümlemesi P_3 'e eşittir.

$$P_4 = P_3 = (A)(DF)(BCG)(E)$$

Böylece $M_{1,11}$ makinesinin denklik bölümlemesi:

$$P = (A)(DF)(BCG)(E)$$

olarak elde edilir. Denklik bölümlemesinde 4 bölüm olduğu için, $M_{1,11}$ 'e denk en küçük makinenin 4 durumu olacaktır. En küçük makinenin durumları:

- A** için S_0 ,
- DF** için S_1 ,
- BCG** için S_2 ,
- E** için S_3

diye adlandırılırsa, $M_{1,11}$ makinesine denk en küçük makinenin durum çizelgesi aşağıdaki gibi bulunur.

ŞD	SD, z	
	x = 0	x = 1
S_0	$S_0, 0$	$S_1, 1$
S_1	$S_2, 0$	$S_1, 1$
S_2	$S_2, 0$	$S_3, 1$
S_3	$S_3, 1$	$S_2, 0$

1.3.2.2. Moore Makinelerinin İndirgenmesi

Bilindiği gibi Moore makinelerinde çıkış işlevi durumlar kümesinden çıkış alfabesine bir eşlemedir. Bu eşlemeyle her duruma bir çıkış simgesi eşlenir. Bir Moore makinesi belirli bir durumda iken belirli bir çıkış simgesi üretir. Bu nedenle hiçbir giriş simgesi uygulanmadan belirli durumlar ya da durum altkümeleri ayırdedilebilir. Başka bir deyişle, Moore makinesinin tüm durumları 0-denk değildir. Moore makinesinin P_0 0-denklik bölümlemesinde, çıkış simgesi kadar bölüm bulunur.

Mealy ve Moore makinelerinin indirgenmesindeki tek fark P_0 denklik bölümlemesinin oluşturulmasıdır. İndirgemenin sonraki adımları benzer biçimde yürütülür. Moore makinelerinin indirgenmesi, örnek bir makine üzerinde görülecektir.

Örnek 1.12.

Moore türü $M_{1,12}$ makinesinin giriş alfabesi $\{0, 1\}$, çıkış alfabesi ise $\{0, 1, 2\}$ alfabesidir. Başlangıç durumu **A** olan makinenin geçiş ve çıkış işlevleri aşağıdaki durum çizelgesi ile tanımlanmaktadır.

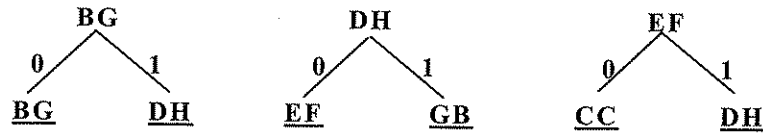
ŞD	SD		z
	x = 0	x = 1	
→ A	C	B	0
B	B	D	1
C	A	H	2
D	E	G	1
E	C	D	2
F	C	H	2
G	G	H	1
H	F	B	1

$M_{1,12}$ makinesi aşağıdaki gibi indirgenir:

$$P_0 = (A)(BDGH)(CEF)$$



$$P_1 = (A)(BG)(DH)(C)(EF)$$



$$P_2 = P_1 = (A)(BG)(DH)(C)(EF)$$

$$P = (A)(BG)(DH)(C)(EF)$$

Denklik bölümlemesinde 5 bölüm olduğu için, $M_{1,12}$ 'ye denk en küçük makinenin 5 durumu olacaktır. En küçük makinenin durumları:

A için S_0 ,

BG için S_1 ,

DH için S_2 ,

C için S_3 ,

EF için S_4

diye adlandırılırsa, $M_{1,12}$ makinesine denk en küçük makinenin durum çizelgesi aşağıdaki gibi bulunur.

ŞD	SD, z		z
	x = 0	x = 1	
→ S_0	S_3	S_1	0
S_1	S_1	S_2	1
S_2	S_4	S_1	1
S_3	S_0	S_2	2
S_4	S_3	S_2	2

1.3.2.3. Deterministik Sonlu Özdevinirlerin (DFA) İndirgenmesi

Bilindiği gibi DFA modeli, Moore makinesinin kısıtlı bir türüdür. Moore modelinde çıkış alfabeti {**kabul**, **red**} gibi ikili bir alfabeyle sınırlanırsa, durumlar çıkış işlevi ile "kabul eden" ve "kabul etmeyen" durumlar olmak üzere ikiye ayrılır ve DFA modeli elde edilir. Buna göre DFA'ların indirgenmesi Moore makinelerinin indirgenmesi ile aynı olacaktır.

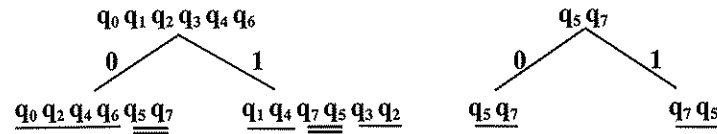
Örnek 1.13.

Deterministik bir özdevinir (DFA) olan $M_{1,13}$ aşağıdaki durum çizelgesi ile tanımlanmaktadır.

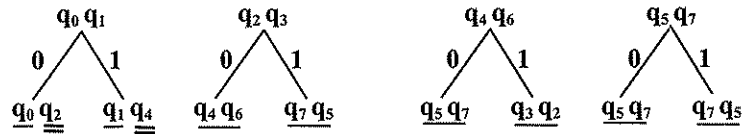
ŞD	SD	
	x = 0	x = 1
→q ₀	q ₀	q ₁
q ₁	q ₂	q ₄
q ₂	q ₄	q ₇
q ₃	q ₆	q ₅
q ₄	q ₅	q ₃
q ₅	q ₅	q ₇
q ₆	q ₇	q ₂
q ₇	q ₇	q ₅

M_{1,13} makinesi aşağıdaki gibi indirgenir:

$$P_0 = (q_0 q_1 q_2 q_3 q_4 q_6)(q_5 q_7)$$



$$P_1 = (q_0 q_1)(q_2 q_3)(q_4 q_6)(q_5 q_7)$$



$$P_2 = (q_0)(q_1)(q_2 q_3)(q_4 q_6)(q_5 q_7)$$

$$P_3 = P_2$$

$$P = (q_0)(q_1)(q_2 q_3)(q_4 q_6)(q_5 q_7)$$

Denklik bölümlemesinde 5 bölüm olduğu için, M_{1,13}'e denk en küçük makinenin 5 durumu olacaktır. En küçük makinenin durumları:

q₀ için S₀,

q₁ için S₁,

q₂ q₃ için S₂,

q₄ q₆ için S₃,

q₅ q₇ için S₄

diye adlandırılırsa, M_{1,13} makinesine denk en küçük makinenin durum çizelgesi aşağıdaki gibi bulunur.

ŞD	SD	
	x = 0	x = 1
→S ₀	S ₀	S ₁
S ₁	S ₂	S ₃
S ₂	S ₃	S ₄
S ₃	S ₄	S ₂
S ₄	S ₄	S ₄

Sorular

S.1.1. Aşağıda sözel olarak tanımlanan her küme için, kümeyi tanıyan bir sonlu özdevinirin (NFA) geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz.

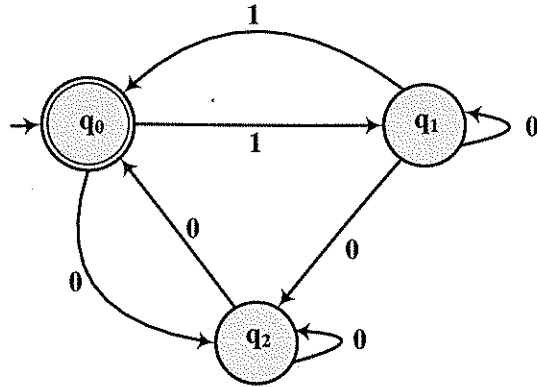
- a) $\{a, b, c\}$ alfabesinde, uzunluğu dördün katı (0, 4, 8, 12, ...) olan ve ilk dört, ikinci dört, üçüncü dört, .. simgeden biri ve yalnız biri **b** olan dizgiler kümesi.

$$L_{S.1.1.1} = \{\lambda, acbc, abac, caab, \dots, abccccab, baaacbaabca, \dots\}$$

- b) $\{1, 2, 3\}$ alfabesinde, rakamların toplamı altının katları (0, 6, 12, 18, ...) olan dizgiler kümesi.

$$L_{S.1.1.2} = \{\lambda, 33, 222, 11121, 123, 3111, 233121, 1122321, 32321331, \dots\}$$

S.1.2. Aşağıda geçiş çizeneği verilen NFA'ya denk bir DFA bulunuz. DFA'nın durumlarını $S_0, S_1, S_2, \dots$ diye adlandırarak geçiş çizelgesi ve geçiş çizeneğini oluşturunuz.



S.1.3. $L_{S.1.3}$ dili $\{a, b, c\}$ alfabesinde, içindeki her **cd** altdizgisinden sonra en az bir **a**, içindeki her **dc** altdizgisinden sonra da en az bir **b** bulunan dizgiler kümesi olarak tanımlanıyor. $L_{S.1.3}$ 'de yer alan dizgilerden birkaç örnek aşağıda görülmektedir.

$$L_{S.1.3} = \{\lambda, a, bc, abc, ccdab, dadcbbadcbba, bcab, abcdaddcbaddcb, \dots\}$$

- a) $L_{S.1.3}$ 'ü tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği olabildiğince az durum içersin ve λ -geçişleri içermesin.
- b) $L_{S.1.3}$ 'ü tanıyan DFA'nın geçiş çizeneğini bulunuz.

S.1.4. $L_{S.1.4}$ dili, $\{a, b, c, d\}$ alfabesinde, içindeki her **abc** altdizgisinden sonra en az 2 tane **d** bulunan dizgiler kümesi olarak tanımlanıyor. $L_{S.1.4}$ 'de yer alan dizgilerden birkaç örnek aşağıda görülmektedir.

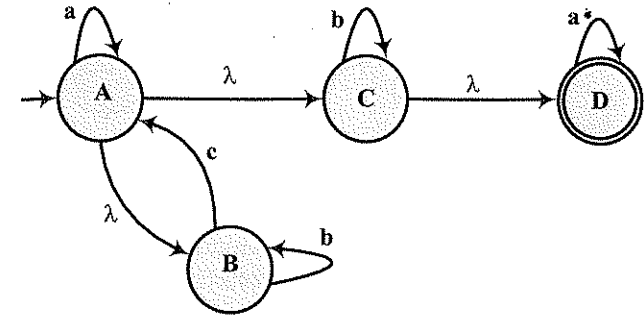
$$L_{S.1.4} = \{\lambda, a, bc, abbd, cda, dabcd, bcabb, ababccccadab, \dots\}$$

- a) $L_{S.1.4}$ 'ü tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği olabildiğince az durum içersin ve λ -geçişleri içermesin.
- b) $L_{S.1.4}$ 'ü tanıyan DFA'nın geçiş çizeneğini bulunuz.

S.1.5. $L_{S.1.5}$ dili $\{a, b, c\}$ alfabesinde içinde **ac** ve **cb** altdizgilerinden en az biri, en az bir kez bulunan dizgiler kümesi olarak tanımlanıyor.

- a) $L_{S.1.5}$ dilini tanıyan FA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişleri içermesin.
- b) Oluşturduğunuz geçiş çizeneği deterministik değilse (NFA), eşdeğer DFA'nın geçiş çizeneğini bulunuz.
- c) Bulduğunuz DFA'yı, eğer denk durumlar içeriyorsa, indirgeyiniz. Nedenini belirterek indirgemeyi sezgisel olarak yapabilirsiniz.

S.1.6.

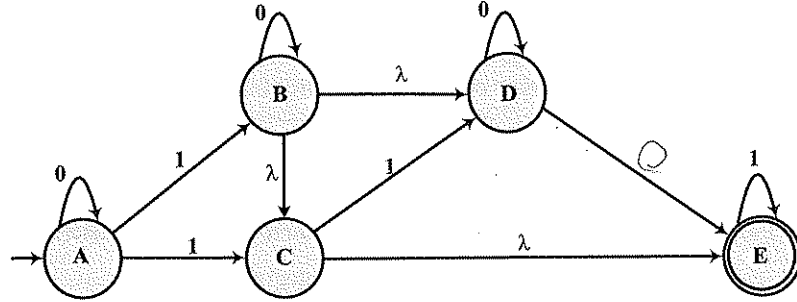


Yukarıda geçiş çizeneği verilen NFA'ya denk en küçük DFA'yı bulmanız isteniyor. Bunun için:

- a) Önce λ -geçişlerini yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz. λ -geçişsiz geçiş çizeneği için geçiş çizelgesini oluşturunuz.
- b) Eşdeğer DFA'yı bulunuz.

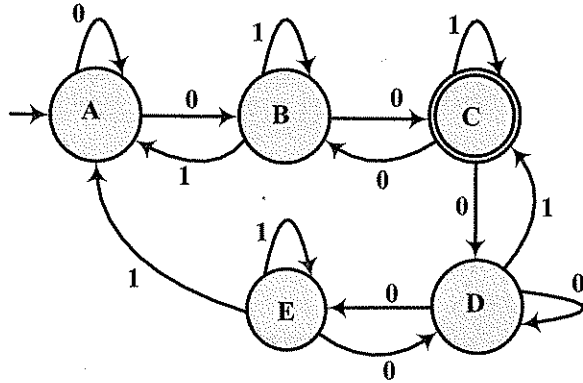
- c) Bulduğunuz DFA denk durumlar içeriyorsa, indirgeyerek en küçük DFA'yı bulunuz.

S.1.7.



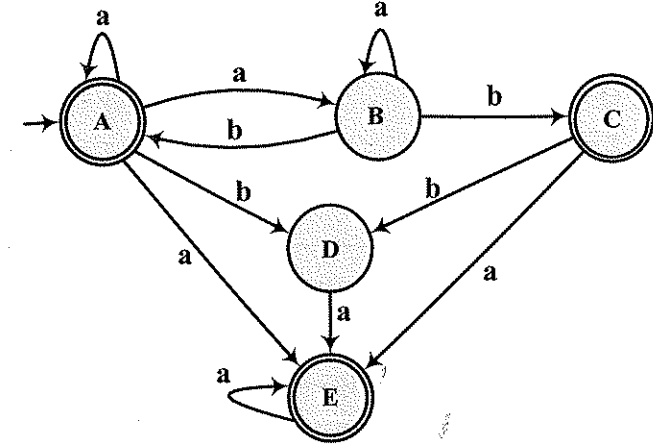
Yukarıdaki geçiş çizeneğindeki λ -geçişlerini tek tek yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz. λ -geçişsiz geçiş çizeneği için geçiş çizelgesini oluşturunuz.

S.1.8.



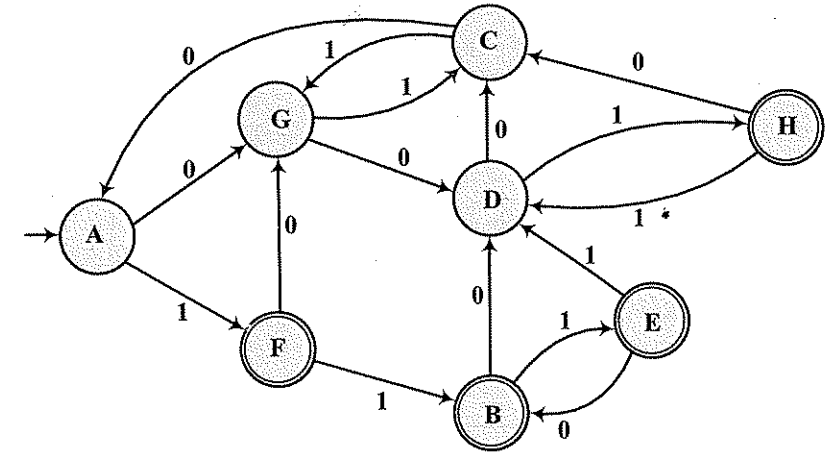
Geçiş çizeneği verilen NFA'ya denk DFA'yı sistematik yöntemle bulmanız isteniyor. Bulduğunuz DFA'nın durumlarını $S_0, S_1, \dots$ diye adlandırdıktan sonra geçiş çizeneğini ve geçiş çizelgesini oluşturunuz.

S.1.9.



Geçiş çizeneği verilen NFA'ya denk DFA'yı sistematik yöntemle bulmanız isteniyor. Bulduğunuz DFA'nın durumlarını $S_0, S_1, \dots$ diye adlandırdıktan sonra geçiş çizeneğini ve geçiş çizelgesini oluşturunuz.

S.1.10.



Yukarıda geçiş çizeneği verilen DFA'yı indirgeyiniz. İndirgenmiş DFA'nın durumlarını $q_0, q_1, q_2, \dots$ diye adlandırarak önce standart biçimdeki geçiş çizelgesini, sonra da geçiş çizeneğini oluşturunuz.

Not: Bir geçiş çizelgesinin standart biçimde olabilmesi için, çizelgenin satırları yukarıdan aşağıya, her satır da soldan sağa tarandığında, durum adlarının ilk geçişlerinin büyüyen sırada olması gerekir.

S.1.11. Aşağıda iki yönlü deterministik bir sonlu özdevinirin (2DFA) tanımı verilmektedir.

$$M_{S.1.11} = \langle Q, \Sigma, \delta, q_0, F \rangle$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_3\}$$

	0	1
$\rightarrow q_0$	(q_1, R)	(q_2, R)
q_1	(q_3, L)	(q_2, L)
q_2	(q_2, R)	(q_3, R)
$\odot q_3$	(q_1, R)	(q_2, L)

Anlık tanım (ID) dizilerini inceleyerek aşağıdaki tümcelerin 2DFA tarafından tanınıp tanınmadığını bulunuz.

- a) $w_1 = 11010$
b) $w_2 = 100101$
c) $w_3 = 011010$

S.1.12. Aşağıda iki yönlü deterministik bir sonlu özdevinirin (2DFA) tanımı verilmektedir.

$$M_{S.1.12} = \langle Q, \Sigma, \delta, q_0, F \rangle$$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_2\}$$

	0	1
$\rightarrow q_0$	(q_0, R)	(q_1, R)
q_1	(q_1, R)	(q_2, R)
$\odot q_2$	(q_2, R)	(q_3, L)
q_3	(q_4, L)	(q_3, L)
q_4	(q_0, R)	(q_4, L)

Anlık tanım (ID) dizilerini inceleyerek aşağıdaki tümcelerin 2DFA tarafından tanınıp tanınmadığını bulunuz.

- a) $w_1 = 10011$
b) $w_2 = 01011$
c) $w_3 = 11011$
d) $w_4 = 10101$

S.1.13. Aşağıda durum çizelgesi verilen Moore makinesine eşdeğer Mealy makinesinin durum çizeneği ve çizelgesini bulunuz.

SD	SD		z
	x = 0	x = 1	
q_0	q_1	q_2	0
q_1	q_3	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	1

S.1.14. Aşağıda durum çizelgesi verilen Mealy makinesine eşdeğer Moore makinesinin durum çizeneği ve çizelgesini bulunuz. Moore makinesinin durumlarını $S_0, S_1, S_2, \dots$ diye adlandırınız. Eğer Moore makinesinin denk durumları varsa, makineyi indirgeyiniz.

SD	SD, z	
	x = 0	x = 1
$\rightarrow A$	A, 0	B, 0
B	C, 0	B, 1
C	C, 1	D, 1
D	A, 1	D, 0

S.1.15. Aşağıda durum çizelgesi verilen Mealy makinesini indirgeyiniz.

ŞD	SD, z	
	x = 0	x = 1
A	C, 1	F, 0
B	H, 1	F, 0
C	E, 0	F, 1
D	F, 1	C, 0
E	F, 1	C, 0
F	D, 0	C, 1
G	H, 1	C, 0
H	E, 0	J, 0
J	G, 0	J, 1

S.1.16. Aşağıdaki geçiş çizelgesi ile tanımlanan DFA'yı indirgeyiniz.

	0	1
→A	A	E
B	C	E
C	D	G
D	F	G
E	F	B
F	C	G
ⓐ	G	D

S.1.17. Aşağıda geçiş çizeneği verilen Mealy makinesini indirgeyiniz. İndirgenmiş makinenin önce geçiş çizelgesini, sonra da geçiş çizeneğini oluşturunuz.

ŞD	SD, z	
	x = 0	x = 1
A	A, 1	F, 0
B	F, 1	C, 0
C	G, 0	C, 1
D	E, 0	H, 1
E	F, 0	A, 1
F	B, 1	C, 0
G	E, 0	D, 1
H	E, 0	G, 1

Bölüm 2

Düzcün Kümeler ve Düzcün Deyimler

2.1. Düzcün Kümeler

Sonlu özdevinir (*finite automata* : FA) modeli, giriş alfabesindeki simgelerden oluşan dizgilerin bir kısmını tanıyan, bir kısmını ise tanımayan bir modeldir. Her sonlu özdevinirin tanıdığı bir dizgiler kümesi vardır. Bu küme, içerdığı dizgiler tek tek yazılarak, ya da içerdığı dizgilerin özellikleri belirtilerek tanımlanabilir. Sonlu özdevinirlerin tanıdığı kümelerin tanımları sözel olarak ya da matematiksel gösterimler kullanılarak yapılabilir. Örneğin $\{0, 1\}$ alfabesinde, sonlu bir özdevinir tarafından tanınan aşağıdaki kümeleri tanımlayabiliriz:

$$P_1 = \{1001, 0110\}$$

P_2 : İçinde ardarda en az üç tane 1 bulunan (ya da 111 alt dizgisini içeren) dizgiler kümesi

P_3 : İlk ve son simgesi 1 olan, en çok 5 uzunluğundaki dizgiler kümesi

$$P_4 = \{0^n 1^m \mid n \geq 1, m \geq 1\}$$

Sonlu özdevinirler tarafından tanınan kümelere **düzcün kümeler** (*regular sets*) denir. Bir küme verildiğinde, bu kümenin düzcün bir küme olup/olmadığı, bu kümeyi tanıyan sonlu bir özdevinirin bulunup/bulunmaması tarafından belirlenir. Eğer kümeyi tanıyan en az bir sonlu özdevinir varsa, bu küme düzcün bir kümedir. Eğer verilen kümeyi tanıyan hiçbir sonlu özdevinir yoksa, bu küme düzcün bir küme değildir. Yukarıda tanımlanan P_1 , P_2 , P_3 ve P_4 kümeleri düzcün kümelere. Çünkü bu kümelerin her birini tanıyan en az bir sonlu özdevinir oluşturulabilir.

$$P_5 = \{0^n 1^n \mid n \geq 1\}$$

P_6 : İçinde eşit sayıda 0 ve 1 bulunan dizgiler kümesi

P_5 ve P_6 kümelerinin her ikisi de düzgün değildir. Çünkü bu kümeleri tanıyan sonlu özdevinirler oluşturulamaz. Tanımlanan bir kümenin düzgün olup olmadığını bulmak için, kümeyi tanıyan bir sonlu özdevinir oluşturulmaya çalışılır. Eğer bunda başarılı olunur ve kümeyi tanıyan sonlu bir özdevinir oluşturulabilirse, kümenin düzgün bir küme olduğu anlaşılır. Bir başka yöntem ise, kümeyi düzgün bir deyimle göstermeye çalışmaktır. Bu konu bir sonraki kesimde incelenecektir (bkz 2.2.). Biçimsel olmayan bir başka yöntem ise, tanımlanan dizgileri tanımak için sonlu özdevinirin sahip olması gereken durum (bellek) miktarını irdelemektir. Örnek olarak P_5 'i tanıyacak bir makineyi (M_5) düşünelim. Bu makinenin, dizginin başındaki 0'ların sayısını saklaması ve 1'lerin sayısının 0'ların sayısına eşit olduğunu denetlemesi gerekir. Küme tanımında, kümedeki dizgilerin n tane 0'dan sonra n tane 1 içerdiği belirtilmekte ve n'nin birden büyük olduğu belirtilmektedir. Ancak n'nin üst sınırı yoktur. n çok büyük bir sayı olabilir. Bu nedenle, sonlu sayıda durumu bulunan, bu nedenle sonlu saklama kapasitesine sahip bir FA ile P_5 'i tanımak mümkün değildir. Başka bir deyişle, P_5 kümesini tanıyan bir makine (M_5) varsa, bu makine sonlu bir özdevinir değildir. Benzer biçimde, P_6 'yı tanıyacak bir makinenin (M_6), dizgiyi taraması ve sürekli olarak, o ana kadar raslanan 0'larla 1'lerin sayılarının farkını saklaması gerekir. Eğer dizginin sonunda, 0'larla 1'lerin sayılarının farkı sıfırsa, dizgi M_6 tarafından tanınacaktır. Ancak 0'larla 1'lerin sayılarının farkının bir üst sınırı yoktur. Bu fark, düşümlenebilecek tüm sonlu sayılardan daha büyük olabilir. Bu nedenle de M_6 bir sonlu özdevinir olamaz. P_5 ve P_6 kümelerini tanıyan sonlu özdevinirler bulunmadığı için, bu kümeler düzgün kümeler değildir.

2.2. Düzgün Deyimler

Düzgün deyimler, düzgün kümeleri biçimsel olarak tanımlamak için kullanılan bir anlatım aracı, bir dildir. Her düzgün deyim, belirli bir alfabe'deki simgelerden oluşturulan dizgilerin bir altkümelerini tanımlar.

Tanım 2.1.

$\{a, b, c, \dots\}$ alfabe'sindeki düzgün deyimler özyineli olarak aşağıdaki gibi tanımlanır:

1. Alfabe'deki her simge bir düzgün deyimdir.

$$a = \{a\},$$

$$b = \{b\}, \dots$$

2. λ ve Φ birer düzgün deyimdir.

$$\lambda = \{\lambda\}$$

$$\Phi = \{\}$$

3. Eğer P ve Q birer düzgün deyimse:

$$P+Q \quad PQ \quad P^*$$

da birer düzgün deyimdir.

Burada üç tane küme işleci tanımlanmaktadır:

$$+ \text{ küme birleşim işlemidir : } P+Q = P \cup Q$$

. (ya da boşluk) ardarda ekleme (concatenation) işlemidir:

$$PQ = \{pq \mid p \in P, q \in Q\}$$

* kapanış (closure) işlemidir. $P^* = \lambda + P + PP + PPP + PPPP + \dots$

4. Yukarıdaki kuralların sonlu sayıda uygulanması ile oluşturulan deyimler düzgün deyimlerdir.

Örnekler:

Aşağıda $\{a, b, c, d\}$ ve $\{0, 1\}$ alfabe'lerinde oluşturulmuş düzgün deyim örnekleri ve bunlara karşı gelen kümeler görülmektedir.

Düzgün Deyim	Tanımladığı Küme
Φ	$\{\}$
λ	$\{\lambda\}$
$00+11$	$\{00, 11\}$
$a(b+c)$	$\{ab, ac\}$
ab^*	$\{a, ab, abb, abbb, abbbb, \dots\}$
$a(bb+cc)d^*$	$\{abb, abbd, abbdd, \dots, acc, accd, accdd, \dots\}$
a^*	$\{\lambda, a, aa, aaa, aaaa, \dots\}$
$(0+1)^*$	$\{\lambda, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, \dots\}$
$a(b+cd)^*a$	$\{aa, aba, acda, acdda, acddda, abca, acba, \dots\}$

2.2.1. Düzgün Deyimlere Karşı Gelen Sonlu Özdevinirlerin Bulunması

Önerme 2.1. Düzgün deyimlerle tanımlanan kümeler düzgün kümelerdir. Bu önermeye göre, bir düzgün deyim verildiğinde, bu düzgün deyimle tanımlanan kümeyi tanıyan bir sonlu özdevinir bulunabilir. Bu önerme, düzgün deyimlere karşı gelen sonlu özdevinirlerin geçiş çizeneklerinin nasıl bulunacağı gösterilerek ispat edilmeye çalışılacaktır. Bunun için de, düzgün deyimlerin özyineli tanımında yer alan her bir madde için ilgili geçiş çizeneğinin nasıl oluşturulacağı gösterilecektir.

$P_1 = a_i$ düzgün deyimi ile tanımlanan küme tek bir giriş simgesi içermektedir. Bu kümeyi tanıyan sonlu özdevinirin geçiş çizeneği Çizim 2.1.a'daki gibi oluşturulabilir.

$P_2 = \lambda$ düzgün deyimi ile tanımlanan küme yalnız boş simgeyi (λ) içeren kümedir. Bu kümeyi tanıyan sonlu özdevinirin geçiş çizeneği Çizim 2.1.b'deki gibi oluşturulabilir.

$P_3 = \Phi$ düzgün deyimi ile tanımlanan küme boş kümedir. Bu kümeyi tanıyan sonlu özdevinirin geçiş çizeneği Çizim 2.1.c'deki gibi oluşturulabilir.

P ve Q birer düzgün deyim olsun. Bu düzgün deyimlerle tanımlanan kümeleri tanıyan geçiş çizeneklerinin bilindiğini varayalım. Eğer P ve Q 'yu tanıyan geçiş çizeneklerinde birden çok başlangıç ve/ya da uç durum varsa, Çizim 2.1.d ve 2.1.e'de görüldüğü gibi, bu geçiş çizeneklerini bir başlangıç durumu, bir de uç durumu bulunan eşdeğer çizeneklere dönüştürebiliriz. Bunun için, eğer çizenekte birden çok başlangıç durumu varsa, çizeneğe yeni bir durum (q_0) eklenerek tek başlangıç durumu bu yeni durum yapılır ve bu yeni durum ile eski başlangıç durumları arasında birer λ -geçiş konulur. Benzer biçimde, eğer çizenekte birden çok uç durumu varsa, çizeneğe yeni bir uç durum (q_1) eklenerek tek uç durum bu yeni durum yapılır ve eski uç durumlar ile bu yeni durum arasında birer λ -geçiş konulur.

Eğer P ve Q birer düzgün deyimse ve bu düzgün deyimleri tanıyan tek başlangıç, tek de uç durumu bulunan geçiş çizenekleri biliniyorsa:

PQ

$P+Q$

P^*

düzgün deyimlerini tanıyan geçiş çizeneklerini Çizim 2.1.f, 2.1.g ve 2.1.h'da görüldüğü gibi oluşturabiliriz.

Yukarıdaki açıklamalar, bir düzgün deyim verildiğinde, bu düzgün deyimi tanıyan geçiş çizeneğinin sistematik biçimde oluşturulabileceğini göstermektedir. Her geçiş çizeneği ile bir sonlu özdevinir (FA) tanımlandığına ve sonlu özdevinirlerin tanıdığı kümeler düzgün kümeler olduğuna göre, "düzgün deyimlerle tanımlanan kümeler düzgün kümelerdir" önermesi (Önerme 2.1) ispatlanmış olmaktadır.

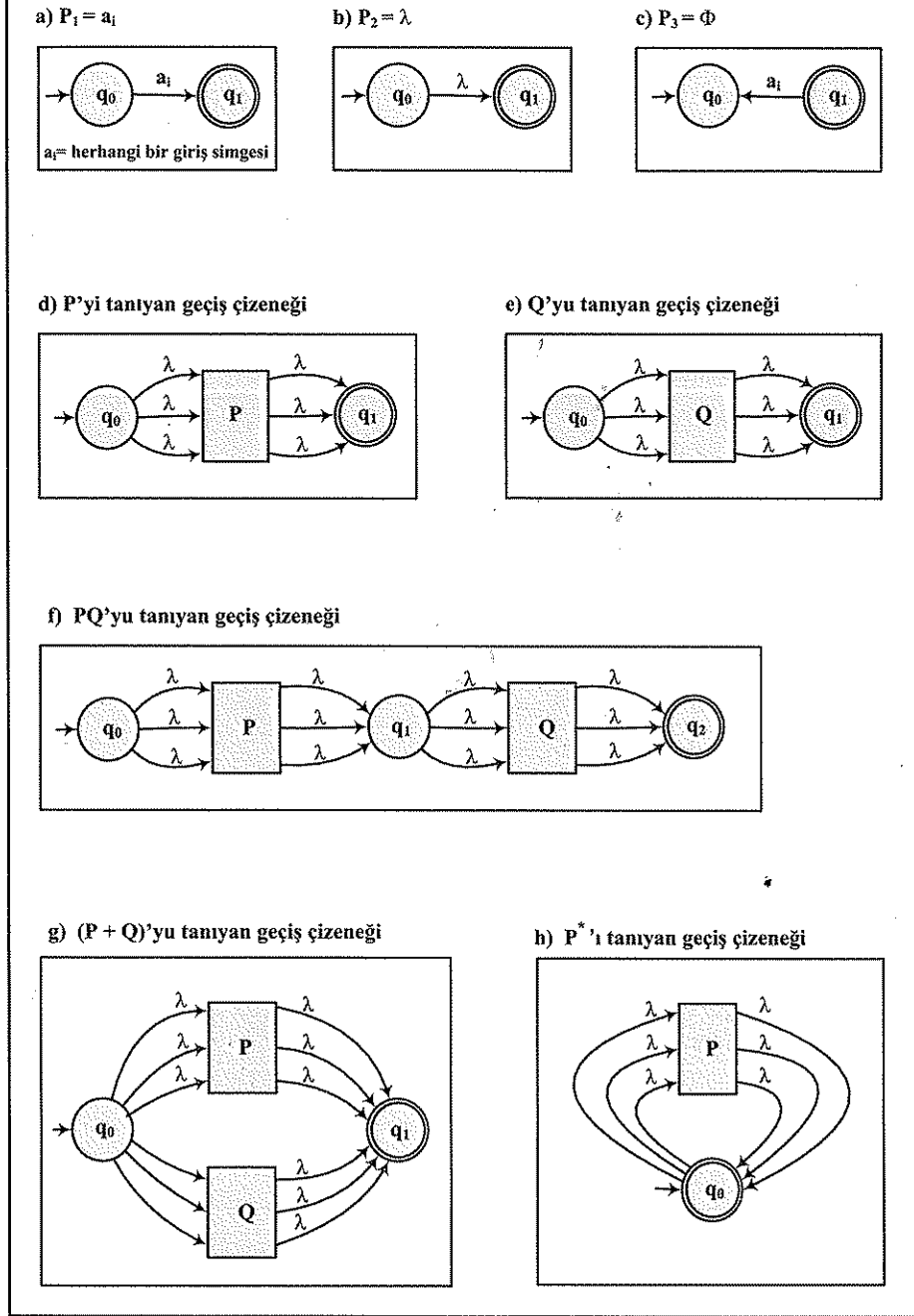
Verilen bir düzgün deyimi tanıyan geçiş çizeneğinin biçimsel yaklaşımla oluşturulması çok uzun olabilir ve bu yöntemle oluşturulan geçiş çizeneği çok sayıda λ -geçiş içerebilir. Örneğin

$$P = (a + bc^*)^*$$

düzgün deyimini tanıyan sonlu özdevinirin geçiş çizeneğini biçimsel yaklaşımla oluşturmak istediğimizde,

önce a 'yı, b 'yi ve c 'yi tanıyan çizenekleri,

sonra c^* 'i tanıyan çizeneği,



Çizim 2.1. Düzgün Deyimlere Karşı Gelen Geçiş Çizeneklerinin Biçimsel Yaklaşımla Oluşturulması

sonra bc^* 'i tanıyan çizeneği,

sonra $(a+bc^*)$ 'i tanıyan çizeneği,

son olarak da $(a+bc^*)^*$ 'i tanıyan çizeneği

oluşturmak gerekir (Çizim 2.2.a). Oysa P 'yi tanıyan geçiş çizeneğini sezgisel yaklaşımla çok daha çabuk oluşturmak mümkündür (Çizim 2.2.b). Çizimde görüldüğü gibi sezgisel yaklaşımla oluşturulan geçiş çizeneği, biçimsel yaklaşımla oluşturulan geçiş çizeneğine oranla çok daha az λ -geçişi içeren, çok daha yalın bir çizenedir.

Çizim 2.3'de düzgün deyimleri tanıyan geçiş çizeneklerinin sezgisel yaklaşımla oluşturulmasına ilişkin bir dizi örnek yer almaktadır. Geçiş çizeneklerinin sezgisel yaklaşımla oluşturulması sırasında dikkat edilmesi gereken önemli hususlardan birkaçı aşağıda yer almaktadır.

1. Eğer düzgün deyimde:

$\dots + a^*b \dots$ ya da
 $(a^*b \dots)^*$

gibi bir örüntü varsa, üzerinde a döngüsü bulunan düğüme λ -geçişi ile gelinmelidir.

2. Eğer düzgün deyimde:

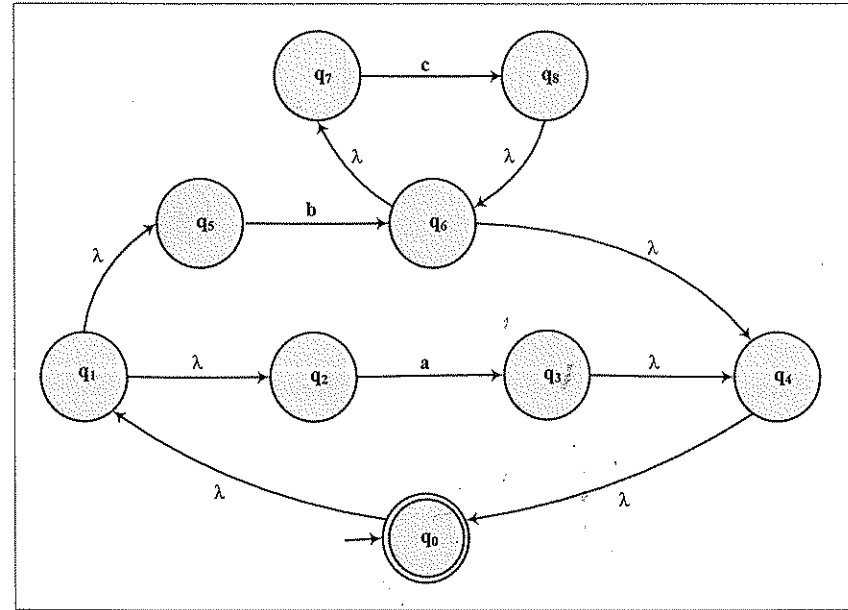
$\dots ab^* + \dots$ ya da
 $\dots ab^*)^*$

gibi bir örüntü varsa, üzerinde b döngüsü bulunan düğümden λ -geçişi ile çıkılmalıdır.

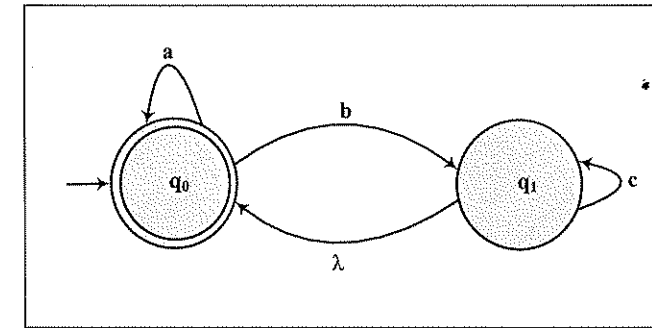
3. Eğer düzgün deyimde:

$\dots a^*b^* \dots$

gibi bir örüntü varsa, üzerinde a döngüsü bulunan düğümden, üzerinde b döngüsü bulunan düğüme λ -geçişi ile geçilmelidir.

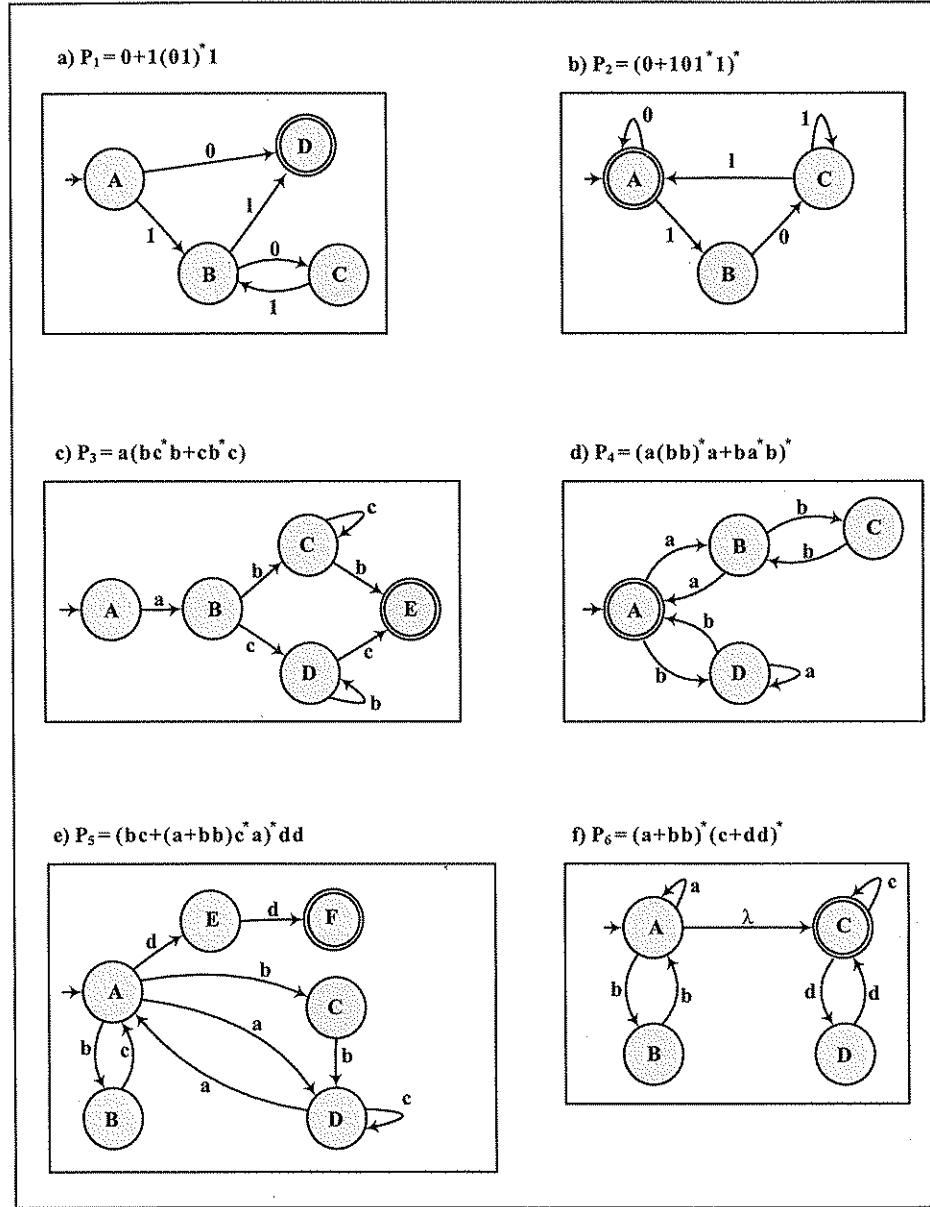


a) Biçimsel Yaklaşımla Oluşturulan Geçiş Çizeneği



b) Sezgisel Yaklaşımla Oluşturulan Geçiş Çizeneği

Çizim 2.2. $P = (a + bc^*)^*$ Düzgün Deyimini Tanıyan FA'nın Geçiş Çizeneğinin Biçimsel ve Sezgisel Yaklaşımla Oluşturulması



Çizim 2.3. Düzgün Deyimlere Karşı Gelen Geçiş Çizeneklerinin Sezgisel Yaklaşım ile Oluşturulması

2.2.2. Sonlu Özdevinirlerin Tanıdığı Kümelerin Birer Düzgün Deyim Olarak Bulunması

Önerme 2.2. Her düzgün küme bir düzgün deyimle gösterilebilir. Düzgün kümeler, sonlu özdevinirler tarafından tanınan kümeler olduğu için, bu önermeye göre, verilen bir sonlu özdevinirin tanıdığı düzgün kümeyi bir düzgün deyimle göstermek mümkündür.

Bu önermenin ispatı için, verilen bir sonlu özdevinire karşı gelen düzgün deyim nasıl elde edileceği gösterilecektir. Düzgün deyim elde edilmesi için kullanılacak yöntem, denklem sistemi çözmeye dayalı olacaktır. Denklem sisteminin nasıl kurulacağı ve çözüleceğini örnekler üzerinde göstermeden önce, denklem sisteminin çözülmesinde yararlanılacak bir özelliği tanımlamak gereklidir.

Önerme 2.3. P , Q ve R aynı alfabede tanımlanmış düzgün deyimler ise ve $P \lambda$ 'yı içermiyorsa:

$$R = Q + RP$$

denkleminin tek çözümü $R = QP^*$ dir.

Denklem sistemlerinin çözümünde kullanılacak bu önermenin ispatı verilmeyecektir.

Tek başlangıç durumu bulunan bir geçiş çizeneği verildiğinde, her durum için aynı adı taşıyan bir değişken (küme değişkeni) tanımlanır. A durumu için tanımlanan A değişkeni, başlangıç durumundan başlayıp A durumunda son bulan tüm dizgileri içeren bir kümeye karşı gelir. Durumlar arası geçişler dikkate alınarak, her A durumu için, sol tarafında bu duruma karşı gelen değişken; sağ tarafında ise, A durumunda son bulan her

$$\delta(B, a_i) = A$$

geçışı için Ba_i çarpımının yer aldığı

$$A = \sum B a_i$$

biçiminde bir denklem kurulur. Daha sonra da, Önerme 2.3'den yararlanılarak denklem sistemi çözülür ve her değişken için bir düzgün deyim elde edilir. Sonlu özdevinirin tanıdığı küme, uç durumlara karşı gelen düzgün deyimlerin küme birleşimi (+) ile gösterilir.

Sonlu özdevinirlere karşı gelen düzgün deyimlerin nasıl bulunacağı iki örnek üzerinde gösterilecektir.

Örnek 2.1. Durum çizeneği Çizim 2.4.a'da görülen sonlu özdevinirin tanıdığı kümeyi bir düzgün deyim olarak bulalım. Makinenin A ve B adlı iki durumu bulunduğu için iki değişkenli aşağıdaki denklem sistemi kurulur:

$$A = \lambda + A0 + B1 \quad (1)$$

$$B = A0 \quad (2)$$

Denklem (1)'de B'nin yerine denklem (2)'deki A0 değeri konulduğunda:

$$A = \lambda + A0 + A01 = \lambda + A(0+01) \quad (3)$$

elde edilir. Denklem (3), Önerme 2.3 uygulanarak çözüldüğünde A'ya karşı gelen aşağıdaki düzgün deyim elde edilir:

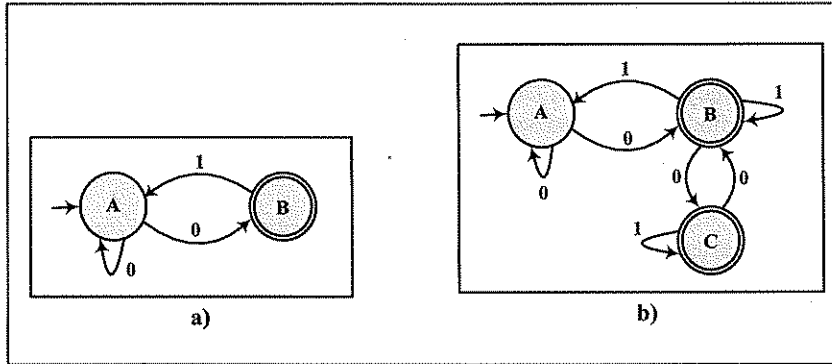
$$A = \lambda(0+01)^* = (0+01)^* \quad (4)$$

Denklem (2)'de A'nın yerine (4)'de elde edilen düzgün deyim konulduğunda, B'ye karşı gelen aşağıdaki düzgün deyim elde edilir:

$$B = A0 = (0+01)^*0 \quad (5)$$

Makinenin tek uç durumu B olduğu için, aşağıdaki düzgün deyim makinenin tanıdığı kümeye karşı gelir:

$$T(M_{2.1}) = (0+01)^*0$$



Çizim 2.4. Örnek 2.1 ve Örnek 2.2'deki Makinelerin Geçiş Çizenekleri

Örnek 2.2. Durum çizeneği Çizim 2.4.b'de görülen sonlu özdevinirin tanıdığı kümeyi bir düzgün deyim olarak bulalım. Makinenin A, B ve C adlı üç durumu bulunduğu için üç değişkenli aşağıdaki denklem sistemi kurulur:

$$A = \lambda + A0 + B1 \quad (1)$$

$$B = A0 + B1 + C0 \quad (2)$$

$$C = B0 + C1 \quad (3)$$

Üçüncü denklem Önerme 2.3 uygulandığında:

$$C = B01^* \quad (4)$$

elde edilir. İkinci denklemde C'nin yerine B01* değeri konularak da:

$$B = A0 + B1 + B01^*0 = A0 + B(1+01^*0) \quad (5)$$

elde edilir. Önerme 2.3 kullanılarak yukarıdaki denklem çözüldüğünde ise:

$$B = A0(1+01^*0)^* \quad (6)$$

elde edilir. Birinci denklemde B'nin yerine yukarıdaki değer konulduğunda ise birinci denklem aşağıdaki biçime dönüşür:

$$A = \lambda + A0 + A0(1+01^*0)^*1 = \lambda + A(0+0(1+01^*0)^*1) \quad (7)$$

Önerme 2.3 kullanılarak yukarıdaki denklem çözüldüğünde ise A'ya karşı gelen aşağıdaki düzgün deyim elde edilir:

$$A = \lambda(0+0(1+01^*0)^*1)^* = (0+0(1+01^*0)^*1)^* \quad (8)$$

Denklem (6)'da A'nın yerine (8)'de bulunan değeri konularak B'ye karşı gelen aşağıdaki düzgün deyim elde edilir.

$$B = (00(1+01^*0)^*1)^*0(1+01^*0)^* \quad (9)$$

Son olarak da denklem (4)'de B'nin yerine (9)'da bulunan değeri konularak C'ye karşı gelen düzgün deyim elde edilir.

$$C = (0+0(1+01^*0)^*1)^*0(1+01^*0)^*01^*$$

Denklem sistemi çözümlü, her bir duruma karşı gelen düzgün deyim elde edildikten sonra, makinenin tanıdığı kümeye karşı gelen düzgün deyim aşağıdaki gibi elde edilir.

$$T(M_{2.2}) = B + C = B + B01^* = B(\lambda + 01^*)$$

$$T(M_{2.2}) = (0+0(1+01^*0)^*1)^*0(1+01^*0)^*(\lambda + 01^*)$$

Sorular

S.2.1. $L_{S.2.1} = ab^*(a+c^*d^*c)b$

- $L_{S.2.1}$ 'i tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişleri içerebilir.
- Eğer oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri tek tek yok ederek λ -geçiş içermeyen denk geçiş çizeneğini bulunuz.

S.2.2. Sonlu durumlu $M_{S.2.2}$ makinesi $\{0, 1\}$ alfabesinde **11**, **101** ya da **110** ile biten dizgileri tanıyan makine olarak tanımlanıyor.

- $M_{S.2.2}$ 'nin tanıdığı düzgün dili ($L_{S.2.2}$) bir düzgün deyim olarak yazınız.
- $M_{S.2.2}$ 'nin deterministik olmayan (NFA) geçiş çizeneğini olabildiğince az durum kullanarak oluşturunuz.
- Durumları $S_0, S_1, S_2, \dots$ diye adlandırarak $M_{S.2.2}$ 'nin deterministik (DFA) geçiş çizelgesi ile geçiş çizeneğini oluşturunuz.
- Makinenin denklik bölümlmesini bulunuz. Eğer oluşturduğunuz makinenin denk durumları varsa, eşdeğer en küçük makinenin durum çizelgesini bulunuz.

S.2.3. $L_{S.2.3}$ dili aşağıdaki düzgün deyimle tanımlanıyor.

$$L_{S.2.3} = (a+bb^*c^*)^*de^*$$

- $L_{S.2.3}$ 'ü tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği (G_0 olarak adlandırılalım) λ -geçişleri içerebilir.
- Eğer G_0 λ -geçişleri içeriyorsa, λ -geçiş içermeyen eşdeğer geçiş çizeneğini bulmanız isteniyor. Bunun için G_0 'da, eğer birden çok λ -geçiş varsa, bu geçişlerin hangi sırada yok edilmesi gerektiğini belirtiniz ve yok edilme sıralarına göre bu geçişleri $\lambda_1, \lambda_2, \lambda_3, \dots$ olarak adlandırınız. Daha sonra λ_1 'i yok ederek G_1 'i; eğer varsa λ_2 'yi yok ederek G_2 'yi, ..., bulunuz.

S.2.4. $L_{S.2.4}$ dili, $\{0, 1, 2\}$ alfabesinde, içinde **012** ya da **210** alt dizgisi (ikisinden en az biri, en az bir kez) bulunan dizgiler kümesi olarak tanımlanıyor. $L_{S.2.4}$ 'ü tanıyan, en az durumlu DFA'yı bulmanız isteniyor. Bunun için:

- $L_{S.2.4}$ 'ü bir düzgün deyimle gösteriniz ve $L_{S.2.4}$ 'ü tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturacağımız geçiş çizeneği λ -geçişleri içermesin.
- Durumlarını $S_0, S_1, \dots$ diye adlandırarak $L_{S.2.4}$ 'ü tanıyan DFA'nın geçiş çizelgesini bulunuz.
- Bulduğunuz DFA'yı indirgeyerek, $L_{S.2.4}$ 'ü tanıyan en az durumlu DFA'nın geçiş çizelge ve çizeneğini (durumları $S, A, B, C, \dots$ diye adlandırarak) bulunuz.

S.2.5. Aşağıda sözlü olarak tanımlanan kümelerden her birinin biçimsel tanımını bir düzgün deyimle veriniz.

- $\{a, b, c\}$ alfabesinde, içindeki her a 'dan önce ve her b 'den sonra en az bir c bulunan dizgiler kümesi.
- $\{a, b, c\}$ alfabesinde, içindeki b 'lerin sayısı ile c 'lerin sayısının toplamı 3 olan dizgiler kümesi.
- $\{a, b, c\}$ alfabesinde, içindeki a 'ların sayısı 3 olan dizgiler kümesi.
- $\{a, b, c\}$ alfabesinde, içinde aa alt dizgisi bulunmayan dizgiler kümesi.
- $\{a, b\}$ alfabesinde, içinde aaa alt dizgisi bulunmayan dizgiler kümesi.
- $\{a, b, c\}$ alfabesinde, içindeki a 'ların sayısı ikinin katı (2, 4, 6, ...) olan dizgiler kümesi.

S.2.6. Aşağıdaki düzgün deyimlerle tanımlanan her küme için, kümeyi tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Eğer oluşturduğunuz çizeneğin λ -geçişleri içeriyorsa, λ -geçişsiz denk geçiş çizeneğini bulunuz.

- $P_1 = (a^*ba^*+cd^*c)^*$
- $P_2 = (010+101+(00+11)^*)^*$

S.2.7. $L_{S.2.7}$ dili, $\{0, 1\}$ alfabesinde, içinde **0100** ya da **1011** alt dizgisi (ikisinden en az biri, en az bir kez) bulunan dizgiler kümesi olarak tanımlanıyor.

- $L_{S.2.7}$ dilini bir düzgün deyimle gösteriniz.
- $L_{S.2.7}$ dilini tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Geçiş çizeneğinde λ -geçişleri yer almasın. Deterministik olmayan bu çizeneğin durumları $A, B, C, \dots$ diye adlandırınız.
- Bulduğunuz NFA'ya denk DFA'nın geçiş çizeneğini oluşturunuz. Deterministik çizeneğin durumları $S_0, S_1, S_2, \dots$ diye adlandırınız. Deterministik çizeneği indirgeyin ve en az durumlu deterministik çizeneği bulunuz.

S.2.8. Aşağıdaki kümelerden her birini tanımlayan bir düzgün deyim yazınız.

- $\{a, b, c\}$ alfabesinde, içinde iki (ve yalnız iki) tane a bulunan dizgiler kümesi.
- $\{a, b\}$ alfabesinde, içindeki a 'ların sayısı üçün katı (3, 6, 9, ...) olan dizgiler kümesi.
- $\{a, b\}$ alfabesinde, içinde aa alt dizgisi bulunmayan dizgiler kümesi.

S.2.9. $L_{S.2.9} = (a^*bc + ab^*c + abc)^*$

Yukarıdaki düzgün deyimle tanımlanan $L_{S.2.9}$ dilini tanıyan bir DFA bulmanız isteniyor. Bunun için:

- Durumları $A, B, C, \dots$ diye adlandırarak $L_{S.2.9}$ 'u tanıyan NFA'nın geçiş çizeneğini oluşturunuz.
- Oluşturduğunuz çizenekte λ -geçişleri varsa, bu geçişleri yok ederek λ -geçişsiz denk çizeneği bulunuz.
- Durumları $q_0, q_1, q_2, \dots$ diye adlandırarak denk DFA'nın geçiş çizelgesini oluşturunuz.

S.2.10. Aşağıdaki düzgün deyimlerle tanımlanan her küme için, kümeyi tanıyan sonlu özdevinirin (NFA) geçiş çizeneğini oluşturunuz. Oluşturduğunuz çizenekte λ -geçişleri varsa, bu geçişleri yok ederek λ -geçişsiz denk çizeneği bulunuz.

- $L_{S.2.10.1} = (ab(c^*d)^*a)^*$
- $L_{S.2.10.2} = (ab)^* + (ab)^* + (a^*b)^*$

S.2.11. $L_{S.2.11} = a(ac^* + b^*a)^*a$

Yukarıdaki düzgün deyimle tanımlanan $L_{S.2.11}$ dilini tanıyan bir DFA bulmanız isteniyor. Bunun için:

- Durumları $A, B, C, \dots$ diye adlandırarak $L_{S.2.11}$ 'i tanıyan NFA'nın geçiş çizeneğini oluşturunuz.
- Oluşturduğunuz çizenekte λ -geçişleri varsa, bu geçişleri yok ederek λ -geçişsiz denk çizeneği bulunuz.
- Durumları $q_0, q_1, q_2, \dots$ diye adlandırarak denk DFA'nın geçiş çizelgesini oluşturunuz.

S.2.12. $L_{S.2.12} = \{(a+bb)^k(c+dd)^m(e+ff)^n \mid k, n \geq 0, m \geq 1\}$

- Yukarıda küme tanımı verilen düzgün dili bir düzgün deyimle gösteriniz.
- Dili tanıyan NFA'nın geçiş çizeneğini oluşturunuz.
- Oluşturduğunuz NFA'ya denk DFA'nın geçiş çizelgesini oluşturunuz.

S.2.13. $L_{S.2.13}$ dili aşağıdaki düzgün deyimle tanımlanıyor.

$$L_{S.2.13} = (ab(c^*d)^*a)^*$$

- $L_{S.2.13}$ 'ü tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz çizeneğe λ -geçiş/geçişleri içeriyorsa, bu geçişleri yok edip λ -geçişsiz denk çizeneği bulunuz (bu çizenekte durumları $A, B, C, \dots$ diye adlandırınız).
- Yukarıda bulduğunuz NFA'ya denk DFA'nın geçiş çizeneğini bulunuz. Deterministik çizenekte durumları $S_0, S_1, S_2, \dots$ diye adlandırınız. Bu son çizeneğe indirgenebilir mi? Evet ise çizeneği indirgeyerek en az durumlu denk çizeneği bulunuz.

S.2.14. $L_{S.2.14}$ dili $\{a, b\}$ alfabesinde a ile başlayıp a ile biten, ilk ve son simgeler arasında da aa alt dizgisini içeren (dolayısıyla uzunluğu en az 4 olan) dizgiler kümesi olarak tanımlanıyor. $L_{S.2.14}$ 'de yer alan dizgilerden kimi örnekler aşağıda görülmektedir.

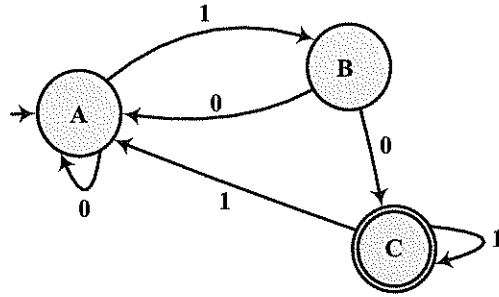
$$L_{S.2.14} = \{aaaa, abaaa, aaaaa, ababbaaba, \dots\}$$

- $L_{S.2.14}$ dilini tanımlayan bir düzgün deyim yazınız.
- $L_{S.2.14}$ dilini tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Eğer geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk çizeneği bulunuz.
- Bulduğunuz NFA'ya denk DFA'nın geçiş çizeneğini oluşturunuz.

S.2.15. Aşağıdaki geçiş çizelgesi ile tanımlanan sonlu özdevinirin tanıdığı dili bir düzgün deyimle gösteriniz.

	0	1
→S	S	A
A	B	A
B	S	C
Ⓢ	S	A

- S.2.16. Aşağıda geçiş çizeneği verilen NFA'nın tanıdığı dili bir düzgün deyim olarak yazınız. NFA'ya denk bir DFA bulunuz. DFA'nın durumlarını $S_0, S_1, S_2, \dots$ diye adlandırarak geçiş çizelgesi ve geçiş çizeneğini oluşturunuz.



S.2.17. $L_{S.2.17} = ab^*cd^*$

- $L_{S.2.17}$ 'yi tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişleri içerebilir.
- Eğer geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçiş içermeyen eşdeğer geçiş çizeneğini bulunuz.

S.2.18. $L_{S.2.18} = a^*bb + c^*dde^*$

- Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu bulunsun. Uç durumların sayısı ise birden çok olabilir.
- Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz.

S.2.19. $L_{S.2.19} = (a + b^*c(a+d)^*)^*bd$

- Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini durumları A, B, C, ... diye adlandırarak ve olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu bulunsun. Uç durumların sayısı ise birden çok olabilir.
- Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz. λ -geçişsiz geçiş çizeneği için geçiş çizelgesini oluşturunuz.

S.2.20. $L_{S.2.20} = a((bc^* + de^*)^* + g(h+k)^*)^*m$

- Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini durumları A, B, C, ... diye adlandırarak ve olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu ve tek uç durum bulunsun.
- Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz. λ -geçişsiz geçiş çizeneği için geçiş çizelgesini oluşturunuz.

S.2.21. $L_{S.2.21} = ab^* + b^*c$

- Yukarıdaki düzgün deyimle tanımlanan dili tanıyan sonlu durumlu özdevinirin ($M_{S.2.21}$) geçiş çizeneğini olabildiğince az durum kullanarak oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz.
- $M_{S.2.21}$ 'nin deterministik (DFA) geçiş çizeneğini bulunuz. (DFA'nın tek başlangıç durumu bulunacak. Uç durum sayısı gerektiği kadar olabilir). DFA'nın durumlarını $q_0, q_1, q_2, \dots$ diye adlandırarak önce standart biçimdeki geçiş çizelgesini, sonra da geçiş çizeneğini oluşturunuz.

Not: Bir geçiş çizelgesinin standart biçimde olabilmesi için, çizelgenin satırları sırayla soldan sağa tarandığında, durum adlarının ilk geçişlerinin büyüyen sırada olması gerekir.

S.2.22. $L_{S.2.22} = a^*(b+c)^* + dde^* + g^*hh$

- Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini durumları A, B, C, ... diye adlandırarak ve olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu ve tek uç durum bulunsun.
- Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz. λ -geçişsiz geçiş çizeneği için geçiş çizelgesini oluşturunuz.

S.2.23. $L_{S.2.23} = (a^*ba^* + c^*d)^*$

- Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini durumları S, A, B, C, ... diye adlandırarak ve olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu (S) ve tek uç durum bulunsun.

- b) Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri uygun sırada yok ederek λ -geçişsiz eşdeğer NFA'nın önce geçiş çizeneğini sonra da geçiş çizelgesini oluşturunuz.

S.2.24. $L_{S.2.24} = a(a^*b + ab^* + (ab)^*)b$

Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini durumları A, B, C, ... diye adlandırarak ve olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu ve tek uç durum bulunsun.

S.2.25. $L_{S.2.25} = ab^* + c^*(d+ab)$

- a) Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini durumları S, A, B, C, ... diye adlandırarak ve olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu (S) ve tek uç durum bulunsun.
- b) Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz eşdeğer NFA'nın önce geçiş çizeneğini sonra da geçiş çizelgesini oluşturunuz.

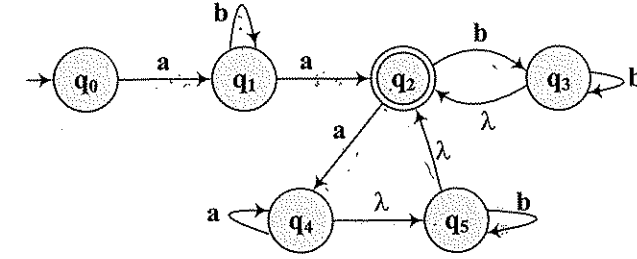
S.2.26. $L_{S.2.26} = 10^*(11)^*0^*1$

- a) Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini durumları S, A, B, C, ... diye adlandırarak ve olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu (S) ve tek uç durum bulunsun.
- b) Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz eşdeğer NFA'nın önce geçiş çizeneğini sonra da geçiş çizelgesini oluşturunuz.

S.2.27. $L_{S.2.27} = ab^*(cd^* + c^*d)bd$

- a) Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu ile tek uç durum bulunsun.
- b) Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk geçiş çizeneğini bulunuz ve geçiş çizelgesini oluşturunuz.

S.2.28.



- a) Yukarıdaki geçiş çizeneği ile tanımlanan NFA'nın ($M_{S.2.28}$) tanıdığı dili tanımlayan düzgün deyim, denklem sistemi yazıp çözmeden, sezgisel olarak yazınız.
- b) λ -geçişlerini yok ederek λ -geçişsiz eşdeğer NFA'nın önce geçiş çizeneğini sonra da geçiş çizelgesini oluşturunuz.
- c) $M_{S.2.28}$ 'e denk deterministik (DFA) geçiş çizeneğini bulunuz. DFA'nın durumlarını A, B, C, ... diye adlandırarak önce standart biçimdeki geçiş çizelgesini, sonra da geçiş çizeneğini oluşturunuz.
- d) Elde ettiğiniz DFA'nın hangi durumlarının denk olduğunu, herhangi bir işlem yapmadan sezgisel olarak söyleyebilir misiniz? (İndirgeme yapılması istenmiyor, yalnız varsa hangi durumların denk olduğu soruluyor.)

Not: Bir geçiş çizelgesinin standart biçimde olabilmesi için, çizelgenin satırları sırayla soldan sağa tarandığında, durum adlarının ilk geçişlerinin büyüyen sırada olması gerekir.

Bölüm

Dilbilgisi ve Diller

Her biçimsel dil belirli bir alfabe üzerinde tanımlanır. Alfabe ise sonlu sayıda simgeden oluşan bir kümedir. Alfabedeki simgelerin ardarda getirilmesi ile dizgiler (*strings*) oluşturulur. Biçimsel dil, bir alfabedeki simgelerden oluşturulan dizgilerin bir kümesidir. Buna göre, bir alfabedeki simgelerden oluşturulabilecek tüm dizgileri içeren kümeyi E ile gösterirsek, bu alfabe üzerinde tanımlanan her dil E'nin bir altkümesidir. E'deki dizgilerden, dilde yer alanlar dilin tümceleridir (*sentences*). Bir alfabe üzerinde tanımlanan biçimsel bir dil, bu alfabedeki simgelerden oluşan dizgileri "geçerli" ve "geçersiz" olmak üzere ikiye ayırır. Dilde yer alan ve dilin tümcelerini oluşturan dizgiler "geçerli", dilde yer almayan dizgiler ise "geçersiz" dizgilerdir. Biçimsel dil açısından dizgi (*string*), tümce (*sentence*) ve sözcük (*word*) terimleri birbirinin yerine kullanılabilir. Bu kitapta, dizgi ve sözcük terimleri eşanlamlı kullanılacaktır. Tümce ise dilde yer alan dizgi ya da sözcükleri anlatmak için kullanılacaktır. Buna göre bir alfabe ve bu alfabe üzerinde tanımlı bir dil düşünüldüğünde, alfabedeki simgelerden oluşturulan ve dilde yer alan (geçerli) dizgiler dilin tümcelerini oluşturacaktır. Dilin hangi tümcelerden oluştuğunu gösteren kurallar bütünü ise dilbilgisi (*grammar*) olarak adlandırılacaktır.

Biçimsel dilbilgisi ve dillerin incelenmesinde, değişik harf grupları değişik alanlarda kullanılır. Harf grupları ile kullanım alanları arasındaki eşleme Çizelge 3.1'de görüldüğü gibidir.

Çizelge 3.1. Harf Grupları ve Kullanım Alanları

Harf Grubu	Örnekler	Kullanım alanı
Latin alfabesinin başındaki büyük harfler	A, B, C, ...	Sözdizim değişkenleri
Latin alfabesinin başındaki küçük harfler ve rakamlar	a, b, c, ..., 0, 1, 2, ..	Uç simgeler
Latin alfabesinin sonundaki büyük harfler	U, V, W, X, Y, Z, ..	Sözdizim değişkeni ya da uç simgeler
Latin alfabesinin sonundaki küçük harfler	u, v, w, x, y, z, ...	Uç simge dizgileri (sözcükler)
Yunan alfabesinin başındaki küçük harfler	$\alpha, \beta, \gamma, \dots$	Tümcesel yapılar

3.1. Dilbilgisi ve Dilin Biçimsel Tanımı

Biçimsel olarak dilbilgisi bir dördütlü olarak tanımlanır:

$$G = \langle V_N, V_T, P, S \rangle$$

V_N : Sözdizim değişkenleri kümesi (sonlu bir küme)

V_T = Uç simgeler kümesi (sonlu bir küme).

V_N ve V_T ayrık kümelerdir: $V_N \cap V_T = \emptyset$

S : Başlangıç değişkeni : $S \in V_N$

P : Yeniden yazma ya da türetme kuralları

$$\alpha \Rightarrow \beta$$

biçimindedir ve “ α ’nın yerine β konulabilir” diye okunur.

En genel (kısıtlamasız) biçimiyle α ve β aşağıdaki gibi tanımlanır:

$$\alpha \in V^+$$

$$\beta \in V^*$$

$$V = V_N \cup V_T$$

$$V^+ = V^* - \{\lambda\}$$

Bir dilbilgisi tarafından tanımlanan dil biçimsel olarak aşağıdaki gibi tanımlanır:

$$L(G) = \{ w \mid w \in V_T^*, S \Rightarrow w \}$$

Yukarıdaki tanıma göre, bir dilin tümceleri, başlangıç simgesinden (S ’den) başlanarak ve yeniden yazma kuralları yeterli sayıda kullanılarak elde edilen uç simge dizgileridir.

$$S \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n \Rightarrow w$$

$\alpha_1, \alpha_2, \dots, \alpha_n$: tümcesel yapılar

w : tümce

Örnek 3.1.

$$G_{3.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{ S \}$$

$$V_T = \{ 0, 1 \}$$

$$P : S \Rightarrow 0S1$$

$$S \Rightarrow 01$$

$G_{3.1}$ tarafından türetilen tümcelerden birkaçını bulalım:

$$S \Rightarrow 01$$

$$S \Rightarrow 0S1 \Rightarrow 0011$$

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$$

Yukarıdaki tümce örneklerinden, $L(G_{3.1})$ dilinin aşağıdaki gibi tanımlanabileceği görülmektedir:

$$L(G_{3.1}) = \{ 0^n 1^n \mid n \geq 1 \}$$

3.2. Dilbilgisi ve Dillerin Sınıflandırılması

Dilbilgisi ve türettikleri diller, yeniden yazma kurallarının özelliklerine göre:

tür-0 ya da kısıtlamasız dilbilgisi ve diller,

tür-1 ya da bağlama-bağımlı dilbilgisi ve diller,

tür-2 ya da bağlamdan-bağımsız dilbilgisi ve diller,

tür-3 ya da düzgün dilbilgisi ve diller

olmak üzere 4 sınıfa ayrılır.

3.2.1. Tür-0 Dilbilgisi ve Dil

Tür-0 ya da kısıtlamasız dilbilgisinin yeniden yazma kuralları

$$\alpha \Rightarrow \beta : \alpha \in V^+ \quad \beta \in V^*$$

biçimindedir. Tür-0 dilbilgisi tarafından türetilen dile tür-0 dil denir. Tür-0 dillere özyineli sayılabilir (*recursively enumerable*) diller de denir. Kısaca "r.e." diye nitelenen özyineli sayılabilir diller için, dilin tümcelerini ardarda türeten bir yordam (*procedure*) bulunabilir. r.e. diller için, bir tümce verildiğinde, bu tümcenin dilde yer alıp almadığını (dolayısıyla geçerli bir tümce olup olmadığını) belirleyen bir yordam da bulunabilir.

Örnek 3.2.

$$G_{3.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, L, R, A, B, C\}$$

$$V_T = \{a\}$$

$$P: S \Rightarrow LAaR$$

$$Aa \Rightarrow aaA$$

$$AR \Rightarrow BR \mid C$$

$$aB \Rightarrow Ba$$

$$LB \Rightarrow LA$$

$$aC \Rightarrow Ca$$

$$LC \Rightarrow \lambda$$

Tür-0 bir dilbilgisi olan $G_{3.2}$ tarafından türetilen tümcelerden birkaçını bulalım:

$$S \Rightarrow LAaR \Rightarrow LaaAR \Rightarrow LaaC \Rightarrow LaCa \Rightarrow LCaa \Rightarrow aa$$

$$S \Rightarrow LAaR \Rightarrow LaaAR \Rightarrow LaaBR \Rightarrow LaBaR \Rightarrow LBaaR$$

$$\Rightarrow LAaaR \Rightarrow LaaAaR \Rightarrow LaaaaAR \Rightarrow LaaaaC$$

$$\Rightarrow LaaaCa \Rightarrow LaaCaa \Rightarrow LaCaaa \Rightarrow LCaaaa \Rightarrow aaaa$$

Yukarıda türetilen örnek tümceler ve dilbilgisinin kuralları dikkatle incelendiğinde $L(G_{3.2})$ dilinin tanımının aşağıdaki gibi olduğu bulunabilir.

$$L(G_{3.2}) = \{a^i \mid i = 2^n, n \geq 1\}$$

3.2.2. Tür-1 Dilbilgisi ve Dil

Tür-1 dilbilgisinin yeniden yazma kuralları

$$\alpha \Rightarrow \beta : \alpha \in V^+ \quad \beta \in V^* \quad |\alpha| \leq |\beta|$$

biçimindedir. Görüldüğü gibi, tür-1 dilbilgisinde, tür-0 dilbilgisine göre $|\alpha| \leq |\beta|$ kısıtlaması getirilmektedir.

Tür-1 dilbilgisine bağlama-bağımlı (*context sensitive*) dilbilgisi de denir. Çünkü tür-1 dilbilgisi, yeniden yazma kurallarının tümü

$$\alpha_1 A \alpha_2 \Rightarrow \alpha_1 \beta \alpha_2 \quad A \in V_N, \quad \alpha_1, \alpha_2, \beta \in V^*$$

biçiminde olan bir normal biçime (*normal form*) dönüştürülebilir. Bu normal biçimde, $\alpha_1 \dots \alpha_2$ bağlamında A 'nın yerine β konulabildiği için, dilbilgisi bağlama bağımlı dilbilgisi olarak adlandırılmaktadır.

Tür-1 dilbilgisi tarafından türetilen dile tür-1 ya da bağlama-bağımlı dil denir. Tür-1 dilin diğer bir adı özyineli (*recursive*) dildir. Kısaca "r." olarak nitelenen özyineli diller için, bir tümce verildiğinde, bu tümcenin dilde yer alıp almadığını (dolayısıyla geçerli bir tümce olup olmadığını) belirleyen bir algoritma bulunabilmektedir. Oysa tür-0 diller için tümcenin dilde yer alıp almadığını belirleyen bir yordam bulunabilmektedir. Algoritmanın her zaman sonlanan bir yordam olduğu düşünüldüğünde, özyineli (r.) dillerin, özyineli sayılabilir (r.e.) dillerin bir alt sınıfı olduğu görülmektedir.

Örnek 3.3.

$$G_{3.3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow aSAB$$

$$S \Rightarrow aAB$$

$$BA \Rightarrow AB$$

$$aA \Rightarrow ab$$

$$bA \Rightarrow bb$$

$$bB \Rightarrow bc$$

$$cB \Rightarrow cc$$

$G_{3.3}$ tarafından türetilen tümcelerden birkaçını bulalım:

$$S \Rightarrow aAB \Rightarrow abB \Rightarrow abc$$

$$S \Rightarrow aSAB \Rightarrow aaABAB \Rightarrow aabBAB \Rightarrow aabABB \Rightarrow aabbbb$$

$$\Rightarrow aabbcB \Rightarrow aabbcc$$

$G_{3.3}$ tür-1 bir dilbilgisidir. Yukarıdaki örneklerden, $L(G_{3.3})$ dilinin aşağıdaki gibi tanımlanabileceği görülmektedir:

$$L(G_{3.3}) = \{a^n b^n c^n \mid n \geq 1\}$$

3.2.3. Tür-2 Dilbilgisi ve Dil

Tür-2 dilbilgisinin yeniden yazma kuralları

$$A \Rightarrow \beta : A \in V_N \quad \beta \in V^*$$

biçimindedir. Yeniden yazma kurallarının sol tarafında tek bir değişken (A) yer almaktadır. Yeniden yazma kuralı, hangi bağlamda olursa olsun, A'nın yerine β konulabileceğini tanımlamaktadır. Bu özelliği nedeniyle tür-2 dilbilgisine bağlamdan-bağımsız (*context free*) dilbilgisi denir.

Tür-2 dilbilgisi tarafından türetilen dillere tür-2 ya da bağlamdan-bağımsız diller denir. Tür-2 dilbilgisi ve diller bilgisayar mühendisliği açısından büyük önem taşır. Çünkü programlama dilleri ve yazılım ürünlerinin birçok kesiminde bu model kullanılır.

Yukarıda verilen dilbilgisi örneklerinden $G_{3.1}$ tür-2 bir dilbilgisidir. Buna karşılık $G_{3.2}$ ve $G_{3.3}$ tür-2 dilbilgileri değildir.

Örnek 3.4.

$$G_{3.4} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aB \mid bA$$

$$A \Rightarrow a \mid aS \mid bAA$$

$$B \Rightarrow b \mid bS \mid aBB$$

$G_{3.4}$ tarafından türetilen tümcelerden birkaçını bulalım:

$$S \Rightarrow bA \Rightarrow baS \Rightarrow baba \Rightarrow baba$$

$$S \Rightarrow aB \Rightarrow abS \Rightarrow abba \Rightarrow abbaS \Rightarrow abbaaB \Rightarrow abbaab$$

$$S \Rightarrow bA \Rightarrow bbaA \Rightarrow bbaSA \Rightarrow bbaaBA \Rightarrow bbaabSA$$

$$\Rightarrow bbaabaBA \Rightarrow bbaababa \Rightarrow bbaababa$$

$G_{3.4}$ tür-2 bir dilbilgisidir. Dilbilgisinin kuralları ve yukarıdaki örnekler dikkatle incelendiğinde, $L(G_{3.4})$ dilinin, $\{a, b\}$ alfabesinde eşit sayıda a ve b içeren dizgiler kümesi olduğu görülür.

3.2.4. Tür-3 Dilbilgisi ve Dil

Tür-3 dilbilgisinin yeniden yazma kuralları

$$A \Rightarrow aB$$

$$A \Rightarrow a$$

$$A \Rightarrow \lambda : A, B \in V_N, a \in V_T$$

biçimindedir. Yeniden yazma kurallarının sol tarafında tek bir değişken (A); sağ tarafında ise ya tek bir uç simge ya da bir uç simge ile bir değişken yer almaktadır. Tür-3 dilbilgisi tarafından türetilen dile tür-3 ya da düzgün (*regular*) dil denir. Daha önce örnek olarak verilen dilbilgilerinden hiçbirisi tür-3 bir dilbilgisi değildir.

Örnek 3.5.

$$G_{3.5} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S \mid 0A \mid 0 \mid \lambda$$

$$A \Rightarrow 0B$$

$$B \Rightarrow 1S$$

$G_{3.5}$ tarafından türetilen tümcelerden birkaçını bulalım:

$$S \Rightarrow 0S \Rightarrow 00$$

$$S \Rightarrow 0A \Rightarrow 00B \Rightarrow 001S \Rightarrow 001$$

$$S \Rightarrow 0S \Rightarrow 00A \Rightarrow 000B \Rightarrow 0001S \Rightarrow 00010A$$

$$\Rightarrow 000100B \Rightarrow 0001001S \Rightarrow 00010010$$

$G_{3.5}$ tür-3 bir dilbilgisidir. Dilbilgisinin kuralları ve yukarıdaki örnekler dikkatle incelendiğinde, $L(G_{3.5})$ dilinin " $\{0, 1\}$ alfabesinde, içindeki her 1'den önce en az iki tane 0 bulunan dizgiler kümesi" olduğu görülür.

3.2.5. Sağ-Doğrusal ve Sol-Doğrusal Dilbilgisi

Yeniden yazma kuralları

$$A \Rightarrow wB$$

$$A \Rightarrow w : A, B \in V_N, w \in V_T^*$$

biçiminde olan dilbilgisine sağ-doğrusal (*right-linear*) dilbilgisi denir. Sağ-doğrusal dilbilgisinin yeniden yazma kurallarının sol tarafında bir değişken, sağ tarafında ise

bir uç simgeler dizgisi ya da bir uç simgeler dizgisi ile bir değişken yer alır. Uç simgeler dizgisi sıfır uzunluğunda bir dizgi de olabilir.

Yeniden yazma kuralları

$$A \Rightarrow Bw$$

$$A \Rightarrow w \quad : \quad A, B \in V_N, \quad w \in V_T^*$$

biçiminde olan dilbilgisine **sol-doğrusal** (*left-linear*) dilbilgisi denir. Sol-doğrusal dilbilgisinin yeniden yazma kurallarının sol tarafında bir değişken, sağ tarafında ise bir uç simgeler dizgisi ya da bir değişken ile bir uç simgeler dizgisi yer alır. Uç simgeler dizgisi sıfır uzunluğunda bir dizgi de olabilir.

Sağ-doğrusal ve sol-doğrusal dilbilgileri tarafından türetilen diller düzgün dillerdir. Tür-3 dilbilgileri tarafından türetilen diller de düzgün diller olduğuna göre tür-3, sağ-doğrusal ve sol-doğrusal dilbilgileri denk dilbilgileridir. Tür-3, sağ-doğrusal ve sol-doğrusal dilbilgilerinin tümü düzgün dilbilgileri; türettikleri diller ise düzgün diller olarak adlandırılır.

Düzgün dilbilgileri için verilen ve birbirine denk olduğu belirtilen üç biçimden, tür-3 sağ-doğrusalın özel bir biçimidir. Başka bir deyişle her tür-3 dilbilgisi aynı zamanda sağ-doğrusaldır. Sağ-doğrusal her dilbilgisi de, ek değişkenler kullanılarak, kolaylıkla tür-3 dilbilgisine dönüştürülebilir. Sağ-doğrusal dilbilgisini tür-3 dilbilgisine dönüştürmek için,

$$A \Rightarrow a_1 a_2 \dots a_n \quad \text{yerine} \quad \begin{cases} A \Rightarrow a_1 A_1 \\ A_1 \Rightarrow a_2 A_2 \\ \dots\dots\dots \\ A_{n-1} \Rightarrow a_n \end{cases}$$

$$A \Rightarrow a_1 a_2 \dots a_n B \quad \text{yerine de} \quad \begin{cases} A \Rightarrow a_1 A_1 \\ A_1 \Rightarrow a_2 A_2 \\ \dots\dots\dots \\ A_{n-1} \Rightarrow a_n B \end{cases}$$

kuralları yazılır. Bu nedenle tür-3 ve sağ-doğrusal dilbilgilerinin denk olduğu açıktır. Sol-doğrusal ve sağ-doğrusal dilbilgilerinin denkliği ise bu kadar açık değildir. Ancak düzgün dilbilgisinin üç biçimi (tür-3, sağ-doğrusal ve sol-doğrusal) tarafından türetilen dillerin sonlu özdevinirler (FA) tarafından tanınan düzgün diller olduğu gösterildiğinde (Bkz. 3.3.1) dolaylı olarak bu üç biçimin eşdeğerliği de ispatlanmış olmaktadır.

Örnek 3.6.

$$G_{3,6} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0A$$

$$A \Rightarrow 10A \mid \lambda$$

$G_{3,6}$ sağ-doğrusal bir dilbilgisidir. Bu dilbilgisi tarafından türetilen tümcelerden birkaçını bulalım:

$$S \Rightarrow 0A \Rightarrow 0$$

$$S \Rightarrow 0A \Rightarrow 010A \Rightarrow 010$$

$$S \Rightarrow 0A \Rightarrow 010A \Rightarrow 01010A \Rightarrow 01010$$

Yukarıdaki örneklerin incelenmesinden, $L(G_{3,6})$ dilinin aşağıdaki gibi tanımlanabileceği görülür:

$$L(G_{3,6}) = 0(10)^* = (01)^*0$$

$G_{3,6}$ tarafından türetilen ve yukardaki düzgün deyimlerle gösterilen dil, aşağıdaki sol-doğrusal ($G'_{3,6}$) ve tür-3 ($G''_{3,6}$) dilbilgileri tarafından da türetilir. Dolayısıyla $G_{3,6}$, $G'_{3,6}$ ve $G''_{3,6}$ aynı dili türeten, denk dilbilgileridir.

$$G'_{3,6} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow S10 \mid 0$$

$$G''_{3,6} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0A$$

$$A \Rightarrow 1B \mid \lambda$$

$$B \Rightarrow 0A$$

3.3. Düzgün Dil - Sonlu Özdevinir İlişkisi

3.3.1. Düzgün Dilbilgisinin Türettiği Dili Tanıyan Sonlu Özdevinirin Bulunması

Önerme 3.1. Bir düzgün dilbilgisi verildiğinde, bu dilbilgisinin türettiği dili tanıyan bir sonlu özdevinir bulunabilir.

Sonlu özdevinirler tarafından tanınan kümelerin düzgün kümeler olduğu tanımlanmıştır (Bkz 2.1). Önerme 3.1'de ise düzgün dilbilgileri tarafından türetilen dillerin sonlu özdevinirler tarafından tanındığı belirtilmektedir. Burdan, düzgün dilbilgileri tarafından türetilen dillerin (düzgün dillerin) düzgün kümeler olduğu sonucu çıkmaktadır.

Önerme 3.1, bir düzgün dilbilgisi verildiğinde, bu dilbilgisinin türettiği dili tanıyan sonlu özdevinirin nasıl bulunacağı gösterilerek ispat edilecektir. Düzgün dilbilgileri için üç farklı biçim olduğu için de, dilbilgisine eşdeğer sonlu özdevinirin nasıl bulunacağı tür-3, sağ-doğrusal ve sol-doğrusal dilbilgileri için ayrı ayrı gösterilecektir.

Tür-3 Dilbilgisinin Türettiği Dili Tanıyan Sonlu Özdevinirin Bulunması

Bir tür-3 dilbilgisi (G) verildiğinde, $T(M) = L(G)$ eşitliğini sağlayan sonlu özdevinir (M) aşağıdaki gibi bulunur.

$$G = \langle V_N, V_T, P, S \rangle$$

$$M = \langle Q, \Sigma, q_0, \delta, F \rangle$$

$$Q = V_N \cup \{C\} \quad C, V_N \text{ 'de bulunmayan yeni bir değişken}$$

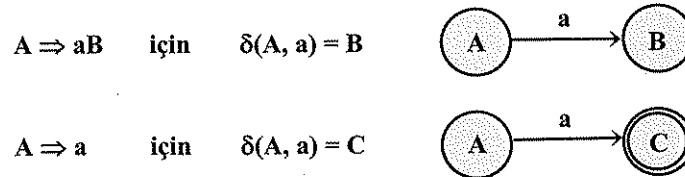
$$\Sigma = V_T$$

$$q_0 = S$$

$$F = \{C\} \quad \text{eğer } L(G) \lambda \text{ 'yı içermiyorsa}$$

$$= \{C, S\} \quad \text{eğer } L(G) \lambda \text{ 'yı içeriyorsa}$$

δ : Sonlu özdevinirin durum geçişleri, dilbilgisinin yeniden yazma kurallarından aşağıdaki gibi türetilir.



Örnek 3.7.

$$G_{3.7} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S \mid 1A$$

$$A \Rightarrow 0B$$

$$B \Rightarrow 0B \mid 1S \mid 1$$

$G_{3.7}$ tür-3 bir dilbilgisidir. Bu dilbilgisinin türettiği dili tanıyan FA aşağıdaki gibi bulunur:

$$M_{3.7} = \langle Q, \Sigma, q_0, \delta, F \rangle$$

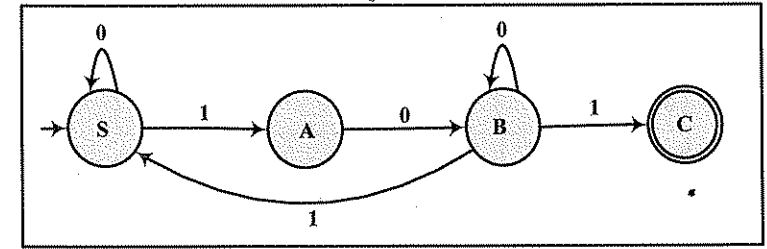
$$Q = \{S, A, B, C\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = S$$

$$F = \{C\}$$

δ : Makinenin durum geçişleri Çizim 3.1'de görülmektedir.



Çizim 3.1. $G_{3.7}$ 'nin türettiği Dili Tanıyan $M_{3.7}$ 'nin Geçiş Çizeneği

Sağ-Doğrusal Dilbilgisinin Türettiği Dili Tanıyan Sonlu Özdevinirin Bulunması

Bir sağ-doğrusal dilbilgisi (G) verildiğinde, $T(M) = L(G)$ eşitliğini sağlayan sonlu özdevinir (M) aşağıdaki gibi bulunur.

$$G = \langle V_N, V_T, P, S \rangle$$

$$M = \langle Q, \Sigma, q_0, \delta, F \rangle$$

$$Q = \{[S], [\lambda], [\alpha_1], [\alpha_2], [\alpha_3], \dots\}$$

α_i : herhangi bir yeniden yazma kuralının sağ tarafının herhangi bir soneki (suffix)

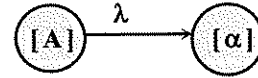
$$\Sigma = V_T$$

$$q_0 = [S]$$

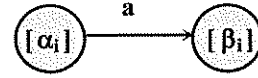
$$F = [\lambda]$$

δ : Sonlu özdevinirin durum geçişleri, dilbilgisinin yeniden yazma kurallarından ve bu kuralların sağ taraflarının soneklerinden aşağıdaki gibi türetilir.

Her $A \Rightarrow \alpha$ yeniden yazma kuralı için için bir λ -geçişi:



İlk simgesi bir uç simge olan her α_i ($\alpha_i = a\beta_i$) için:



Örnek 3.8.

$$G_{3.8} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S \mid 1S \mid 111A$$

$$A \Rightarrow 0A \mid 1A \mid \lambda$$

$G_{3.8}$ sağ-doğrusal bir dilbilgisidir. Bu dilbilgisinin türettiği dili tanıyan makine ($M_{3.8}$) aşağıdaki gibi bulunur.

$$M_{3.8} = \langle Q, \Sigma, q_0, \delta, F \rangle$$

$$Q = \{[S], [\lambda], [0S], [1S], [111A], [11A], [1A], [A], [0A]\}$$

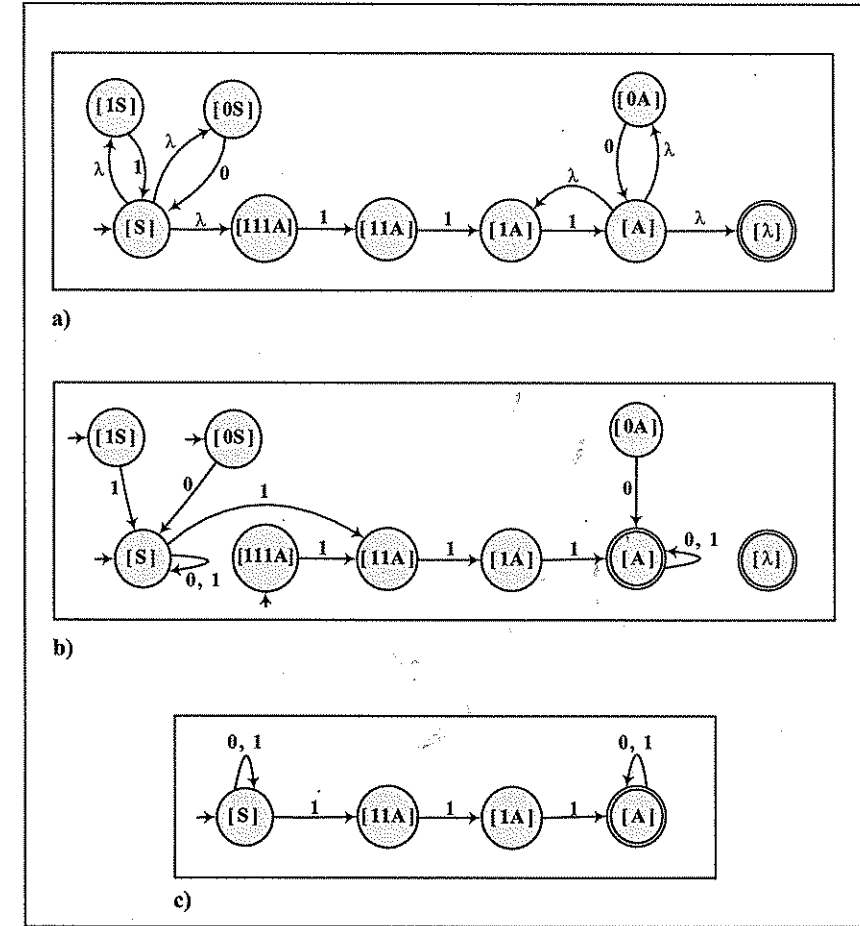
$$\Sigma = \{0, 1\}$$

$$q_0 = [S]$$

$$F = [\lambda]$$

δ : Makinenin durum geçişleri Çizim 3.2.a'da görülmektedir.

Çizim 3.2.a'daki geçiş çizeneğinde λ -geçişleri yok edildiğinde Çizim 3.2.b'deki geçiş çizeneği elde edilir. Bu geçiş çizeneğindeki gereksiz durumlar çıkarıldığında da, Çizim 3.2.c'deki geçiş çizeneği elde edilir.



Çizim 3.2. $G_{3.8}$ 'in Türettiği Dili Tanıyan $M_{3.8}$ 'in Bulunması

Sol-Doğrusal Dilbilgisinin Türettiği Dili Tanıyan Sonlu Özdevinirin Bulunması

Sol-dorusal bir dilbilgisi (G) verildiğinde, bu dilbilgisinin türettiği dili tanıyan makine (M) aşağıdaki gibi bulunur.

1. Dilbilgisindeki tüm yeniden yazma kurallarının sağ tarafları tersine çevrilerek G^R elde edilir. G^R sağ-doğrusal bir dilbilgisidir.
2. G^R 'yi tanıyan makine (M^R) bulunur.
3. M^R tersine çevrilerek M elde edilir. Bunun için :
 - Başlangıç durumu uç durum, uç durumlar ise başlangıç durumu yapılır.
 - Tüm durum geçişleri tersine çevrilir.

Örnek 3.9.

$$G_{3,9} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0 \mid S10$$

$G_{3,9}$ sol-doğrusal bir dilbirisidir. Bu dilbirisinin türettiği dili tanıyan makine ($M_{3,9}$) aşağıdaki gibi bulunur.

Önce, yeniden yazma kuralları tersine çevrilerek $G_{3,9}^R$ elde edilir.

$$G_{3,9}^R = \langle V_N, V_T, P^R, S \rangle$$

$$P^R: S \Rightarrow 0 \mid 01S$$

Sonra $G_{3,9}^R$ 'un türettiği dili tanıyan sonlu özdevinir ($M_{3,9}^R$) bulunur.

$$M_{3,9}^R = \langle Q^R, \Sigma, q_0^R, \delta^R, F^R \rangle$$

$$Q^R = \{[S], [\lambda], [0], [01S], [1S]\}$$

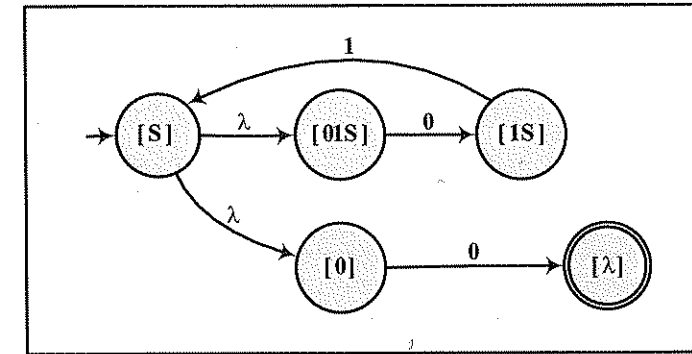
$$\Sigma = \{0, 1\}$$

$$q_0^R = [S]$$

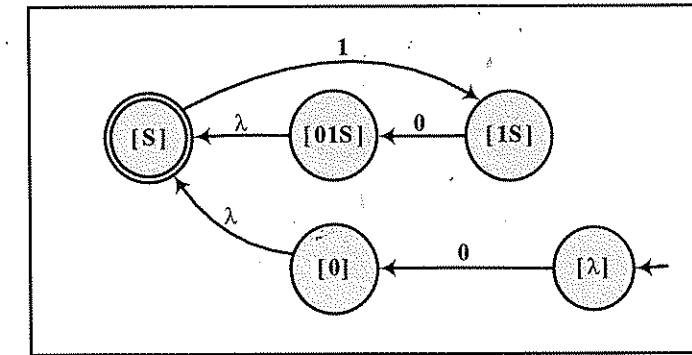
$$F^R = [\lambda]$$

δ^R : Makinenin durum geçişleri Çizim 3.3.a'da görülmektedir.

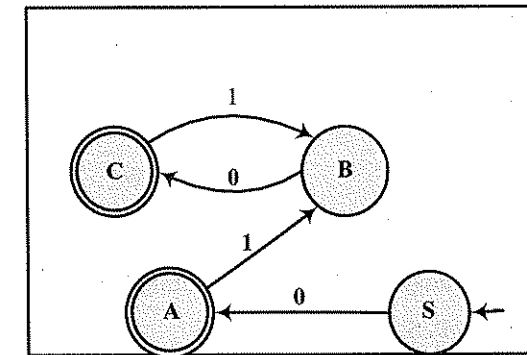
Son olarak da $M_{3,9}^R$ tersine çevrilerek $M_{3,9}$ elde edilir. $M_{3,9}$ için elde edilen geçiş çizeneği Çizim 3.3.b'de görülmektedir. Bu geçiş çizeneğindeki λ -geçişleri yok edilip, yararsız durum atılıp, durumlar yeniden adlandırıldığında ise Çizim 3.3.c'de görülen geçiş çizeneği elde edilir.



a)



b)



c)

Çizim 3.3. $G_{3,9}$ 'un türettiği Dili Tanıyan $M_{3,9}$ 'un Bulunması

3.3.2. Sonlu Özdevinirin Tanıdığı Dili Türeten Düzgün Dilbilgisinin Bulunması

Bir sonlu özdevinir (M) verildiğinde, $L(G) = T(M)$ eşitliğini sağlayan tür-3 dilbilgisi (G) aşağıdaki gibi bulunur.

$$M = \langle Q, \Sigma, q_0, \delta, F \rangle$$

$$G = \langle V_N, V_T, P, S \rangle$$

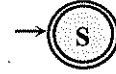
$$V_N = Q$$

$$V_T = \Sigma$$

$$S = q_0$$

δ : Dilbilgisinin yeniden yazma kuralları, sonlu özdevinirin durum geçişlerinden aşağıdaki gibi türetilir.

- her $A \xrightarrow{a} B$ için : $A \Rightarrow aB$
- her $A \xrightarrow{a} C$ için : $A \Rightarrow aC$ ve $A \Rightarrow a$
- eğer başlangıç durumu bir uç durum ise : $S \Rightarrow \lambda$



Örnek 3.10.

$$M_{3,10} = \langle Q, \Sigma, S, \delta, F \rangle$$

$$Q = \{S, A, B, C, D, E\}$$

$$V_T = \{a, b, c, d\}$$

$$F = \{E\}$$

$M_{3,10}$ 'nin geçiş çizeneği Çizim 3.4'de görülmektedir.

Bu makinenin tanıdığı dili türeten tür-3 dilbilgisi aşağıdaki gibi bulunur.

$$G_{3,10} = \langle V_N, V_T, P, S \rangle$$

$$V_N = Q = \{S, A, B, C, D, E\}$$

$$V_T = \Sigma = \{a, b, c, d\}$$

$$P: S \Rightarrow aA \mid bS \mid dS \mid dB$$

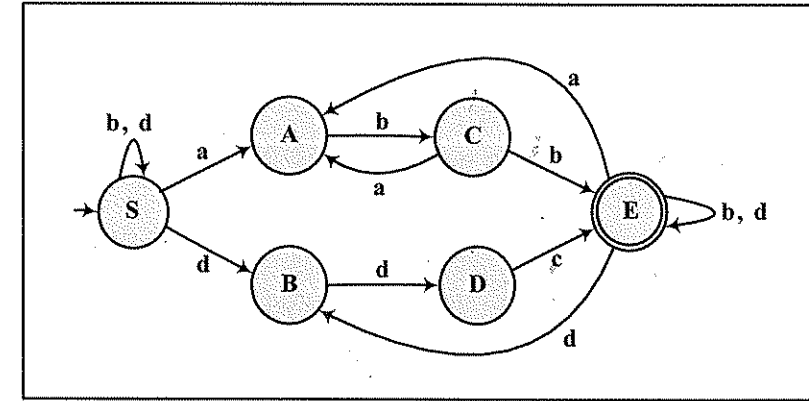
$$A \Rightarrow bC$$

$$B \Rightarrow dD$$

$$C \Rightarrow aA \mid bE \mid b$$

$$D \Rightarrow cE \mid c$$

$$E \Rightarrow aA \mid dB \mid bE \mid dE \mid b \mid d$$



Çizim 3.4. $M_{3,10}$ 'nin Geçiş Çizeneği

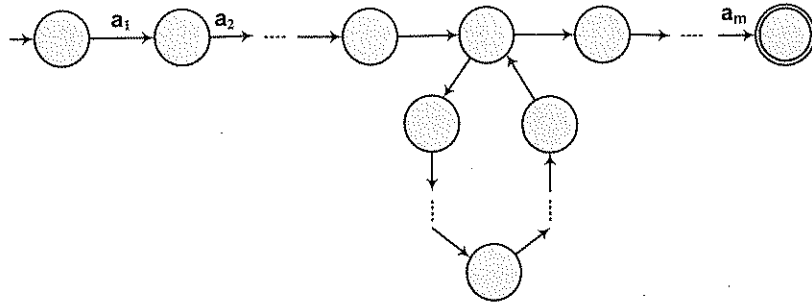
3.4. Düzgün Dillerin Kimi Özellikleri

Pumping Lemma

L düzgün bir dil olsun. L 'yi tanıyan bir ya da, kendi aralarında denk birçok sonlu özdevinir vardır. L 'yi tanıyan özdevinirlerden durum sayısı en küçük olanın (M) durum sayısının n olduğunu varsayalım. Eğer, L 'nin sözcüklerinden birinin uzunluğu n 'den büyük ise:

$$w = a_1 a_2 a_3 \dots a_m \quad |w| = m > n$$

M w 'yi tanıırken en az bir durumdan en az iki kere geçer. Başka bir deyişle, M 'de w 'ye karşı gelen yol mutlaka döngülüdür.



.Bu durumda **M**

$$w = u v^i t \quad i \geq 0$$

biçimindeki tüm tümceleri tanır ve bu tümcelerin tümü L' 'de yer alır. Dolayısıyla L sonsuz sayıda tümce içeren bir dildir.

Birçok dilin düzgün olmadığını ispatlamak için kullanılabilen ve *pumping lemma* olarak bilinen bu özellik şu şekilde özetlenebilir:

“Her düzgün dil için öyle bir tamsayı (**n**) vardır ki, eğer dilde uzunluğu bu değerden büyük bir tümce (**w**) varsa, bu tümce

$$w = u \vee t$$

biçiminde yazılabilir ve dil

$$w = uv^i t \quad i \geq 0$$

biçimindeki tüm tümceleri içerir”.

Düzgün dillerin kapahlık özellikleri

Düzgün diller küme birleşim, kesişim ve tümlleme işlemlerinde kapalıdır. Buna göre eğer L_1 ve L_2 düzgün diller ise:

$$L_1 \cup L_2,$$

$$L_1 \cap L_2,$$

L_1^2

de birer düzgün dildir.

Eğer L_1 dili, Σ alfabesi üzerinde tanımlı ise, L_1 'in tümüleri

$$L'_1 = \Sigma^* - L_1$$

olarak tanımlanır.

Sorular

S.3.1. $L_{S.3.1} = a a^* (b c^*)^*$

- $L_{S,3.1}$ ’i tanıyan NFA’nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişleri içerebilir.
- Eğer oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri tek tek yok ederek λ -geçişli içermeyen denk geçiş çizeneğini bulunuz.
- $L_{S,3.1}$ ’i türeten ve $(A \Rightarrow \lambda)$ biçiminde yeniden yazma kuralı içermeyen bir düzgün dilbilgisi tanımlayınız.

S.3.2. Aşağıda sözel olarak tanımlanan dillerden her birinin düzgün bir dil olup olmadığını belirtiniz. Eğer dil düzgün bir dil ise, dilin tanımını bir düzgün deyimle yazınız. Eğer dil düzgün değilse, dilin biçimsel tanımını matematiksel bir ifadeyle veya küme tanımıyla gösteriniz.

- a) $\{a, b, c\}$ alfabelinde, içinde en az bir tane a bulunan; ve içindeki her a 'dan hemen sonra en az 2 tane b yer alan dizgiler kümesi.
- b) $\{a, b, c\}$ alfabelinde, uzunluğu sıfırdan büyük ve 4'ün katları olan ve iki kez yansıtma ile elde edilen dizgiler kümesi. Kümedeki her dizgi, bir alt dizgiden iki kez yansıtma ile elde edilmektedir.

$$L_{s.3.2.2} = \{aaaa, abbaabba, abccbaabccba, \dots\}$$

- c) $\{a, b, c\}$ alfabesinde, içindeki tüm a 'lar tüm b 'lerden önce, tüm c 'ler ise tüm b 'lerden sonra yer alan; b 'lerin sayısı a 'ların sayısı ile c 'lerin sayısının toplamına eşit olan; en az 2 uzunluğundaki dizgiler kümesi.
- d) $\{a, b, c\}$ alfabesinde, içindeki b 'lerin sayısı üçün katı (0 dahil) olan dizgiler kümesi.

S.3.3. Aşağıda tanımı verilen dillerden her birinin türünü belirtiniz ve bu dili türeten bir dilbilgisinin biçimsel tanımını veriniz.

- a) $\{0, 1\}$ alfabesinde, içinde çift sayıda (en az 2 olmak üzere) 1 bulunan dizgiler kümesi.
- b) $L_{S.3.3.2} = \{a^n b^i c^n d^j \mid n \geq 1, i \geq 1, j \geq 1\}$

S.3.4. $L_{S.3.4}$ dili aşağıdaki düzgün deyimle tanımlanıyor:

$$L_{S.3.4} = a^* (bb^* + cc^*) d^*$$

- a) $L_{S.3.4}$ dilini sözel olarak tanımlayınız. $L_{S.3.4}$ 'ü tanıyan sonlu özdevinirin (NFA) geçiş çizeneğini oluşturunuz (çizenekte başlangıç durumunu S, diğer durumları ise A, B, C, ... diye adlandırınız).
- b) Oluşturduğunuz çizenek λ -geçiş/geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk çizeneği bulunuz.
- c) $L_{S.3.4}$ dilini türeten bir tür-3 dilbilgisi oluşturunuz (bunun için b'de bulduğunuz λ -geçişsiz çizeneği kullanınız).

S.3.5.

a) $L_{S.3.5.1} = \{a^n b^m c^k d^p \mid n > 0, m > 0, p > n, k > m\}$

Bu dilin türü nedir? Bu dili türeten bir dilbilgisi yazınız.

b) $L_{S.3.5.2} = \{(ab)^n (ba)^m \mid n > 1, m > 1\}$

Bu dilin türü nedir? Bu dili türeten bir dilbilgisi yazınız.

c) $L_{S.3.5.3} = \{\alpha \beta \beta^R \alpha^R \mid \alpha \in (0+1)^*, \beta \in (0+1)^*\}$

Bu dilin türü nedir? Bu dili türeten bir dilbilgisi yazınız.

Not: α ve β ile $\{0, 1\}$ alfabesindeki herhangi bir dizgi gösterilmektedir. α^R ve β^R ise, α ve β 'nin tersidir. Örneğin $\alpha = 01101$ ise, $\alpha^R = 10110$ 'dır.

S.3.6. $L_{S.3.6}$ dili, $\{0, 1\}$ alfabesinde, özyineli olarak aşağıdaki gibi tanımlanıyor.

- 0 ve 1 $L_{S.3.6}$ 'nın birer tümcesidir : $0 \in L_{S.3.6}$ ve $1 \in L_{S.3.6}$
- Eğer $w \in L_{S.3.6}$ 'nın bir tümcesi ise $(w \in L_{S.3.6})$, $0w1$ ve $1w0$ da $L_{S.3.6}$ 'nın birer tümcesidir ($0w1 \in L_{S.3.6}$ ve $1w0 \in L_{S.3.6}$).

- a) $L_{S.3.6}$ dilini tanımlayan matematiksel bir gösterim oluşturunuz.
- b) $L_{S.3.6}$ dilini türeten bir dilbilgisi yazınız. Bu dilbilgisinin türü nedir?

S.3.7. $L_{S.3.7}$ dili aşağıdaki düzgün deyimle tanımlanıyor:

$$L_{S.3.7} = (aa^* + c(bb^*)^*) dd$$

- a) $L_{S.3.7}$ dilini tanıyan sonlu özdevinirin (NFA) geçiş çizeneğini oluşturunuz (çizenekte başlangıç durumunu S, diğer durumları ise A, B, C, ... diye adlandırınız).

- b) Oluşturduğunuz çizenek λ -geçiş/geçişleri içeriyorsa, bu geçişleri yok edip λ -geçişsiz denk çizeneği bulunuz.
- c) $L_{S.3.7}$ dilini türeten bir tür-3 dilbilgisi oluşturunuz (bunun için b'de bulduğunuz λ -geçişsiz çizeneği kullanınız).

S.3.8. Sağ-doğrusal $G_{S.3.8}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.3.8} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, X, Y\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow abX \mid aY$$

$$X \Rightarrow aaY \mid bX$$

$$Y \Rightarrow aX \mid b$$

- a) $L(G_{S.3.8})$ 'i tanıyan sonlu özdevinirin (NFA) geçiş çizeneğini oluşturunuz.
- b) Denklem sistemi yazıp çözmeden, $L(G_{S.3.8})$ 'i bir düzgün deyim olarak yazınız.

S.3.9. Aşağıda tanımlanan her dilbilgisinin türettiği dilin doğru ve eksiksiz tanımını bir düzgün deyim, bir küme tanımı ya da bir sözel tanım olarak veriniz.

a) $S \Rightarrow XY$

$$X \Rightarrow aXb \mid ab$$

$$Y \Rightarrow cY \mid c$$

b) $S \Rightarrow KLM$

$$K \Rightarrow a$$

$$L \Rightarrow KLM \mid cN$$

$$M \Rightarrow b \mid bb$$

$$N \Rightarrow cN \mid \lambda$$

c) $S \Rightarrow aAd$

$$A \Rightarrow aAd \mid B$$

$$B \Rightarrow bBc \mid bc$$

S.3.10. Aşağıda tanımlanan dillerden her birinin türünü belirtiniz ve dili türeten bir dilbilgisi oluşturunuz.

- a) $L_{S.3.10.1} = \{a^n b^m \mid n \geq 1, m \geq 1\}$
 b) $L_{S.3.10.2} = \{a^n b^m \mid n \geq 2, n \leq m \leq 2n\}$
 c) $L_{S.3.10.3} = \{\alpha \alpha^R \beta \beta^R \mid \alpha, \beta \in (0+1)^*\}$

S.3.11. $L_{S.3.11}$ dili, $\{0, 1\}$ alfabesinde, uzunluğu ikinin katı $(0, 2, 4, \dots)$ olan dizgiler kümesi olarak tanımlanıyor.

$$L_{S.3.11} = \{w \mid w \in (0+1)^*, |w| = 2n, n \geq 0\}$$

- a) $L_{S.3.11}$ dilini tanıyan DFA'nın geçiş çizeneğini oluşturunuz (geçiş çizeneği deterministik olacak ve λ -geçiş içermeyecek).
 b) $L_{S.3.11}$ dilini türeten ve tek sözdizim değişkeni (S) içeren bir dilbilgisi tanımlayınız.

S.3.12. Aşağıda $\{a, b\}$ alfabesinde üç dil tanımı verilmektedir. Bu dillerden her birini türeten bir dilbilgisi tanımlayınız. Tanımladığınız dilbilgisinin türünü belirtiniz. Eğer tanımladığınız dilbilgisi düzgün bir dilbilgisi ise, bu dilbilgisinin türettiği dili tanıyan NFA'nın geçiş çizeneğini oluşturunuz.

- a) $L_{S.3.12.1}$: içinde en çok üç tane a bulunan dizgiler kümesi.
 b) $L_{S.3.12.2} = \{a^n b^{n-3} \mid n \geq 3\}$
 c) $L_{S.3.12.3} = \{a^n b^m \mid n \geq 1, 2n \geq m \geq 3n\}$

S.3.13. Aşağıda tanımlanan dilbilgisi ile türetilen 3 tümce bulunuz ve dilbilgisinin türettiği dilin küme tanımını oluşturunuz.

$$G_{S.3.13} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C, D, E\}$$

$$V_T = \{a\}$$

$$P: S \Rightarrow ABaC$$

$$Ba \Rightarrow aaB$$

$$BC \Rightarrow DC \mid E$$

$$aD \Rightarrow Da$$

$$AD \Rightarrow AB$$

$$aE \Rightarrow Ea$$

$$AE \Rightarrow \lambda$$

S.3.14. $G_{S.3.14}$ düzgün dilbilgisi aşağıdaki gibi tanımlanıyor.

$$G_{S.3.14} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow aA$$

$$A \Rightarrow bB$$

$$B \Rightarrow bB \mid cS \mid d$$

- a) $L(G_{S.3.14})$ 'ü tanıyan λ -geçişsiz NFA'nın geçiş çizeneğini bulunuz.
 b) Denklem sistemi yazıp çözmeden, $L(G_{S.3.14})$ 'ü tanımlayan bir düzgün deyim yazınız.

S.3.15. $L_{S.3.15} = a + ((b+c)^*)d^*$

- a) $L_{S.3.15}$ 'i tanıyan sonlu özdevinirin (NFA) geçiş çizeneğini bulunuz.
 b) Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk çizeneği bulunuz.
 c) $L_{S.3.15}$ 'i türeten bir düzgün dilbilgisi tanımlayınız.

S.3.16. Aşağıdaki düzgün dilbilgisi veriliyor.

$$G_{S.3.16} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, B\}$$

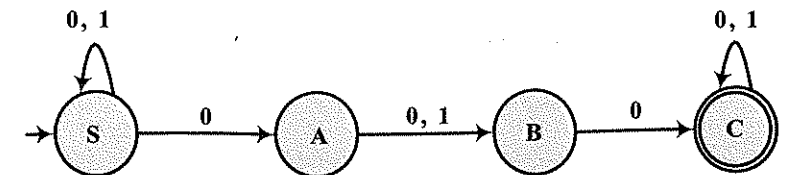
$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow cS \mid dS \mid bB \mid b \mid c$$

$$B \Rightarrow aS \mid bB \mid a \mid d$$

- a) $L(G_{S.3.16})$ 'yı tanıyan NFA'nın (λ -geçişsiz) geçiş çizeneğini bulunuz.
 b) Denklem sistemi yazıp çözerek, $L(G_{S.3.16})$ 'yı tanımlayan bir düzgün deyim bulunuz.

S.3.17.



- Denklemleri yazıp çözmeden, yukarıdaki geçiş çizeneğinin (NFA) tanıdığı düzgün dili bir düzgün deyim olarak yazınız.
- Bu düzgün dili türeten bir düzgün dilbilgisi oluşturunuz.
- Yukarıdaki NFA'ya denk, en az durumlu DFA'nın geçiş çizelgesi ile geçiş çizeneğini bulunuz.

S.3.18. Aşağıdaki düzgün dilbilgisi veriliyor.

$$G_{S.3.18} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{0, 1\}$$

$$P : S \Rightarrow 0S \mid 1A \mid 1 \mid \lambda$$

$$A \Rightarrow 1A \mid 0S \mid 0$$

- $L(G_{S.3.18})$ 'i tanıyan NFA'nın (λ -geçişsiz) geçiş çizeneğini bulunuz.
- Denklemleri yazıp çözerek $L(G_{S.3.18})$ 'i tanımlayan bir düzgün deyim bulunuz.

S.3.19. Aşağıda sözel olarak tanımlanan her düzgün dil için, önce dili tanıyan λ -geçişsiz geçiş çizeneğini oluşturunuz. Sonra da dili türeten sağ-doğrusal bir dilbilgisi tanımlayınız. Tanımladığınız dilbilgisinde gerekiyorsa ($S \Rightarrow \lambda$) kuralı yer alsın. Ancak bunun dışında hiçbir kuralda λ yer almasın.

- $\{0, 1\}$ alfabesinde, içinde ardarda ikiden çok 0 bulunmayan (başka bir deyişle içinde 000 altdizgisi bulunmayan) dizgiler kümesi.
- $\{a, b\}$ alfabesinde, içinde çift sayıda (0, 2, 4, ...) b bulunan dizgiler kümesi.

S.3.20. $L_{S.3.20} = 0^*(1(2+3))^*$

- $L_{S.3.20}$ 'yi tanıyan sonlu özdevinirin (NFA) geçiş çizeneğini bulunuz.
- Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçişsiz denk çizeneği bulunuz.
- $L_{S.3.20}$ 'yi türeten bir düzgün dilbilgisi tanımlayınız.
- $L_{S.3.20}$ 'yi türeten iki değişkenli bir sol-doğrusal dilbilgisi tanımlayınız.

S.3.21. Aşağıdaki düzgün dilbilgisi veriliyor.

$$G_{S.3.21} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{x, y\}$$

$$P : S \Rightarrow xA \mid xS \mid yB \mid y$$

$$A \Rightarrow xS \mid yA$$

$$B \Rightarrow yS \mid xA \mid yA$$

- $L(G_{S.3.21})$ 'i tanıyan NFA'nın (λ -geçişsiz) geçiş çizeneğini bulunuz.
- Denklemleri yazıp çözerek $L(G_{S.3.21})$ 'i tanımlayan bir düzgün deyim bulunuz.

S.3.22. $L_{S.3.22}$ dili aşağıdaki düzgün deyimle tanımlanıyor.

$$L_{S.3.22} = a(bc(d+e^*))^*f$$

- $L_{S.3.22}$ 'yi tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişleri içerebilir.
- Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, λ -geçişlerini yok ederek λ -geçişsiz eşdeğer çizeneği bulunuz.
- $L_{S.3.22}$ 'yi türeten bir dilbilgisi tanımlayınız.

S.3.23. $L_{S.3.23} = 10^*(01^*+1^*0^*01)$

- $L_{S.3.23}$ dilini tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturacağınız geçiş çizeneği λ -geçişleri içermesin.
- $L_{S.3.23}$ dilini türeten bir dilbilgisi tanımlayınız.

S.3.24. Aşağıdaki dilbilgisinin türettiği dili tanımlayan bir düzgün deyim yazınız.

$$G_{S.3.24} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{0, 1\}$$

$$P : S \Rightarrow 0A \mid 1B$$

$$A \Rightarrow 0B \mid 0$$

$$B \Rightarrow 1C \mid 0B \mid 1$$

$$C \Rightarrow 0A$$